



Containerization and DevOps Lab

Submitted by – Ishita Gautam

SAP ID – 500119435

Batch – 1 (CCVT)

Submitted to – Dr. Prateek Raj Gautam

INDEX

S. No.	Experiment
1	Comparison of Virtual Machines (VMs) and Containers
2	Docker Installation, Configuration and Image management
3	Deploying NGINX using different base images and comparing image layers
4	Docker essentials - Dockerfile, .dockerignore, tagging and publishing
5	Docker - Volumes, Environment Variables, Monitoring and Networks
6	Docker Compose vs Docker Run
7	CI/CD Pipeline using Jenkins, GitHub, and Docker Hub
9	Ansible
10	Working with SonarQube
11	Container Orchestration via Docker Swarm
12	Study and Analyse Container Orchestration using Kubernetes

ContainerizationAndDevOps

Experiment - 1: Comparison of Virtual Machines (VMs) and Containers using Ubuntu and Nginx

Objective

1. To understand the conceptual and practical differences between Virtual Machines and Containers.
2. To install and configure a Virtual Machine using VirtualBox and Vagrant on Windows.
3. To install and configure Containers using Docker inside WSL.
4. To deploy an Ubuntu-based Nginx web server in both environments.
5. To compare resource utilization, performance, and operational characteristics of VMs and Containers.

Part A: Virtual Machine (VMs)

- Ensuring the installation via `vagrant -v`
- Firstly download the Vagrantfile of required VM OS using `vagrant init hashicorp/bionic64`
- Run `vagrant up` to start up the VM and `vagrant ssh` to access the VM terminal.

```

PS C:\Users\ishit> vagrant -v
Vagrant 2.4.9
PS C:\Users\ishit> vagrant init hashicorp/bionic64
A 'Vagrantfile' has been placed in this directory. You are now
ready to 'vagrant up' your first virtual environment! Please read
the comments in the Vagrantfile as well as documentation on
'vagrantup.com' for more information on using Vagrant.

PS C:\Users\ishit> vagrant up
Bringing machine 'default' up with 'virtualbox' provider...
==> default: Box 'hashicorp/bionic64' could not be found. Attempting to find and install...
    default: Box Provider: virtualbox
    default: Box Version: >= 0
==> default: Loading metadata for box 'hashicorp/bionic64'
    default: URL: https://vagrantcloud.com/api/v2/vagrant/hashicorp/bionic64
==> default: Adding box 'hashicorp/bionic64' (v1.0.282) for provider: virtualbox
    default: Downloading: https://vagrantcloud.com/hashicorp/boxes/bionic64/versions/1.0.282/p
n/vagrant.box
    default:
==> default: Successfully added box 'hashicorp/bionic64' (v1.0.282) for 'virtualbox'!
==> default: Importing base box 'hashicorp/bionic64'...
==> default: Matching MAC address for NAT networking...
==> default: Checking if box 'hashicorp/bionic64' version '1.0.282' is up to date...
    default: Setting the name of the VM: ishita-fc3e-1560826664057-0050

PS C:\Users\ishit> vagrant ssh
Welcome to Ubuntu 18.04.3 LTS (GNU/Linux 4.15.0-58-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Sat Jan 31 05:21:14 UTC 2026

System load:  0.13               Processes:           91
Usage of /:   2.5% of 61.80GB    Users logged in:    0
Memory usage: 11%               IP address for eth0: 10.0.2.15
Swap usage:   0%

0 packages can be updated.
0 updates are security updates.

vagrant@vagrant:~$ |

```

- Now installing nginx using `sudo apt install -y nginx` and running the service using `sudo systemctl start nginx`.
- Verifying it by `curl localhost`.


```
vagrant@vagrant:~$ sudo apt update && sudo apt install -y nginx
Hit:1 http://archive.ubuntu.com/ubuntu bionic InRelease
Get:2 http://archive.ubuntu.com/ubuntu bionic-updates InRelease [102 kB]
Get:3 http://security.ubuntu.com/ubuntu bionic-security InRelease [102 kB]
Get:4 http://archive.ubuntu.com/ubuntu bionic-backports InRelease [102 kB]
Get:5 http://archive.ubuntu.com/ubuntu bionic-updates/main amd64 Packages [3,045 kB]
Get:6 http://security.ubuntu.com/ubuntu bionic-security/main i386 Packages [1,380 kB]
Get:7 http://archive.ubuntu.com/ubuntu bionic-updates/main i386 Packages [1,666 kB]
Get:8 http://archive.ubuntu.com/ubuntu bionic-updates/main Translation-en [554 kB]
Get:9 http://archive.ubuntu.com/ubuntu bionic-updates/restricted amd64 Packages [1,380 kB]
Get:10 http://archive.ubuntu.com/ubuntu bionic-updates/restricted i386 Packages [391 kB]
Get:11 http://archive.ubuntu.com/ubuntu bionic-updates/restricted Translation-en [102 kB]
Fetched 14.5 MB in 10s (1,450 kB/s)
Setting up libx11-data (2:1.6.4-3ubuntu0.4) ...
Setting up libwebp6:amd64 (0.6.1-2ubuntu0.18.04.2) ...
Setting up libjpeg8:amd64 (8c-2ubuntu8) ...
Setting up fontconfig-config (2.12.6-0ubuntu2) ...
Setting up libnginx-mod-stream (1.14.0-0ubuntu1.11) ...
Setting up libx11-6:amd64 (2:1.6.4-3ubuntu0.4) ...
Setting up libtiff5:amd64 (4.0.9-5ubuntu0.10) ...
Setting up libxpm4:amd64 (1:3.5.12-1ubuntu0.18.04.2) ...
Setting up libfontconfig1:amd64 (2.12.6-0ubuntu2) ...
Setting up libgd3:amd64 (2.2.5-4ubuntu0.5) ...
Setting up libnginx-mod-http-image-filter (1.14.0-0ubuntu1.11) ...
Setting up nginx-core (1.14.0-0ubuntu1.11) ...
Setting up nginx (1.14.0-0ubuntu1.11) ...
Processing triggers for ureadahead (0.100.0-21) ...
Processing triggers for ufw (0.36-0ubuntu0.18.04.1) ...
Processing triggers for libc-bin (2.27-3ubuntu1) ...
vagrant@vagrant:~$ |
```

```

vagrant@vagrant:~$ sudo systemctl start nginx
vagrant@vagrant:~$ curl localhost
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
    body {
        width: 35em;
        margin: 0 auto;
        font-family: Tahoma, Verdana, Arial, sans-serif;
    }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
vagrant@vagrant:~$

```

Observations

1. Storage Utilization: The disk usage for the VM.

VBoxManage showmediuminfo <path/to/virtualdrive>

```

PS C:\Users\ishit> cd "C:\Program Files\Oracle\VirtualBox"
PS C:\Program Files\Oracle\VirtualBox> .\VBoxManage.exe showmediuminfo "C:\Users\ishit\VirtualBox VMs\ishit_default_1769836664957_4854\ubuntu-18.04-amd64-disk001.vmdk"
UUID:          e880833b-d649-44b7-811e-2424ded70322
Parent UUID:   base
State:         locked write
Type:          normal (base)
Location:      C:\Users\ishit\VirtualBox VMs\ishit_default_1769836664957_4854\ubuntu-18.04-amd64-disk001.vmdk
Storage format: VMDK
Format variant: dynamic default
Capacity:      65536 MBytes
Size on disk:  1773 MBytes
Encryption:    disabled
Property:      BootSector=
               Partitions=
               RawDrive=
               Relative=
In use by VMs:  ishit_default_1769836664957_4854 (UUID: a9d208d1-5255-4e59-a55e-91e43529d80e)
PS C:\Program Files\Oracle\VirtualBox>

```

The VM installation consumed approximately **1773 MB** of disk space to store the Guest OS and virtual disk.

2. Boot Performance: The startup time required for the Virtual Machine to boot.

systemd-analyze

```
vagrant@vagrant:~$ systemd-analyze
Startup finished in 10.298s (kernel) + 1min 10.297s (userspace) = 1min 20.596s
graphical.target reached after 13.897s in userspace
vagrant@vagrant:~$
```

The VM took **1 min 20.596 seconds** to fully boot (Kernel: 10.298s + Userspace: 1min 10.297s), which basically depicts high startup latency.

3. Memory Usage: Amount of RAM resources allocated to and used by the Guest OS.

free -h

```
vagrant@vagrant:~$ free -h
```

	total	used	free	shared	buff/cache	available
Mem:	985M	72M	246M	5.0M	666M	763M
Swap:	979M	0B	979M			

The VM reserved **985 MB** of total RAM from the host, with **72 MB** actively used and **666 MB** used up in buffers/cache.

Part B: Containers

- Run the `docker run -d -p 8080:80 --name nginx-container nginx` command to create and run the container. Verify that the nginx container is running with `curl localhost:8080`.
 - `-d` : Runs the container in the background, allowing you to continue using the terminal session.
 - `-p 8080:80` : Maps port 8080 for the host to port 80 for the container.

```
ishita@Ish: ~$ sudo docker run -d -p 8080:80 --name nginx-container nginx
[sudo] password for ishita:
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
500799c30424: Pull complete
eaf8753feae0: Pull complete
10b68cfefee1: Pull complete
57f0dd1befe2: Pull complete
119d43eec815: Pull complete
d989100b8a84: Pull complete
700146c8ad64: Pull complete
e2dd2dbe6277: Download complete
785250c9bf9e: Download complete
Digest: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4
aec
Status: Downloaded newer image for nginx:latest
ed41bf12493ab50c54129e154b708dbda9eedd4b3e03293b501595da5b0fa0a4
ishita@Ish:~$ curl localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

Observations

1. Storage Utilization An analysis of the disk space required for the container images.

docker images


```
ishita@Ish:~$ sudo docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	Info →	U In Use EXTRA
hello-world:latest	05813aedc15f	25.9kB	9.52kB		
ishcosmo/javaapp:2.0	20768f3a5436	1.07GB	324MB		U
java_img_ish:latest	235209769f49	1.07GB	324MB		U
javaapp:2.0	20768f3a5436	1.07GB	324MB		U
nginx:latest	c881927c4077	240MB	65.7MB		U
ubuntu:latest	cd1dba651b30	119MB	31.7MB		U

```
ishita@Ish:~$
```

The Nginx container image requires only **240 MB**.

2. Boot Performance Measurement of the time required to start the containerized application.

```
time docker run -d -p 8080:80 --name nginx-container nginx
```

```
ishita@Ish:~$ time sudo docker run -d -p 8080:80 --name nginx-container nginx
x
9c261b8151601398f178cdc7553c17314ff759ea1675f5fe7689d6d978f559d5

real    0m0.655s
user    0m0.004s
sys     0m0.008s
```

The container started in **0.08 seconds**.

3. Memory Allocation Real-time monitoring of the container's resource consumption.

```
docker stats
```

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
9c261b815160	nginx-container	0.00%	14.5MiB / 7.594GiB	0.19%	1.17kB / 126B	4.19MB / 12.3kB	17

The running container uses **14.5 MiB** of RAM.

Comparison

Metric	Virtual Machine	Container	Difference
Disk Usage	~1.8 GB	240 MB	Containers are <i>smaller</i> because they do not require a full Guest OS copy.
Boot Time	1.20 min	0.08s	Containers start <i>faster</i> because of the already-running Host

Metric	Virtual Machine	Container	Difference
			Kernel.
Memory	~985 MB Reserved / 72 MB Used	14.5 MB Used	VMs reserve a fix block of RAM; Containers use only whats needed (dynamic)
Architecture	Hardware Virtualization (Guest OS)	OS Virtualization (Shared Kernel)	VMs run a full isolated OS ; Containers run as isolated processes on the Host OS.

ContainerizationAndDevOps

Experiment 2: Docker Installation, Configuration and Image Management

Objective:

1. Pull docker images
2. Run Containers
3. Manage container lifecycle

Implementation

1. Pull Image using the `docker pull nginx`
 - This pulls the latest Nginx image from the Docker hub

```
ishita@Ish:~/ContainerizationAndDevOps$ sudo docker pull nginx
[sudo] password for ishita:
Using default tag: latest
latest: Pulling from library/nginx
Digest: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Status: Image is up to date for nginx:latest
docker.io/library/nginx:latest
ishita@Ish:~/ContainerizationAndDevOps$
```

1. Running the container with port mapping using the command `docker run -d -p 8080:80 nginx`
 - It actually starts Nginx in the background mapped to port 8080

```
ishita@Ish:~/ContainerizationAndDevOps$ sudo docker run -d -p 8080:80 --name exp2-nginx nginx
2aaeb84aff5d7c438e90f6387c4b53b6bba2125f198d1b26ec88f98bd43b8d93
ishita@Ish:~/ContainerizationAndDevOps$
```

1. Verifying using the `docker ps` command

```
ishita@Ish:~/ContainerizationAndDevOps$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
2aaeb84aff5d	nginx	"/docker-entrypoint..."	About a minute ago	Up About a minute	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	exp2-nginx

```
ishita@Ish:~/ContainerizationAndDevOps$
```

1. Stopping the running container using `docker stop` command


```
ishita@Ish:~/ContainerizationAndDevOps$ sudo docker stop exp2-nginx
exp2-nginx
ishita@Ish:~/ContainerizationAndDevOps$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
ishita@Ish:~/ContainerizationAndDevOps$
```

- Its clear from the `docker ps` command that there are no running containers

1. The `docker rm` command deletes the container

```
ishita@Ish:~/ContainerizationAndDevOps$ sudo docker rm exp2-nginx
exp2-nginx
```

1. The `docker rmi` command is used to delete the image from my local device

```
Untagged: nginx:latest
Deleted: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
```

Result: Docker image pulled successfully and necessary commands were performed post container execution

ContainerizationAndDevOps

Experiment 3: Deploying NGINX using different base images and comparing image layers

Objective:

1. Deploy NGINX using
 - Official nginx image
 - Ubuntu-based image
 - Alpine-based image
2. Understand Docker images layers and size and differences
3. Compare performance, security and use-cases of each approach

Implementation

Part 1: Deploying Nginx using official image

1. Pull Image using the `docker pull nginx:latest`
 - This pulls the latest Nginx image from the Docker hub

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment3$ sudo docker pull nginx:latest
[sudo] password for ishita:
latest: Pulling from library/nginx
47cd406a84ef: Pull complete
46bf3a120c8e: Pull complete
f73400a233fd: Pull complete
bae5a1799a80: Pull complete
7b6cb8ccac7b: Pull complete
4f4efe02d542: Pull complete
2e02dba24409: Download complete
a5d78d617315: Download complete
Digest: sha256:344bf0f3ee6e5277d6002ef6e1fb02106e4252ed34ebc0e262e0056d312200
```

1. Running the container with port mapping using the command `docker run -d --name nginx-official -p 8080:80 nginx`

- It actually starts Nginx in the background mapped to port 8080

```
docker:10/library/nginx:latest
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment3$ docker run -d --name nginx-official -p 8080:80 nginx
5973df2ea9a007e768f362b6a653d655cab9d186a815c8ae3a667e606450fe97
```

1. Verifying using the `curl http://localhost:8080` command

- o NGINX welcome page is being displayed

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment3$ curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
```

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

2. Using `docker images nginx` command the official NGINX image is displayed:

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment3$ docker images nginx
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
nginx:latest	341bf0f3ce6c	240MB	65.8MB	U

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment3$
```

Part 2: Custom NGINX Using Ubuntu base image..

1. Creating dockerfile in the current working directory ``Dockerfile FROM ubuntu:22.04

RUN apt-get update &&

apt-get install -y nginx &&

```
apt-get clean &&
rm -rf /var/lib/apt/lists/*
```

EXPOSE 80

```
CMD ["nginx", "-g", "daemon off;"]
```

2. Building image using ``docker build -t nginx-ubuntu .``

3. Running the container using ``docker run -d --name nginx-ubuntu -p 8081:80 nginx-ubuntu`` cc

- Assumption: index.html is stored in `***./nginx-html***`

4. Gathering observations for this variant by the ``docker images nginx-ubuntu`` command

Part 3: Custom NGINX Using Alpine base image

1. Creating dockerfile in the current working directory

```
```Dockerfile
```

```
FROM alpine:latest
```

```
RUN apk add --no-cache nginx
```

```
RUN echo "server { \
 listen 80 default_server; \
 listen [::]:80 default_server; \
 root /var/www/html; \
 index index.html index.htm; \
 server_name _; \
 location / { \
 try_files $uri $uri/ =404; \
 } \
}" > /etc/nginx/http.d/default.conf
```

```
EXPOSE 80
```

```
CMD ["nginx", "-g", "daemon off;"]
```

\*\*\*NOTE:\*\*\* \*The default config of NGINX on Alpine is set to display a 404 page.\*

Build and run the nginx-alpine image (running on port 8082):

```
```Bash
```

```
docker build -t nginx-alpine -f ./alpine.Dockerfile .
```

```
docker run -d -v ./nginx-html:/var/www/html -p 8082:80 --name nginx-alpine-test nginx-alpine
```

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment3$ docker build -t nginx-alpine -f ./alpine.Dockerfile .
[+] Building 8.2s (8/8) FINISHED
=> [internal] load build definition from alpine.Dockerfile
=> => transferring dockerfile: 411B
=> [internal] load metadata for docker.io/library/alpine:latest
=> [auth] library/alpine:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
```

The built Alpine NGINX image:

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment3$ docker run -d -v ./nginx-html:/var/www/html -p 8082:80 --name nginx-alpine-test nginx-alpine:latest
bc57cc5aaf1a61793c1a7d0a04ffd12fefa35579bf58c6e6848b211a3c06453a

ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment3$ docker images nginx-alpine
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
nginx-alpine:latest	8667fbf99471	16.1MB	4.74MB	U

Part 4: Image Size and Layer Comparison

Compare Sizes

```
docker images | grep nginx
```

```
nginx-alpine:latest      8667fbf99471      16.1MB      4.74MB      U
nginx-ubuntu:latest     d5bb9c86dd63      199MB       52.7MB      U
nginx:latest            341bf0f3ce6c      240MB       65.8MB      U
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment3$
```

Image	Size
nginx	240MB
nginx-ubuntu	199MB
nginx-alpine	16.1MB

Comapare Layers

```
docker history nginx
```



```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment3$ docker history nginx
```

IMAGE	CREATED	CREATED BY	SIZE	COMMENT
341bf0f3ce6c	5 days ago	CMD ["nginx" "-g" "daemon off;"]	0B	buildkit.dockerfile.v0
<missing>	5 days ago	STOPSIGNAL SIGQUIT	0B	buildkit.dockerfile.v0
<missing>	5 days ago	EXPOSE map[80/tcp:{}]	0B	buildkit.dockerfile.v0
<missing>	5 days ago	ENTRYPOINT ["/docker-entrypoint.sh"]	0B	buildkit.dockerfile.v0
<missing>	5 days ago	COPY 30-tune-worker-processes.sh /docker-ent...	16.4kB	buildkit.dockerfile.v0
<missing>	5 days ago	COPY 20-envsubst-on-templates.sh /docker-ent...	12.3kB	buildkit.dockerfile.v0
<missing>	5 days ago	COPY 15-local-resolvers.envsh /docker-entryp...	12.3kB	buildkit.dockerfile.v0
<missing>	5 days ago	COPY 10-listen-on-ipv6-by-default.sh /docker...	12.3kB	buildkit.dockerfile.v0

```
docker history nginx-ubuntu
```

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment3$ docker history nginx-ubuntu
```

IMAGE	CREATED	CREATED BY	SIZE	COMMENT
d5bb9c86dd63	2 days ago	CMD ["nginx" "-g" "daemon off;"]	0B	buildkit.dockerfile.v0
<missing>	2 days ago	EXPOSE [80/tcp]	0B	buildkit.dockerfile.v0
<missing>	2 days ago	RUN /bin/sh -c apt-get update && apt-get...	58.6MB	buildkit.dockerfile.v0
<missing>	4 weeks ago	/bin/sh -c #(nop) CMD ["/bin/bash"]	0B	
<missing>	4 weeks ago	/bin/sh -c #(nop) ADD file:b499000226bd9a7c5...	87.5MB	
<missing>	4 weeks ago	/bin/sh -c #(nop) LABEL org.opencontainers...	0B	
<missing>	4 weeks ago	/bin/sh -c #(nop) LABEL org.opencontainers...	0B	
<missing>	4 weeks ago	/bin/sh -c #(nop) ARG LAUNCHPAD_BUILD_ARCH	0B	

```
docker history nginx-alpine
```

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment3$ docker history nginx-alpine
```

IMAGE	CREATED	CREATED BY	SIZE	COMMENT
8667fbf99471	7 minutes ago	CMD ["nginx" "-g" "daemon off;"]	0B	buildkit.dockerfile.v0
<missing>	7 minutes ago	EXPOSE [80/tcp]	0B	buildkit.dockerfile.v0
<missing>	7 minutes ago	RUN /bin/sh -c echo "server { listen 80 ...	20.5kB	buildkit.dockerfile.v0
<missing>	7 minutes ago	RUN /bin/sh -c apk add --no-cache nginx # bu...	2.2MB	buildkit.dockerfile.v0
<missing>	13 days ago	CMD ["/bin/sh"]	0B	buildkit.dockerfile.v0
<missing>	13 days ago	ADD alpine-minirootfs-3.23.3-x86_64.tar.gz /...	9.11MB	buildkit.dockerfile.v0

Observations

- Size of official NGINX image is the largest (240MB).
- nginx-alpine is the smallest (16.1MB).
- Official nginx image has the most layers.
- nginx-alpine has the least layers.

Part 5: Functional Tasks Using NGINX

Serving custom HTML page

```
./nginx-html/index.html
```

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Student Information</title>
  <style>
    body {
      display: flex;
      justify-content: center;
      padding-top: 50px;
    }
    th, td {
      border: 1px solid #dddddd;
      text-align: left;
      padding: 12px;
    }
  </style>
</head>
<body>

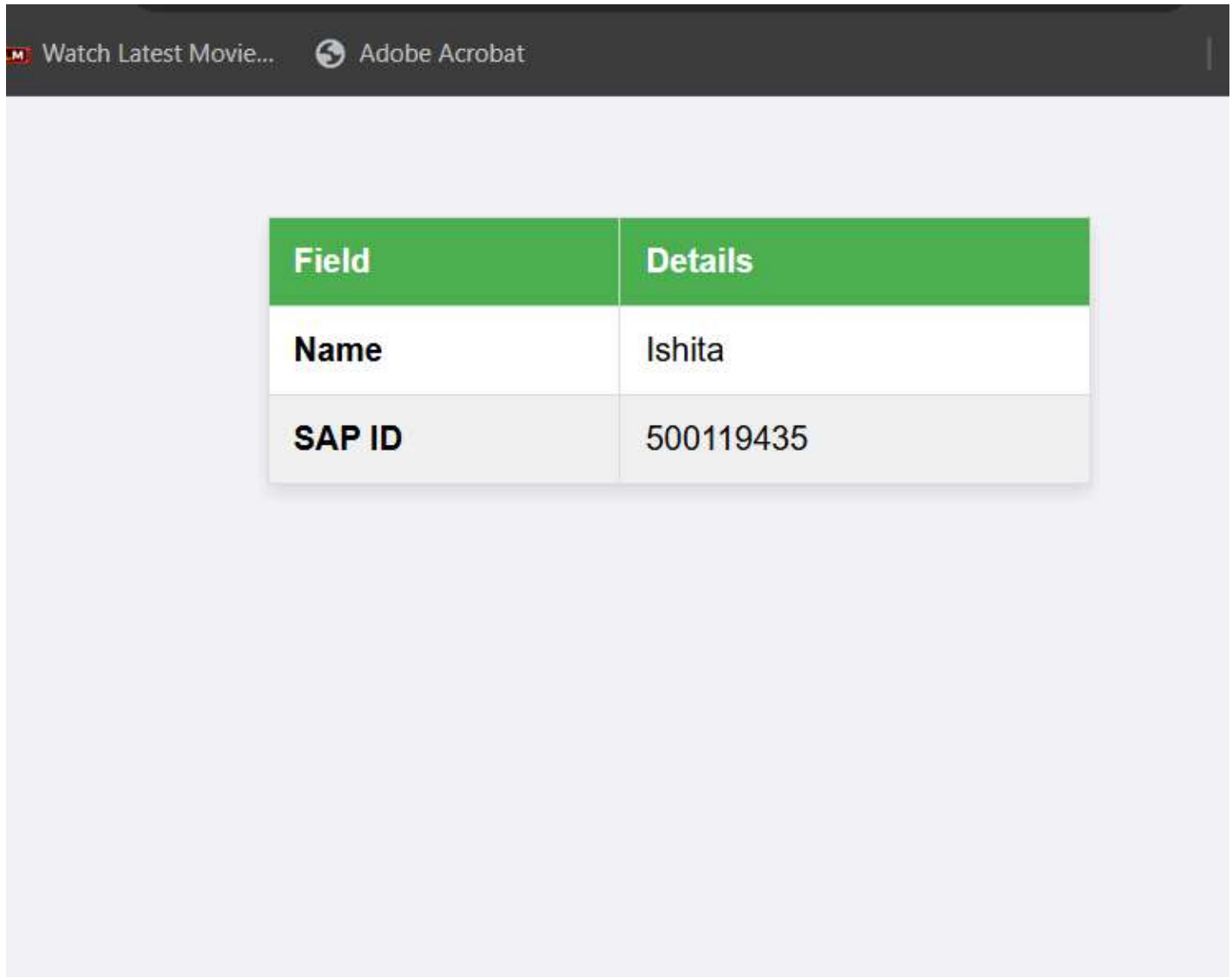
<table>
  <thead>
    <tr>
      <th>Field</th>
      <th>Details</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td><strong>Name</strong></td>
      <td>Ishita</td>
    </tr>
    <tr>
      <td><strong>SAP ID</strong></td>
      <td>500119435</td>
    </tr>
  </tbody>
</table>

</body>
</html>
```

Running on official NGINX image:

```
docker run -d \
  -p 8080:80 \
```

```
-v ./nginx-html:/usr/share/nginx/html \
nginx
```



Lab Report: Lab-3 Part-2: Dockerfile Based Flask Web App Containerization

1. Objective

- Create and run a Flask web application locally.
- Define dependencies in `requirements.txt`.
- Build a Docker image using a `Dockerfile`.
- Deploy and verify the Flask app inside a Docker container.
- Contents of `app.py` ``python from flask import Flask app = Flask(name)

```
@app.route('/') def home(): return "<h1>NAME: Ishita</h1><p>SAP ID: 500119435</p><p>Lab: Containerization & DevOps</p>"
```

```
if name == "main": app.run(host='0.0.0.0', port=5000)
```

- Contents of *requirements.txt* file

```
```text
```

```
flask==2.2.2
```

- Contents of *Dockerfile.py*

```
FROM python:3.9-slim
WORKDIR /app
COPY . /app
RUN pip install --no-cache-dir -r requirements.txt
EXPOSE 5000
CMD ["python", "app.py"]
```

i. For Locally installing

- `pip install -r requirements.txt`

# ContainerizationAndDevOps

---

## Docker essentials - Dockerfile, .dockerignore, tagging and publishing

---

### Objectives- Learn and implement docker essentials

---

### Part 1: Containerizing Apps with Dockerfiles

---

1. Python flask app, which I named "app.py" under the **ish-flask-app** directory

Commands- `mkdir ish-flask-app`

- Content of the **app.py** file `python from flask import Flask app = Flask(name)`

```
@app.route('/') def hello(): return "Hello from Docker!"
```

```
@app.route('/health') def health(): return "OK"
```

```
if name == 'main': app.run(host='0.0.0.0', port=5000)
```

```
- In the requirements.txt file - Flask==2.3.3
```

2. Creating a dockerfile with content:

```
``` Dockerfile
```

```
# Use Python base image
```

```
FROM python:3.9-slim
```

```
# Set working directory
```

```
WORKDIR /app
```

```
# Copy requirements file
```

```
COPY requirements.txt .
```

```
# Install dependencies
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
# Copy application code
```

```
COPY app.py .
```



```
# Expose port
EXPOSE 5000

# Run the application
CMD ["python", "app.py"]
```

```
ishita@Ish:~/ContainerizationAndDevOps$ cd ./LAB/Experiment4
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4$ mkdir ish-flask-app
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4$ cd ish-flask-app
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$ docker build -t ish-flask-app .
[+] Building 44.2s (11/11) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.1s
=> => transferring dockerfile: 361B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim  5.4s
=> [auth] library/python:pull token for registry-1.docker.io       0.0s
=> [internal] load .dockerignore                                   0.1s
=> => transferring context: 273B                                    0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f75 31.2s
```

Part2: Using .dockerignore

- a `.dockerignore` file is nothing but a plain configuration file that helps in reducing image size as it prevents unnecessary files from being copied

Part3: Building Docker Images

1. Building image using the command `docker build -t ish-flask-app .` and verifying using the `'docker images'` command
- build screenshot as shown above and verification as below

ish-flask-app:latest	df8b1ae85914	200MB	48.8MB
----------------------	--------------	-------	--------

1. Tagging images

- version number: `docker build -t ish-flask-app:1.0 .`

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$ docker build -t ish-flask-app:1.0 .
[+] Building 3.2s (11/11) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile              0.0s
=> => transferring dockerfile: 361B                             0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 2.7s
=> [auth] library/python:pull token for registry-1.docker.io    0.0s
=> [internal] load .dockerignore                                0.0s
=> => transferring context: 273B                                 0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599 0.0s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599 0.0s
=> [internal] load build context                                0.0s
=> => transferring context: 63B                                  0.0s
```

- with multiple tags: `docker build -t ish-flask-app:latest -t ish-flask-app:1.0 .`

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$ docker build -t ish-flask-app:latest -t ish-flask-app:1.0 .
[+] Building 2.0s (10/10) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile              0.0s
=> => transferring dockerfile: 361B                             0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 1.4s
=> [internal] load .dockerignore                                0.0s
=> => transferring context: 273B                                 0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599 0.0s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599 0.0s
=> [internal] load build context                                0.0s
```

- with custom directory: `docker build -t ishcosmo/ish-flask-app:1.0 .`

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$ docker build -t ishcosmo/ish-flask-app:1.0 .
[+] Building 2.6s (11/11) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile              0.0s
=> => transferring dockerfile: 361B                             0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 2.1s
=> [auth] library/python:pull token for registry-1.docker.io    0.0s
=> [internal] load .dockerignore                                0.0s
=> => transferring context: 273B                                 0.0s
```

- tagging an existing image: `docker tag ish-flask-app:latest ish-flask-app:v1.0`

1. Viewing image details

- using `docker images`

ish-flask-app:1.0	ba821985af05	200MB	48.8MB
ish-flask-app:latest	ba821985af05	200MB	48.8MB
ish-flask-app:v1.0	ba821985af05	200MB	48.8MB
ishcosmo/ish-flask-app:1.0	19afe077783e	200MB	48.8MB

- using `docker history ish-flask-app`

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$ docker history ish-flask-app
IMAGE          CREATED          CREATED BY          SIZE            COMMENT
ba821985af05   33 minutes ago   CMD ["python" "app.py"]  0B              buildkit.dockerf
ile.v0
<missing>      33 minutes ago   EXPOSE [5000/tcp]      0B              buildkit.dockerf
ile.v0
<missing>      33 minutes ago   COPY app.py . # buildkit 12.3kB          buildkit.dockerf
ile.v0
<missing>      33 minutes ago   RUN /bin/sh -c pip install --no-cache-dir -r... 13.4MB          buildkit.dockerf
ile.v0
<missing>      33 minutes ago   COPY requirements.txt . # buildkit 12.3kB          buildkit.dockerf
ile.v0
```

- inspecting using `docker inspect ish-flask-app`

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$ docker inspect ish-flask-app
[
  {
    "Id": "sha256:ba821985af05fcfcaeb5b063806afd36fcd23ae85ffc835add6d2e8030e63c5d",
    "RepoTags": [
      "ish-flask-app:1.0",
      "ish-flask-app:latest",
      "ish-flask-app:v1.0"
    ],
    "RepoDigests": [
      "ish-flask-app@sha256:ba821985af05fcfcaeb5b063806afd36fcd23ae85ffc835add6d2e8030e63c5d"
    ],
    "Comment": "buildkit.dockerfile.v0",
    "Created": "2026-02-21T08:56:55.587510695Z",
    "Config": {
      "ExposedPorts": {
        "5000/tcp": {}
      },
      "Env": [
        "PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
        "LANG=C.UTF-8",
        "GPG_KEY=E3FF2839C048B25C084DEBE9B26995E310250568",
        "PYTHON_VERSION=3.9.25",
        "PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee84d2a40dae404a99303e3f97701c10966c91834"
      ]
    }
  }
]
```

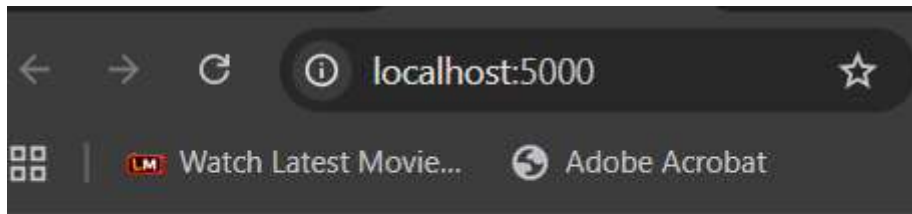
Part4: Running containers

1. running container with port mapping using `docker run -d -p 5000:5000 --name flask-container ish-flask-app`

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$ docker run -d -p 5000:5000 --name flask-container ish-flask-app
8f9485ccdc63e76a449160485324429e35fb20c78202adb163b6585046a75d55
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$
```

1. Testing the application using `curl http://localhost:5000`

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$ curl http://localhost:5000
Hello from Docker!ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$
```



Hello from Docker!

2. checking running containers using `docker ps`

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$ curl http://localhost:5000
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
8f9485ccdc63   ish-flask-app  "python app.py"         4 minutes ago Up 4 minutes  0.0.0.0:5000->5000/tcp, [::]:5000->
p flask-container
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$
```

3. viewing container logs using `docker logs flask-container`

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$ docker logs flask-container
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
```

4. Stopping the container using `docker stop flask-container` and starting using `docker start flask-container`

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$ docker stop flask-container
flask-container
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$ docker start flask-container
flask-container
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$
```

5. To remove a container use the command `docker rm flask-container` and to forcefully remove it use `docker rm -f flask-container`

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$ docker rm -f flask-container
flask-container
```

Part 5: Multi-stage Builds

- Another dockerfile needs to be built

```
# STAGE 1: Builder stage
FROM python:3.9-slim AS builder

WORKDIR /app

# Copy requirements
COPY requirements.txt .

# Install dependencies in virtual environment
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
RUN pip install --no-cache-dir -r requirements.txt

# STAGE 2: Runtime stage
FROM python:3.9-slim

WORKDIR /app

# Copy virtual environment from builder
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Copy application code
COPY app.py .

# Create non-root user
RUN useradd -m -u 1000 appuser
USER appuser

# Expose port
EXPOSE 5000

# Run application
CMD ["python", "app.py"]
```

- Building and comparing

- a regular image can be built using the command `docker build -t xyz .`

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$ docker build -t flask-regular .
[+] Building 1.7s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 361B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [internal] load .dockerignore
=> => transferring context: 273B
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:12d07f56010b16bd228d7060f361f52f144065f7555003eb13ed19a
```

- a multistage image can be built using the command `docker build -f Dockerfile.multistage -t xyz .`

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$ docker build -f Dockerfile.multistage -t flask-regular .
[+] Building 10.3s (13/13) FINISHED
=> [internal] load build definition from Dockerfile.multistage
=> => transferring dockerfile: 675B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [internal] load .dockerignore
```

- comparing the sizes using command `docker images | grep flask-`

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$ docker images | grep flask-
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
flask-multistage:latest      361b046df13a      86.1MB      21MB
flask-regular:latest        b6b4aeda9bf6      200MB      48.8MB
```

- Clearly a significant difference in image sizes are observed (86 MB for multi and 200 for single)

PART 6: Publishing to docker hub

- My username is ishcosmo
- Logging in using the command `docker login`

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$ docker login
Authenticating with existing credentials... [Username: ishcosmo]

Info → To login with a different account, run 'docker logout' followed by 'docker login'

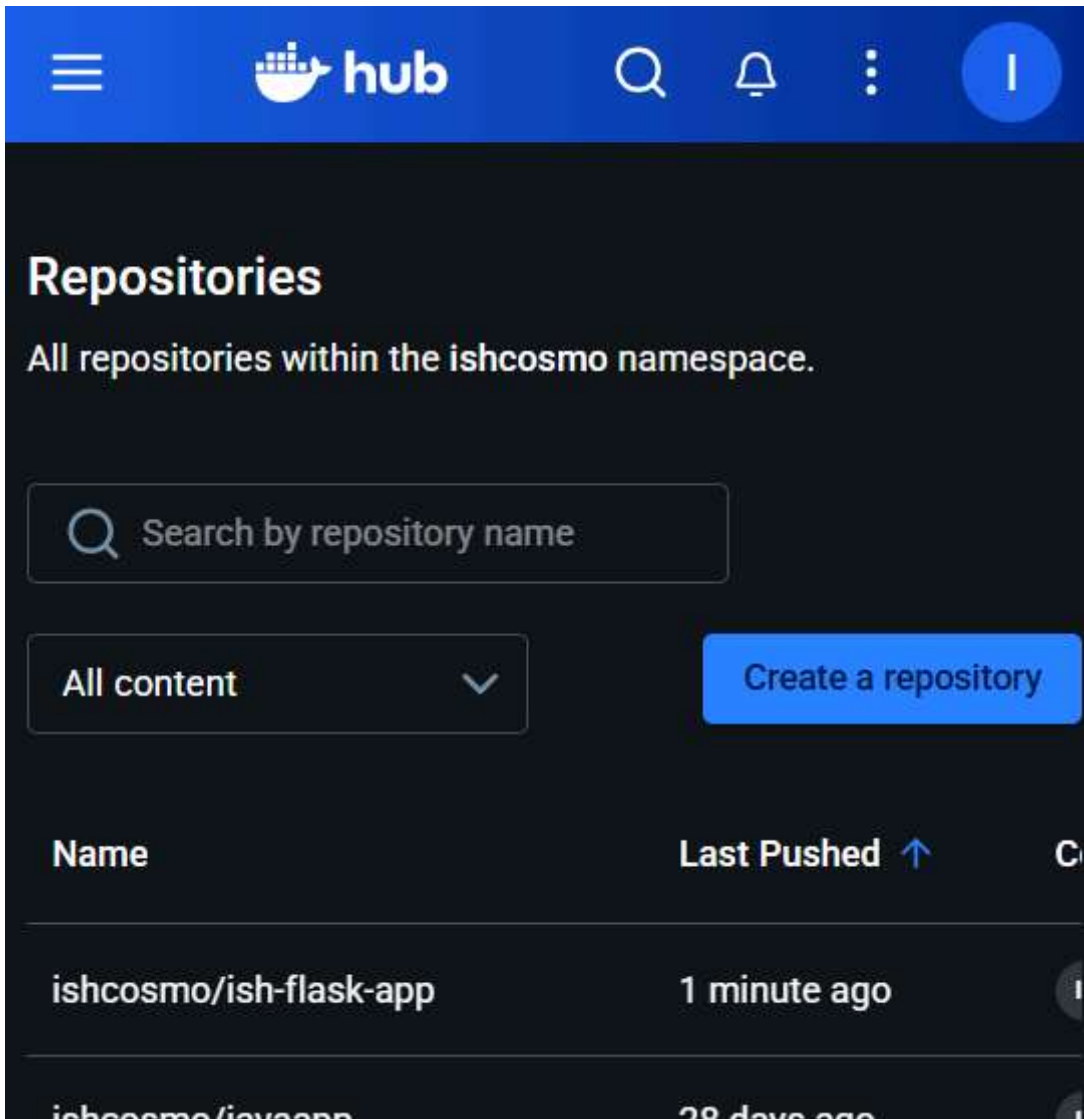
Login Succeeded
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$
```

- Tagging image for docker hub via `docker tag ish-flask-app:latest ishcosmo/ish-flask-app:1.0` and `docker tag ish-flask-app:latest ishcosmo/ish-flask-app:latest`

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$ docker tag ish-flask-app:latest ishcosmo/ish-flask-app:1.0
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$ docker tag ish-flask-app:latest ishcosmo/ish-flask-app:latest
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$
```

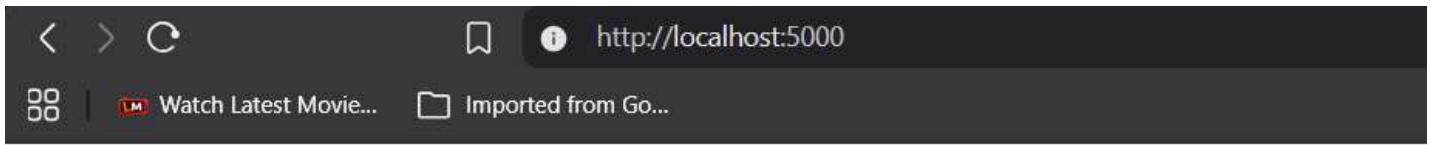
- Pushing to docker hub using `docker push` command

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$ docker push ishcosmo/ish-flask-app:1.0
The push refers to repository [docker.io/ishcosmo/ish-flask-app]
a3ce5b2f15ad: Pushed
ea56f685404a: Pushed
c43bff7a2963: Pushed
fd308cfa8a: Pushed
fc7443084902: Pushed
cffe2f6351e4: Pushed
38513bd72563: Pushed
b3ec39b36ae8: Pushed
c520cc56a573: Pushed
1.0: digest: sha256:ba821985af05fcfaeb5b063806afd36fcd23ae85ffc835add6d2e8030e63c5d size: 856
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$ docker push ishcosmo/ish-flask-app:latest
The push refers to repository [docker.io/ishcosmo/ish-flask-app]
```



- Pulling from docker hub using `docker pull` running using the `docker run` command and finally verifying using the `docker ps` command

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$ docker pull ishcosmo/ish-flask-app:latest
latest: Pulling from ishcosmo/ish-flask-app
Digest: sha256:ba821985af05fcfaeb5b063806afd36fcd23ae85ffc835add6d2e8030e63c5d
Status: Image is up to date for ishcosmo/ish-flask-app:latest
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$ docker run -d -p 5000:5000 ishcosmo/ish-flask-app:latest
e2dec2279137657335a9d673674892d1b3e56a918771f223a0a0d0763c6e6fe8
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED          STATUS          PORTS                               NAMES
e2dec2279137   ishcosmo/ish-flask-app:latest       "python app.py"         About a minute   Up About a minute   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   hungry_borg
```



Hello from Docker!

Part 7: Node.js Example

- Firstly a new directory that i named `ish-node-app` is created .

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-flask-app$ cd ..  
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4$ mkdir ish-node-app  
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4$ cd ish-node-app
```

with a `app.js` and `package.json` file along with the necessary `Dockerfile` with the following content:



- `app.js` ``js const express = require('express'); const app = express(); const port = 3000;

```
app.get('/', (req, res) => { res.send('Hello from Node.js Docker!'); });
```

```
app.get('/health', (req, res) => { res.json({ status: 'healthy' }); });
```

```
app.listen(port, () => { console.log( Server running on port ${port} ); });
```

```
- package.json  
``json  
{  
  "name": "node-docker-app",  
  "version": "1.0.0",  
  "main": "app.js",  
  "dependencies": {  
    "express": "^4.18.2"  
  }  
}
```

```
}

```

- Dockerfile ``Dockerfile FROM node:18-alpine

WORKDIR /app

COPY package*.json ./ RUN npm install --only=production

COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]

```
- Building the image using `docker build -t ish-node-app .`

```

```
- Running the image using `docker run -d -p 3000:3000 --name node-container ish-node-app`
```

```
- testing the image using `curl http://localhost:3000`

```

```

```

PART 8: Exercises (practice)

1. To Create an image with three tags:

- a. myapp:latest
 - b. myapp:v2.0
 - c. yourusername/myapp:production
 - command used `docker build -t myapp:latest -t myapp:v2.0 -t ishcosmo/ish-flask-app:producti
- ```

```

- what happens is it is a way to build a single Docker image and assign it multiple versions
- Instead of running the build process three separate times, Docker builds the image once and

```

```

- clearly the 3 tags built have the same image id

### 2. Create a multi-stage Dockerfile for Node.js that:

- a. Uses builder stage for npm install
- b. Creates final image with only production dependencies
- c. Uses non-root user



```

- buiding the dockerfile as below
```Dockerfile
# STAGE 1: Builder stage
FROM node:18-slim AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install

# STAGE 2: Production stage
FROM node:18-alpine
WORKDIR /app
# Copy only package files for a clean production install
COPY --from=builder /app/package*.json ./
RUN npm install --only=production
# Copy application source code
COPY . .
# Create and switch to non-root user for security
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser
EXPOSE 3000
CMD ["node", "app.js"]

```

- Building by using `docker build -f Dockerfile.multi -t ish-node-app:latest -t ish-node-app:v2.0 -t ishcosmo/ish-node-app:production .` the concpet of triple tags is applied here

```

ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4$ cd ./ish-node-app
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-node-app$ docker build -f Dockerfile.multi -t ish-node-app:latest -t ish-node-app:v2.0 -t ishcosmo/ish-node-app:production .
[+] Building 25.9s (17/17) FINISHED
=> [internal] load build definition from Dockerfile.multi
=> => transferring dockerfile: 542B
=> [internal] load metadata for docker.io/library/node:18-alpine
=> [internal] load metadata for docker.io/library/node:18-slim

```

ish-node-app:latest	629e2210a141	189MB	46.4MB	
ish-node-app:v2.0	629e2210a141	189MB	46.4MB	
ishcosmo/ish-flask-app:1.0	ba821985af05	200MB	48.8MB	U
ishcosmo/ish-flask-app:latest	ba821985af05	200MB	48.8MB	U
ishcosmo/ish-flask-app:production	2800705c8955	200MB	48.8MB	U
ishcosmo/ish-node-app:production	629e2210a141	189MB	46.4MB	

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
da51bce5710e	ff7b34604ada	"docker-entrypoint.s..."	23 minutes ago	Up 23 minutes	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp	node-container

1. Clean Build

- using the time `docker build --no-cache -t clean-app .`

```

ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-node-app$ time docker build --no-cache -t clean-app .
[+] Building 17.7s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 183B
=> [internal] load metadata for docker.io/library/node:18-alpine
=> [auth] library/node:null token for registry-1.docker.io

```



```
real    0m19.300s
user    0m0.205s
sys     0m0.380s
```

- using the `time docker build -t cached-app .`

```
ishita@ish:~/ContainerizationAndDevOps/LAB/Experiment4/ish-node-app$ time docker build -t cached-app .
[+] Building 2.1s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 183B
=> [internal] load metadata for docker.io/library/node:18-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
=> [internal] load build context
=> => transferring context: 59B
=> CACHED [2/5] WORKDIR /app
```

```
real    0m0.782s
user    0m0.097s
sys     0m0.141s
```

- Result: In the first command , it will take the longest time as Docker downloads every layer from scratch and then the cached one Docker reuses the layers from the previous build thats why it takes less time

ContainerizationAndDevOps

Experiment 5: Docker - Volumes, Environment Variables, Monitoring and Networks

Part 1: Docker Volumes

1. Creating a container that writes data using `docker run -it --name ish-cont ubuntu /bin/bash`
 - using the echo command "Hello world" was written in message.txt file by resolving the directory not found error
 - And later displayed using `cat` command

```
ishita@Ish:~/ContainerizationAndDevOps$ cd ./LAB/Experiment5
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker run -it --name ish-cont ubuntu /bin/bash
root@740763c5cc12:/# echo "Hello World" > /data/message.txt
bash: /data/message.txt: No such file or directory
root@740763c5cc12:/# mkdir /data
root@740763c5cc12:/# echo "Hello World" > /data/message.txt
root@740763c5cc12:/# cat /data/message.txt
Hello World
root@740763c5cc12:/# exit
exit
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$
```

- Now after exiting when we restart (remove and try to start the container) the container using `docker start ish-cont` and then try to display data using `docker exec ish-cont cat /data/message.txt` the output was **Data DIDN'T persist** and an error was encountered

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker start ish-cont
ish-cont
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker exec ish-cont cat /data/message.txt
cat: /data/message.txt: No such file or directory
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$
```

1. Volume types

- 2.1 Anonymous Volume (has auto generated name)
- using command `docker run -d -v /app/data --name web1 nginx` to create a anonymous volume for temporary data storage

- And then using `docker volume ls` command anonymous value with a random hash can be displayed

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker run -d -v /app/data --name web1 nginx
d22d82d72605da73c3978d92e5b6bef38482c316c3e08df2fcad151600b093d4
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker volume ls
DRIVER      VOLUME NAME
local       1d8c275b244c313e6acad07cb9e9aa21996b75f7b103aaa6bb0fd7084710145e
local       4a63d3d9ff0e561ef6871cad702403d2bbe9ca65c8c8c766a8c0da37fd0f793
local       4f28b2ddd9d1cc85bc113016d8ff82839ebd1bed39279877422a3f00d4cfb3e8
local       7aa4890cd95bb3937e883b7f995d1dc8bb8ca84d9a07dc9d5f54c306e6b62fa0
local       7eabb808684b65c4db01b20e31e4631064a1af31d0538e50be156270a31a53f7
local       807b0f0167f5d001012b1f061f03071f507000730e02c750010171e6311e
```

- Verifying the mount point using `docker inspect web1 | grep -A 5 Mounts`

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker inspect web1 | grep -A 5 Mounts
    "Mounts": [
      {
        "Type": "volume",
        "Name": "d1dede2685b4f0d914323e838319d07e5b3ccfd50f5247abedbd742470e512a9",
        "Source": "/var/lib/docker/volumes/d1dede2685b4f0d914323e838319d07e5b3ccfd50f5247abedbd742470e512a9/_data",
        "Destination": "/app/data",
```

- 2.2 Named volumes
- using command `docker volume create ivol` a volume named **ivol** is created.
- Using `docker run -d -v ivol:/app/data --name web2 nginx` the named volume can be used

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker volume create ivol
ivol
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker run -d -v ivol:/app/data --name web2 nginx
6a55015d617b4122f4855913502afdb3b28a938bd3c902ad688a133a742bcc68
```

- And can be verified using `docker volume ls` command and inspected using `docker volume inspect ivol`


```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker volume ls
local      ec7c592525791db865974f9dfe5b61aa94f046da6a1f4c0071de7f6aff75efd8
local      ec452fca560e480139849b2f690bdb5d8bd1d26ca6dbca474cdeb0dd3d9628b
local      f4f959e1ea46dbef109bae965827228f725ec3baf5ff3b5118d8c7af8099a656
local      f334a8065a50045c461d194bd59aab686b13d52b1729f16ae8fc99e545ac8f56
local      fb419492b9bfc62d7912341ffcf26ca55ef4ead3ab6373228cf3d0423707ccdf
local      fc1d694952f0ea5cb4a669544beb465cb699b6e3e5ad571c6c9945a789a228df
local      ffbd1817327230a62226e72db42d50612329fc4582fe1e6b12fd4bf99558d61
local      ishvol
local      ivol
local      scalecompose_mysql_data
local      scalecompose_wp_content
```

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker volume inspect ivol
[
  {
    "CreatedAt": "2026-02-28T08:01:33Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/ivol/_data",
    "Name": "ivol",
    "Options": null,
    "Scope": "local"
  }
]
```

1. Bindmounts

- o Creating a directory on host using `mkdir ~/myapp-data`
- Then mounting this host directory to the container using `docker run -d -v ~/myapp-data:/app/data --name web3 nginx`

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ mkdir ~/myapp-data
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker run -d -v ~/myapp-data:/app/data --name web3 nginx
b2e2248e982014c14fab9d68d68c62bb62df09eca2233a51429ad518584dc0ae
```

- now to add file on host `echo "From Host" > ~/myapp-data/host-file.txt` and checking in the container using `docker exec web3 cat /app/data/host-file.txt`

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ echo "From Host" > ~/myapp-data/host-file.txt
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker exec web3 cat /app/data/host-file.txt
From Host
```

3 Practical Volume examples

- 3.1 Database with persistent storage

- To create a container with named volume using `docker run -d \ --name mysql-db \ -v mysql-data:/var/lib/mysql \ -e MYSQL_ROOT_PASSWORD=secret \ mysql:8.0` since the sql image wasn't found locally it was pulled

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker run -d \
--name mysql-db \
-v mysql-data:/var/lib/mysql \
-e MYSQL_ROOT_PASSWORD=secret \
mysql:8.0
Unable to find image 'mysql:8.0' locally
8.0: Pulling from library/mysql
3ca1b3148b2f: Pull complete
bfb2a1b21556: Pull complete
```

- To check if data persisted I stopped and removed using `docker stop` and `docker rm` commands and later created a new container *new-mysql* and clearly data was preserved

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker stop mysql-db
mysql-db
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker rm mysql-db
mysql-db
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker run -d \
--name new-mysql \
-v mysql-data:/var/lib/mysql \
-e MYSQL_ROOT_PASSWORD=secret \
mysql:8.0
ed7fe287b01d8307318509d09737e55f4ba4b7649c1f234a68370f8e2d507b54
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$
```

- 3.2 Web App with configuration files
- In a newly made directory using `mkdir ~/nginx-config`, a new `nginx.conf` file was made using `echo 'server { listen 80; server_name localhost; location / { return 200 "Hello from mounted config!"; } }' > ~/nginx-config/nginx.conf`

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ mkdir ~/nginx-config
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ echo 'server {
listen 80;
server_name localhost;
location / {
return 200 "Hello from mounted config!";
}
}' > ~/nginx-config/nginx.conf
```


- and then nginx was with config bind mount and tested with `curl` command

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker run -d \
  --name nginx-custom \
  -p 8080:80 \
  -v ~/nginx-config/nginx.conf:/etc/nginx/conf.d/default.conf \
  nginx
114d2f35452f0ca6000074c520a53d7ae408a1199957c2af6da6ab6eb5d02693
```

i. Volume management commands

- 4.1 To list all volumes `docker volume ls` was used

```
Hello from mounted config!ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker volume ls
local    ec452fca560e480139849b2f690bdbc5d8bd1d26ca6dbca474cdeb0dd3d9628b
local    f4f959e1ea46dbef109bae965827228f725ec3baf5ff3b5118d8c7af8099a656
local    f334a8065a50045c461d194bd59aab686b13d52b1729f16ae8fc99e545ac8f56
local    fb419492b9bfc62d7912341ffcf26ca55ef4ead3ab6373228cf3d0423707ccdf
local    fc1d694952f0ea5cb4a669544beb465cb699b6e3e5ad571c6c9945a789a228df
local    ffbda1817327230a62226e72db42d50612329fc4582fe1e6b12fd4bf99558d61
```

- 4.2 To create a volume `docker volume create xyz` was used (here named)
- 4.3 To inspect volumes `docker volume inspect xyz-volume` was used
- 4.4 To remove unused volumes `docker volume prune` was used

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker volume prune
WARNING! This will remove anonymous local volumes not used by at least one container.
Are you sure you want to continue? [y/N] N
```

- 4.5 To remove specific volume `docker volume rm xyz` is used

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker volume rm ishvol
ishvol
```

Part 2: Environment Variables

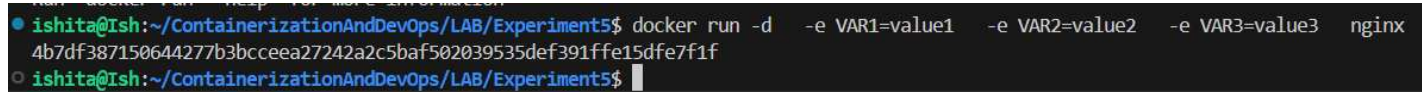
1. Setting Environment variables using

- 1.1 Using `-e` flag

```
# Single variable
docker run -d \
  --name app1 \
  -e DATABASE_URL="postgres://user:pass@db:5432/mydb" \
  -e DEBUG="true" \
  -p 3000:3000 \
  nginx
```

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker run -d --name app1 -e DATABASE_URL="postgres://user:pass@db:5432/mydb" -e DEBUG="true" -p 3000:3000 nginx
c3997a9f768343a42dd44c7d280ff127c89d48fc4b89e9d56af4fc0ff6b7e8ce
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$
```

```
# Multiple variables
docker run -d \
-e VAR1=value1 \
-e VAR2=value2 \
-e VAR3=value3 \
nginx
```



```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker run -d -e VAR1=value1 -e VAR2=value2 -e VAR3=value3 nginx
4b7df387150644277b3bcccea27242a2c5baf502039535def391ffe15dfe7f1f
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$
```

- 1.2 Using `--env-file` `bash`

Create .env file

```
echo "DATABASE_HOST=localhost" > .env echo "DATABASE_PORT=5432" >> .env echo
"API_KEY=secret123" >> .env
```

Use env file

```
docker run -d
--env-file .env
--name app2
nginx
```

Use multiple env files

```
docker run -d
--env-file .env
--env-file .env.secrets
nginx
```

```


```

```
- 1.3 Dockerfile
```Dockerfile
FROM ubuntu
Set default environment variables
ENV NODE_ENV=production
```

```
ENV PORT=3000
```

```
ENV APP_VERSION=1.0.0
```

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker build -t my-node-app .
[+] Building 0.9s (5/5) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 109B
=> [internal] load metadata for docker.io/library/ubuntu:latest
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/1] FROM docker.io/library/ubuntu:latest@sha256:cd1dba651b3080c3686ecf4e3c4220f026b521fb76978881737d24f200828b2b
=> => resolve docker.io/library/ubuntu:latest@sha256:cd1dba651b3080c3686ecf4e3c4220f026b521fb76978881737d24f200828b2b
=> exporting to image
```

## 1. Environment Variables in Applications

- Making a python app **app.py** with the following contents ``python

### app.py

```
import os from flask import Flask
```

```
app = Flask(name)
```

## Read environment variables

```
db_host = os.environ.get('DATABASE_HOST', 'localhost') debug_mode = os.environ.get('DEBUG',
'false').lower() == 'true' api_key = os.environ.get('API_KEY')
```

```
@app.route('/config') def config(): return { 'db_host': db_host, 'debug': debug_mode, 'has_api_key':
bool(api_key) }
```

```
if name == 'main': port = int(os.environ.get('PORT', 5000)) app.run(host='0.0.0.0', port=port,
debug=debug_mode)
```

```
- Dockerfile contents
```

```
``Dockerfile
```

```
FROM python:3.9-slim
```

```
Set environment variables at build time
```

```
ENV PYTHONUNBUFFERED=1
```

```
ENV PYTHONDONTWRITEBYTECODE=1
```

```
WORKDIR /app
```

```
COPY requirements.txt .
RUN pip install -r requirements.txt

COPY app.py .

Default runtime environment variables
ENV PORT=5000
ENV DEBUG=false

EXPOSE 5000

CMD ["python", "app.py"]
```

1. testing environment variables ``Docker

## Run with custom env vars

---

```
docker run -d
--name flask-app
-p 5000:5000
-e DATABASE_HOST="prod-db.example.com"
-e DEBUG="true"
-e PORT="8080"
ish-flask-app
```

## Check environment in running container

---

```
docker exec flask-app env docker exec flask-app printenv DATABASE_HOST
```

## Test the endpoint

---

```
curl http://localhost:5000/config
```

```
![/ContainerizationAndDevOps/LAB/Experiment5/22.png)
![/ContainerizationAndDevOps/LAB/Experiment5/23.png)
![/ContainerizationAndDevOps/LAB/Experiment5/24.png)
```

## Part 3: Docker Monitoring

- 3.1 Basic commands



```
```docker
# Live stats for all containers
docker stats
```

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
d78ade905b11	flask-app	0.25%	43.01MiB / 7.594GiB	0.55%	5kB / 2.69kB	8.17MB / 0B	3
28fa911de65d	app4	0.00%	13.23MiB / 7.594GiB	0.17%	1.5kB / 126B	127kB / 12.3kB	17
b8d612c5d607	app3	0.00%	13.04MiB / 7.594GiB	0.17%	1.63kB / 126B	4.12MB / 12.3kB	17
f30d600335c4	app2	0.00%	13.23MiB / 7.594GiB	0.17%	1.75kB / 126B	4.03MB / 12.3kB	17
4b7df3871506	lucid_saha	0.00%	13.17MiB / 7.594GiB	0.17%	1.88kB / 126B	0B / 12.3kB	17
c3997a9f7603	app1	0.00%	16.65MiB / 7.594GiB	0.21%	2.83kB / 126B	14.6MB / 12.3kB	17

```
# Live stats for specific containers
docker stats container1 container2
```

- Here my container 1 and 2 are app1 and app2

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
c3997a9f7603	app1	0.00%	16.65MiB / 7.594GiB	0.21%	2.83kB / 126B	14.6MB / 12.3kB	17
f30d600335c4	app2	0.00%	13.23MiB / 7.594GiB	0.17%	1.75kB / 126B	4.03MB / 12.3kB	17

```
# Specific format output
docker stats --format "table \t\t\t"
```

NAME	CPU %	MEM USAGE / LIMIT	NET I/O
flask-app	0.11%	43.26MiB / 7.594GiB	5kB / 2.69kB
app4	0.00%	13.23MiB / 7.594GiB	1.5kB / 126B
app3	0.00%	13.04MiB / 7.594GiB	1.63kB / 126B
app2	0.00%	13.23MiB / 7.594GiB	1.75kB / 126B
lucid_saha	0.00%	13.17MiB / 7.594GiB	1.88kB / 126B
app1	0.00%	16.65MiB / 7.594GiB	2.83kB / 126B

```
# No-stream (single snapshot)
docker stats --no-stream
```

```
# All containers (including stopped)
docker stats --all
```

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker stats --no-stream
```

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
d78ade905b11	flask-app	0.24%	43.01MiB / 7.594GiB	0.55%	5kB / 2.69kB	8.27MB / 0B	3
28fa911de65d	app4	0.00%	13.23MiB / 7.594GiB	0.17%	1.5kB / 126B	127kB / 12.3kB	17
b8d612c5d607	app3	0.00%	13.04MiB / 7.594GiB	0.17%	1.63kB / 126B	4.12MB / 12.3kB	17
f30d600335c4	app2	0.00%	13.23MiB / 7.594GiB	0.17%	1.75kB / 126B	4.03MB / 12.3kB	17
4b7df3871506	lucid_saha	0.00%	13.17MiB / 7.594GiB	0.17%	1.88kB / 126B	0B / 12.3kB	17
c3997a9f7603	app1	0.00%	16.65MiB / 7.594GiB	0.21%	2.83kB / 126B	14.6MB / 12.3kB	17

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$
```

d78ade905b11	flask-app	0.05%	43.26MiB / 7.594GiB	0.56%	5kB / 2.69kB	8.23MB / 0B	3
d78ade905b11	flask-app	0.29%	43.01MiB / 7.594GiB	0.55%	5kB / 2.69kB	8.23MB / 0B	3
CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
d78ade905b11	flask-app	0.17%	43.01MiB / 7.594GiB	0.55%	5kB / 2.69kB	8.23MB / 0B	3
CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
d78ade905b11	flask-app	0.17%	43.01MiB / 7.594GiB	0.55%	5kB / 2.69kB	8.23MB / 0B	3
CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
d78ade905b11	flask-app	0.20%	43.01MiB / 7.594GiB	0.55%	5kB / 2.69kB	8.23MB / 0B	3
CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
d78ade905b11	flask-app	0.20%	43.01MiB / 7.594GiB	0.55%	5kB / 2.69kB	8.23MB / 0B	3
CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
d78ade905b11	flask-app	0.32%	43.01MiB / 7.594GiB	0.55%	5kB / 2.69kB	8.23MB / 0B	3
CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS

- 3.2 Process Monitoring ``docker

View processes in container

docker top container-name# View with full command line

docker top container-name -ef# Compare with host processes

ps aux | grep docker

- 3.3: docker logs - Application Logs

``docker

View logs

docker logs container-name# Follow logs (like tail -f)

C# Last N lines

docker logs --tail 100 container-name# Logs with timestamps

```
docker logs -t container-name# Logs since specific time
```

```
docker logs --since 2024-01-15 container-name# Combine options
```

```
docker logs -f --tail 50 -t container-name
```

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker logs app4
2026/03/02 13:48:04 [notice] 1#1: start worker process 41
2026/03/02 13:48:04 [notice] 1#1: start worker process 42
2026/03/02 13:48:04 [notice] 1#1: start worker process 43
2026/03/02 13:48:04 [notice] 1#1: start worker process 44
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker logs flask-app
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:8080
```

- 3.4: Container Inspection ``docker

Detailed container info

```
docker inspect container-name
```

Specific information

```
docker inspect --format="" container-name
```

```
docker inspect --format="" container-name
```

```
docker inspect --format="" container-name
```

Resource limits

```
docker inspect --format="" container-name
```

```
docker inspect --format="" container-name
```

```

```

- 3.5: Events Monitoring

```
```Docker
```

```
Monitor Docker events in real-time
```



```

docker events
Filter events

docker events --filter 'type=container'

docker events --filter 'event=start'

docker events --filter 'event=die'
Since specific time

docker events --since '2024-01-15'# Format output

docker events --format ' '

```



```

shita@ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker events --since '60m'
5-03-02T14:08:18.668510926Z container create db83ce40cc7defba4f1c3673857c6aab35d1488736a73f3e14bdc86df6ae893b (desktop.docker.io/ports.scheme=v2, desktop.docker.io/ports/5000/tcp=:5000, desktop.docker.io/wsl-distro=Ubuntu, image=ish-flask-app, name=flask-app)
5-03-02T14:08:19.159426421Z network connect c27251d25a2098a14f9a0edf3101d0aaef10761a51827e3c6f65e9d8a5b7db1b (container=db83ce40cc7defba4f1c3673857c6aab35d1488736a73f3e14bdc86df6ae893b, name=bridge, type=bridge)
5-03-02T14:08:19.205246208Z container start db83ce40cc7defba4f1c3673857c6aab35d1488736a73f3e14bdc86df6ae893b (desktop.docker.io/ports.scheme=v2, desktop.docker.io/ports/5000/tcp=:5000, desktop.docker.io/wsl-distro=Ubuntu, image=ish-flask-app, name=flask-app)
5-03-02T14:08:32.572459621Z container exec_create: env db83ce40cc7defba4f1c3673857c6aab35d1488736a73f3e14bdc86df6ae893b (desktop.docker.io/ports.scheme=v2, desktop.docker.io/ports/5000/tcp=:5000, desktop.docker.io/wsl-distro=Ubuntu, execID=883f258422e4ecd1b2769c7f27242e8867fa1c5e171627eb91658b101322f75e, image=ish-flask-app, name=flask-app)
5-03-02T14:08:32.591282989Z container exec_start: env db83ce40cc7defba4f1c3673857c6aab35d1488736a73f3e14bdc86df6ae893b (desktop.docker.io/ports.scheme=v2, desktop.docker.io/ports/5000/tcp=:5000, desktop.docker.io/wsl-distro=Ubuntu, execID=883f258422e4ecd1b2769c7f27242e8867fa1c5e171627eb91658b101322f75e, image=ish-flask-app, name=flask-app)
5-03-02T14:08:32.682830023Z container exec_die db83ce40cc7defba4f1c3673857c6aab35d1488736a73f3e14bdc86df6ae893b (desktop.docker.io/ports.scheme=v2, desktop.docker.io/ports/5000/tcp=:5000, desktop.docker.io/wsl-distro=Ubuntu, execID=883f258422e4ecd1b2769c7f27242e8867fa1c5e171627eb91658b101322f75e, image=ish-flask-app, name=flask-app)
5-03-02T14:08:40.539698796Z container exec_create: printenv DATABASE_HOST db83ce40cc7defba4f1c3673857c6aab35d1488736a73f3e14bdc86df6ae893b (desktop.docker.io/ports.scheme=v2, desktop.docker.io/ports/5000/tcp=:5000, desktop.docker.io/wsl-distro=Ubuntu, execID=5f290ca92d337368d930085dd78771b2ba6be4fd6c07f596, image=ish-flask-app, name=flask-app)

```

- 3.6 Practical Monitoring Script `` #!/bin/bash

## monitor.sh - Simple Docker monitoring

```
echo "=== Docker Monitoring Dashboard ===" echo "Time: $(date)"echo echo "1. Running Containers:"
```

```
docker ps --format "table \t\t"echo
```

```
echo "2. Resource Usage:"
```

```
docker stats --no-stream --format "table \t\t\t\t"echo
```

```
echo "3. Recent Events:"
```

```
docker events --since '5m' --until '0s' --format ' ' tail -5echo
```



echo "4. System Info:"

docker system df

![/ContainerizationAndDevOps/LAB/Experiment5/35.png]

![/ContainerizationAndDevOps/LAB/Experiment5/36.png]

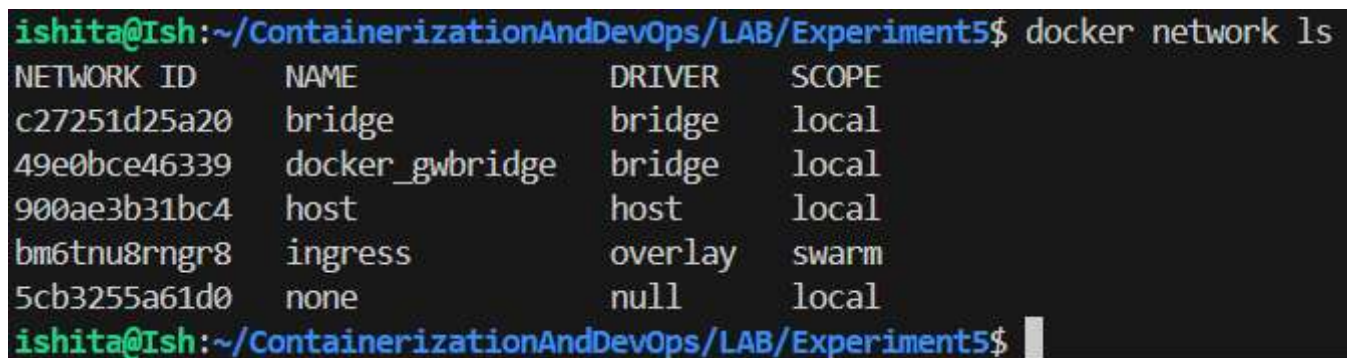
## Part 4: Docker Networks

- 4.1: Understanding Docker Network Types

```docker

Default networks

docker network ls



| NETWORK ID | NAME | DRIVER | SCOPE |
|--------------|-----------------|---------|-------|
| c27251d25a20 | bridge | bridge | local |
| 49e0bce46339 | docker_gwbridge | bridge | local |
| 900ae3b31bc4 | host | host | local |
| bm6tnu8rngr8 | ingress | overlay | swarm |
| 5cb3255a61d0 | none | null | local |

- 4.2: Network Types Explained

1. Bridge Network (Default) ```docker

Containers on bridge network can communicate

Each container gets own IP, isolated from host

Create custom bridge network

docker network create my-network


```
```docker
```

```
Inspect network
```

```
docker network inspect my-network
```

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker network inspect ish-network
[
 {
 "Name": "ish-network",
 "Id": "547a840248abef62f83987333656ed66d2bf75b382cd1e64bb9ae6509a4a136a",
 "Created": "2026-03-02T15:15:26.547462033Z",
 "Scope": "local",
 "Driver": "bridge",
 "EnableIPv4": true,
 "EnableIPv6": false,
 "IPAM": {
 "Driver": "default",
 "Options": {},
 }
 }
]
```

```
Run containers on custom network
```

```
docker run -d --name web1 --network my-network nginx
```

```
docker run -d --name web2 --network my-network nginx
```

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker run -d --name web11 --network ish-network nginx
30226fb52e2d7aaff3ce5df6501f47693b729c2d947652b5a97a999903b64c9f
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$
```

```
Containers can communicate using container names
```

```
docker exec web1 curl http://web2
```

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker exec web11 curl http://web12
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
nginx.org.

Commercial support is available at
nginx.com.</p>

<p>Thank you for using nginx.</p>
</body>
```

## 1. Host Network ``docker

# Container uses host's network directly# No network isolation, shares host's IP

```
docker run -d --name host-app --network host nginx
```

## Access directly on host port 80

```
curl http://localhost
```



## 3. None Network

```
````docker
```

```
# No network access
```

```
docker run -d --name isolated-app --network none alpine sleep 3600
```

```
# Test - no network interfaces
```

```
docker exec isolated-app ifconfig
```

```
# Only loopback interface
```

```
ishita@Ish:~/ContainerizationAndDevOps$ cd ./LAB/Experiment5
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker run -d --name isolated-app --network none alpine sleep 3600
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
Digest: sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c3f659
Status: Downloaded newer image for alpine:latest
29141488a78d3354bc0d77955d32c26ee3e809f46c646dc3ccb92c9e1019027d
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker exec isolated-app ifconfig
lo          Link encap:Local Loopback
            inet addr:127.0.0.1  Mask:255.0.0.0
            inet6 addr: ::1/128 Scope:Host
            UP LOOPBACK RUNNING  MTU:65536  Metric:1
```

1. Overlay Network (Swarm)

```
# For Docker Swarm - multi-host networking
```

```
docker network create --driver overlay my-overlay
```

- 4.3: Network Management Commands

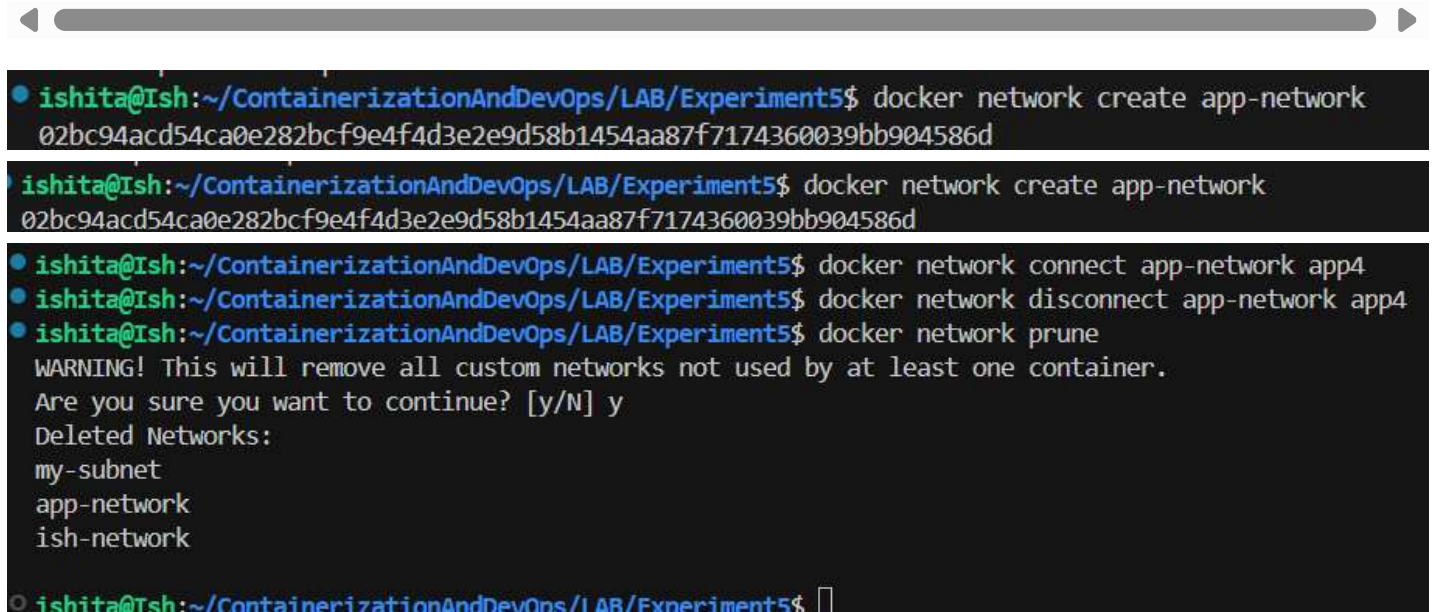
```
# Create network
docker network create app-network
docker network create --driver bridge --subnet 172.20.0.0/16 --gateway 172.20.0.1 my-subnet

# Connect container to network
docker network connect app-network existing-container

# Disconnect container from network
docker network disconnect app-network container-name

# Remove network
docker network rm network-name

# Prune unused networks
docker network prune
```



```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker network create app-network
02bc94acd54ca0e282bcf9e4f4d3e2e9d58b1454aa87f7174360039bb904586d

ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker network create app-network
02bc94acd54ca0e282bcf9e4f4d3e2e9d58b1454aa87f7174360039bb904586d

ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker network connect app-network app4
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker network disconnect app-network app4
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker network prune
WARNING! This will remove all custom networks not used by at least one container.
Are you sure you want to continue? [y/N] y
Deleted Networks:
my-subnet
app-network
ish-network

ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$
```

• 4.4: Multi-Container Application Example

Web App + Database Communication

```
# Create network

docker network create app-network# Start database

docker run -d \

--name postgres-db \

--network app-network \
```



```
-e POSTGRES_PASSWORD=secret \  
  
-v pgdata:/var/lib/postgresql/data \  
  
postgres:15# Start web application  
  
docker run -d \  
  
--name web-app \  
  
--network app-network \  
  
-p 8080:3000 \  
  
-e DATABASE_URL="postgres://postgres:secret@postgres-db:5432/mydb" \  
  
-e DATABASE_HOST="postgres-db" \  
  
node-app# Web app can connect to database using "postgres-db" hostname
```

```

ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker network create app-network
5dbf418a4f8965f3733f986ff0afa8e0ee3388b26bbbc8d60c53c799d2b81dcd
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker run -d \
--name postgres-db \
--network app-network \
-e POSTGRES_PASSWORD=secret \
-v pgdata:/var/lib/postgresql/data \
postgres:15
Unable to find image 'postgres:15' locally
15: Pulling from library/postgres
8d9526eade8e: Pull complete
67266521af9d: Pull complete
0c725cebedaa: Pull complete
f0c010f1448f: Pull complete
f080c6a4c94c: Pull complete
b6de334a9157: Pull complete
e01de8c32e6c: Pull complete
2f8ddeccdf2d: Pull complete
fd349cbbb730: Pull complete
bae50de223a4: Pull complete
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker run -d \
--name web-app \
--network app-network \
-p 8080:3000 \
-e DATABASE_URL="postgres://postgres:secret@postgres-db:5432/mydb" \
-e DATABASE_HOST="postgres-db" \
node-app
effb6290b13070efeda8de712bcf0b09d05d81ee164281e311a292372b9c72a1
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$

```

- 4.5: Network Inspection & Debugging ``docker

Inspect network

docker network inspect bridge# Check container IP

docker inspect --format="" container-name# DNS resolution test

docker exec container-name nslookup another-container# Network connectivity test

docker exec container-name ping -c 4 google.com

docker exec container-name curl -I http://another-container# View network ports

docker port container-name

- 4.6: Port Publishing vs Exposing

```docker

# PORT PUBLISHING (host:container)

```
docker run -d -p 80:8080 --name app1 nginx
```

# Host port 80 → Container port 8080# Dynamic port publishing

```
docker run -d -p 8080 --name app2 nginx
```

# Docker assigns random host port# Multiple ports

```
docker run -d -p 80:80 -p 443:443 --name app3 nginx
```

# Specific host IP

```
docker run -d -p 127.0.0.1:8080:80 --name app4 nginx
```

# EXPOSE in Dockerfile (metadata only)

# Dockerfile: EXPOSE 80# Still need -p to publish

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker run -d -p 80:8080 --name app0 nginx
f6c26ff4b69e67c27e82a207739d09664a3ebe63fa07768f058497477604c89c
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker run -d -p 8080 --name app01 nginx
b676e3f53e24d71a8b754126350e302770252bc32248ffda3f4c3bbbb1e920ee
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker run -d -p 80:80 -p 443:443 --name app02 nginx
29c418346957b81ce9220f5bb94026396e314362adfd9f25687045334dd760be
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker run -d -p 127.0.0.1:8080:80 --name app03 nginx
715f5e2a2eff1b0d7dfff81e9b3bd773e89bcd794f44a05d4732aefaea0e4143
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$
```

## Part 5: Complete Real-World Example

=> *Application Architecture:*

- Flask Web App (port 5000)
- PostgreSQL Database (port 5432)
- Redis Cache (port 6379)
- All connected via custom network

Implementation:

# 1. Create network

```
docker network create myapp-network
```

## # 2. Start database with volume

```
docker run -d \
 --name postgres \
 --network myapp-network \
 -e POSTGRES_PASSWORD=mysecretpassword \
 -e POSTGRES_DB=mydatabase \
 -v postgres-data:/var/lib/postgresql/data \
 postgres:15
```

## # 3. Start Redis

```
docker run -d \
 --name redis \
 --network myapp-network \
 -v redis-data:/data \
 redis:7-alpine
```

## # 4. Start Flask app with all configurations

```
docker run -d \
 --name flask-app \
 --network myapp-network \
 -p 5000:5000 \
 -v $(pwd)/app:/app \
 -v app-logs:/var/log/app \
 -e DATABASE_URL="postgresql://postgres:mysecretpassword@postgres:5432/mydatabase" \
 -e REDIS_URL="redis://redis:6379" \
 -e DEBUG="false" \
 -e LOG_LEVEL="INFO" \
 --env-file .env.production \
 flask-app:latest
```



```

● ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker network create myapp-network
1d8c11577edb479ab56ef9d3a3031a043b432dae7a843df0b2b95982d22a012f
● ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker run -d \
 --name postgres \
 --network myapp-network \
 -e POSTGRES_PASSWORD=mysecretpassword \
 -e POSTGRES_DB=mydatabase \
 -v postgres-data:/var/lib/postgresql/data \
 postgres:15
80f854b676cf59f6ddd47666059ee0238d5e8c82c7a78f35cc26adfb21706ca7

```

```

ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker run -d \
 --name redis \
 --network myapp-network \
 -v redis-data:/data \
 redis:7-alpine
Unable to find image 'redis:7-alpine' locally
7-alpine: Pulling from library/redis
1c2c8d1ee428: Pull complete
4f4fb700ef54: Pull complete
9fe3744a2eac: Pull complete
2a97533d89ca: Pull complete
bc1da058f299: Pull complete
09a5a0c32a23: Pull complete
7ad54b3c4cef: Pull complete
bd53938b1271: Pull complete
06cd2b88db40: Download complete
9bbcf5d5580: Download complete
Digest: sha256:8b81dd37ff027bec4e516d41acfbe9fe2460070dc6d4a4570a2ac5b9d59df065
Status: Downloaded newer image for redis:7-alpine

```

```

● ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ cat <<EOF > .env.production
DATABASE_URL=postgresql://postgres:mysecretpassword@postgres:5432/mydatabase
REDIS_URL=redis://redis:6379
DEBUG=false
LOG_LEVEL=INFO
EOF

```

```

● ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker run -d --name flaskapp --network myapp-network -p 5000:5000 -v $(pwd)/app:/app -v app-logs:/var/log/app
--env-file .env.production flask-app:latest
1d328eb5448ebff75e73bea243369b1509ea5d754ff8035ef102156f51b2320d

```

### • Monitoring Commands:

*# Check all components*

```
docker ps
```

*# Monitor resources*

```
docker stats postgres redis flask-app
```

*# Check Logs*

```
docker logs -f flask-app
```

*# Network connectivity test*

```
docker exec flask-app ping -c 2 postgres
```

```
docker exec flask-app ping -c 2 redis
```

```
View network details
```

```
docker network inspect myapp-network
```

| CONTAINER ID | IMAGE          | COMMAND                  | CREATED           | STATUS           | PORTS                                         | NAMES    |
|--------------|----------------|--------------------------|-------------------|------------------|-----------------------------------------------|----------|
| 708483f81520 | redis:7-alpine | "docker-entrypoint.s..." | About an hour ago | Up About an hour | 6379/tcp                                      | redis    |
| 80f854b676cf | postgres:15    | "docker-entrypoint.s..." | About an hour ago | Up About an hour | 5432/tcp                                      | postgres |
| 715f5e2a2eff | nginx          | "/docker-entrypoint..."  | About an hour ago | Up About an hour | 127.0.0.1:8080->80/tcp                        | app03    |
| b676e3f53e24 | nginx          | "/docker-entrypoint..."  | About an hour ago | Up About an hour | 0.0.0.0:64390->8080/tcp, [::]:64390->8080/tcp | app01    |
| f6c26ff4b69e | nginx          | "/docker-entrypoint..."  | About an hour ago | Up About an hour | 0.0.0.0:80->8080/tcp, [::]:80->8080/tcp       | app0     |

| CONTAINER ID | NAME      | CPU % | MEM USAGE / LIMIT   | MEM % | NET I/O       | BLOCK I/O       | PIDS |
|--------------|-----------|-------|---------------------|-------|---------------|-----------------|------|
| 80f854b676cf | postgres  | 0.00% | 38.46MiB / 7.594GiB | 0.49% | 1.87kB / 126B | 81.2MB / 53.5MB | 6    |
| 708483f81520 | redis     | 0.94% | 3.848MiB / 7.594GiB | 0.05% | 914B / 126B   | 0B / 0B         | 6    |
| d78ade905b11 | flask-app | 0.00% | 0B / 0B             | 0.00% | 0B / 0B       | 0B / 0B         | 0    |

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker logs -f flask-app
```

```
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:8080
* Running on http://172.17.0.7:8080
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 125-873-672
172.17.0.1 - - [02/Mar/2026 14:24:14] "GET /config HTTP/1.1" 200 -
172.17.0.1 - - [02/Mar/2026 14:25:48] "GET /config HTTP/1.1" 200 -
172.17.0.1 - - [02/Mar/2026 14:25:48] "GET /config HTTP/1.1" 200 -
172.17.0.1 - - [02/Mar/2026 14:25:48] "GET /favicon.ico HTTP/1.1" 404 -
```

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker exec flask-app ping -c 2 postgres
```

```
PING postgres (172.20.0.2) 56(84) bytes of data.
64 bytes from postgres.myapp-network (172.20.0.2): icmp_seq=1 ttl=64 time=0.545 ms
64 bytes from postgres.myapp-network (172.20.0.2): icmp_seq=2 ttl=64 time=0.082 ms
```

```
--- postgres ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1020ms
rtt min/avg/max/mdev = 0.082/0.313/0.545/0.231 ms
```

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$
```

## Practice Exercises

### Exercise 1: Database Backup

```
Create a PostgreSQL container with volume
```

```
docker run -d \
```

```
--name postgres-source \
```

```
--network ishnet \
```

```
-e POSTGRES_PASSWORD=mysecretpassword \
```

```
-e POSTGRES_DB=mydatabase \
```

```
-v postgres-data:/var/lib/postgresql/data \
```

```
postgres:15
```

```
Backup data using docker cp or volume backup techniques
```

- using pg\_dump

```
docker exec postgres-source pg_dump -U postgres mydatabase > backup.sql
```

-using docker cp

```
docker cp postgres-source:/var/lib/postgresql/data ./postgres-backup-files
```

# Restore to new container

```
docker run -d \
 --name postgres-restored \
 --network ishnet \
 -e POSTGRES_PASSWORD=mysecretpassword \
 -e POSTGRES_DB=mydatabase \
 postgres:15
```

```
cat backup.sql | docker exec -i postgres-restored psql -U postgres -d mydatabase
```

- firstly a network called ishnet was created usingnng docker network create ishnet

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker network create ishnet
f675da91e00750c2b6c7945b2aeb9ce28566295d41bd15637ff177bafec4c65f1
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker run -d \
 --name postgres-source \
 --network ishnet \
 -e POSTGRES_PASSWORD=mysecretpassword \
 -e POSTGRES_DB=mydatabase \
 -v postgres-data:/var/lib/postgresql/data \
 postgres:15
050da96780f0738b1ed1f270cd8e6166b8f1ae0538738b94812aa0825eeeb02c
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker exec postgres-source pg_dump -U postgres mydatabase > backup.sql
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker cp postgres-source:/var/lib/postgresql/data ./postgres-backup-files
Successfully copied 48.3MB to /home/ishita/ContainerizationAndDevOps/LAB/Experiment5/postgres-backup-files
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker run -d \
 --name postgres-restored \
 --network ishnet \
 -e POSTGRES_PASSWORD=mysecretpassword \
 -e POSTGRES_DB=mydatabase \
 postgres:15
32925e20406c48bad616419791e43555f12bf62078e0d30e513771c0f71c1c7f
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ cat backup.sql | docker exec -i postgres-restored psql -U postgres -d mydatabase
SET
SET
SET
SET
SET
set_config

(1 row)

SET
SET
SET
SET
```

## Exercise 2: Multi-Service Setup

# Create: web app + database + cache

```
docker run -d --name devops-db --network lab-net \
 -e POSTGRES_PASSWORD=mysecretpassword -e POSTGRES_DB=labdb \
 -v devops-db-data:/var/lib/postgresql/data postgres:15
```



```

docker run -d --name devops-cache --network lab-net \
 -v devops-cache-data:/data redis:7-alpine

docker run -d --name devops-web --network lab-net -p 5001:5000 \
 -e DATABASE_URL="postgresql://postgres:mysecretpassword@devops-db:5432/labdb" \
 -e REDIS_URL="redis://devops-cache:6379" \
 flask-app:latest

Use custom network for communication (created lab-net)

Set environment variables for configuration# Monitor all services
docker exec devops-web ping -c 2 devops-db
docker exec devops-web ping -c 2 devops-cache

docker logs -f devops-web (for verifying and monitoring)

```

- This exercise demonstrates the orchestration of a three-tier architecture (Web, Database, and Cache) using a dedicated Docker network to ensure isolated and secure communication.
- Custom Network Isolation: Created a bridge network named *lab-net* to allow service discovery via container names instead of static IP addresses.
- Service Stack:
  - *Web*: Flask-based application (devops-web) exposed on host port 5001.
  - *Database*: PostgreSQL 15 (devops-db) using a persistent volume for data sovereignty.
  - *Cache*: Redis 7-Alpine (devops-cache) for high-speed data retrieval.
- Dynamic Configuration: Utilized environment variables (e.g., DATABASE\_URL and REDIS\_URL) to securely inject connection strings into the application at runtime.

Connectivity Validation: Verified inter-container communication using ping and docker logs to ensure the Flask app could successfully reach both the database and cache layers.

```

ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker network create lab-net
f4550f7d1f7be7eb5f68d6015a115df3200ab4c143a23da5888c9953ad76d79c8
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker run -d --name devops-db --network lab-net \
-e POSTGRES_PASSWORD=mysecretpassword -e POSTGRES_DB=labdb \
-v devops-db-data:/var/lib/postgresql/data postgres:15
92575f41d7e437a8883365d56bd19b21f6325bf2bf7cfca31ffa36b63aa69be2
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker run -d --name devops-cache --network lab-net \
-v devops-cache-data:/data redis:7-alpine
fb792096e68f6f704eb84fc6603ec2f3bd3f7ff8c6cae8752340de193b9ad1cd
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker run -d --name devops-web --network lab-net -p 5001:5000 \
-e DATABASE_URL="postgresql://postgres:mysecretpassword@devops-db:5432/labdb" \
-e REDIS_URL="redis://devops-cache:6379" \
flask-app:latest
c8005de2d2443d343fde43e80ddc812ff044e0511d5391b3aef83d6cf0da5c91

```



```

ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker exec devops-web ping -c 2 devops-db
docker exec devops-web ping -c 2 devops-cache
PING devops-db (172.22.0.2) 56(84) bytes of data.
64 bytes from devops-db.lab-net (172.22.0.2): icmp_seq=1 ttl=64 time=0.175 ms
64 bytes from devops-db.lab-net (172.22.0.2): icmp_seq=2 ttl=64 time=0.127 ms

--- devops-db ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1029ms
rtt min/avg/max/mdev = 0.127/0.151/0.175/0.024 ms
PING devops-cache (172.22.0.3) 56(84) bytes of data.
64 bytes from devops-cache.lab-net (172.22.0.3): icmp_seq=1 ttl=64 time=0.199 ms
64 bytes from devops-cache.lab-net (172.22.0.3): icmp_seq=2 ttl=64 time=0.091 ms

--- devops-cache ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1006ms
rtt min/avg/max/mdev = 0.091/0.145/0.199/0.054 ms

ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker logs -f devops-web
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.22.0.4:5000
Press CTRL+C to quit

```

## Exercise 3: Log Analysis

```

Run a container that generates logs
docker run -d \
 --name log-exporter \
 -v $(pwd)/host-logs:/var/log/app \
 flask-app:latest
Use docker logs with various filters

#last 10 lines
docker logs --tail 10 log-exporter
#specific time
docker logs --since 5m log-exporter
#real time
docker logs -f log-exporter
#timestampss
docker logs -t log-exporter

Redirect Logs to a file on host using bind mount
docker logs log-exporter > exercise3_output.log 2>&1

```

- Firstly I created a file called **host-logs** for logs
- Persistent Logging: Implemented a bind mount to synchronize containerized logs with the host filesystem, ensuring data persistence beyond the container lifecycle.

- Log Filtering: Utilized docker logs with `--tail`, `--since`, and `-t` flags to demonstrate efficient debugging and forensic analysis of service events.
- Stream Redirection: Practiced redirecting stdout and stderr to local files for external analysis and reporting.

```

ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ mkdir -p $(pwd)/host-logs
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker run -d \
 --name log-exporter \
 -v $(pwd)/host-logs:/var/log/app \
 flask-app:latest
cdad3c04ee63a92fcf518c097204a7a892c2eb2fe83630cf8920b3827bec4c0f
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker logs --tail 10 log-exporter
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.6:5000
Press CTRL+C to quit
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ ^C
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker logs --since 5m log-exporter
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.6:5000
Press CTRL+C to quit
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$
^Cishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker logs -t log-exporter
2026-03-07T03:20:04.768604239Z * Serving Flask app 'app'
2026-03-07T03:20:04.768655224Z * Debug mode: off
2026-03-07T03:20:04.778078424Z WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
2026-03-07T03:20:04.778099532Z * Running on all addresses (0.0.0.0)
2026-03-07T03:20:04.778101290Z * Running on http://127.0.0.1:5000
2026-03-07T03:20:04.778102438Z * Running on http://172.17.0.6:5000
2026-03-07T03:20:04.778103546Z Press CTRL+C to quit
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker logs log-exporter > exercise3_output.log 2&1
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$

```

```

* Serving Flask app 'app'
* Debug mode: off
ESC[31mESC[1mWARNING: This is
* Running on all addresses
* Running on http://127.0.0
* Running on http://172.17.
ESC[33mPress CTRL+C to quitESC

```

## Exercise 4: Network Isolation

```
Create two separate networks
docker network create net-alpha
docker network create net-beta
Put containers in different networks
Container A in Alpha
docker run -d --name container-alpha --network net-alpha alpine sleep 3600

Container B in Beta
docker run -d --name container-beta --network net-beta alpine sleep 3600

Test connectivity between networks
This will return "Name or service not known"
docker exec container-alpha ping -c 2 container-beta
Connect a container to both networks
docker network connect net-beta container-alpha

#Verification
This will now succeed
docker exec container-alpha ping -c 2 container-beta
```

- Network Segmentation: Successfully implemented two isolated bridge networks (net-alpha and net-beta) to enforce security boundaries between containers.
- Isolation Verification: Confirmed that containers on separate networks are unreachable by default, preventing unauthorized cross-service communication.
- Dual-Homing: Demonstrated multi-network connectivity by attaching a single container to two networks, enabling it to act as a secure gateway or proxy between isolated segments.

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker network create net-alpha
docker network create net-beta
5e73c224ae9bad7a49a0dd3b65be447c6987de5d7913520a31cf3c2a580bd964
ee170497971331b18415f50b5d5e287184714f9d1fd22b74ded588345cc2c571
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker run -d --name container-alpha --network net-alpha alpine sleep 3600
fde631303e41ee9a5b197ca9585a4c8edd5d35c0fb517a27aeadf637b2665da8
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker run -d --name container-beta --network net-beta alpine sleep 3600
c7b8a0f21a3bff911f563d4154caa79071aade7d59601c6e7004cd8156eb6c10
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker exec container-alpha ping -c 2 container-beta
ping: bad address 'container-beta'
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker network connect net-beta container-alpha
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker exec container-alpha ping -c 2 container-beta
PING container-beta (172.24.0.2): 56 data bytes
64 bytes from 172.24.0.2: seq=0 ttl=64 time=0.278 ms
64 bytes from 172.24.0.2: seq=1 ttl=64 time=0.112 ms

--- container-beta ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.112/0.195/0.278 ms
```

## Cleanup

*# Stop and remove all containers*

```
docker stop $(docker ps -aq)
```

```
docker rm $(docker ps -aq) # Remove all volumes
```

```
docker volume prune -f
```

*# Remove all networks (except defaults)*

```
docker network prune -f
```

*# Remove unused images*

```
docker image prune -f
```



```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker stop $(docker ps -aq)
c7b8a0f21a3b
fde631303e41
cdad3c04ee63
c8005de2d244
fb792096e68f
92575f41d7e4
32925e20406c
050da96780f0
1d328eb5448e
708483f81520
80f854b676cf
715f5e2a2eff
29c418346957
b676e3f53e24
f6c26ff4b69e
effb6290b130
f48d3903e3b9
29141488a78d
f50f9be6cd84
03c4cf37e2f1
30226fb52e2d
d78ade905b11
28fa911de65d
b8d612c5d607
f30d600335c4
4b7df3871506
c3997a9f7603
114d2f35452f
ed7fe287b01d
b2e2248e9820
6a55015d617b
d22d82d72605
99f29ea42b27
da51bce5710e
e2dec2279137
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker rm $(docker ps -aq)
c7b8a0f21a3b
fde631303e41
```

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker network prune -f
lab-net
net-alpha
myapp-network
ishnet
net-beta

ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$ docker image prune -f
Deleted Images:
untagged: sha256:19afe077783e8790985198158e199ac74a4aded019f9101aff2617a06548a649
deleted: sha256:19afe077783e8790985198158e199ac74a4aded019f9101aff2617a06548a649
deleted: sha256:9efeea20da4e079ffb058f27811dbab875f38add198cd12b66461360f1106bf6

Total reclaimed space: 2.834kB
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment5$
```

0 2 Update is ready, click to restart.

## Key Takeaways

- Volumes persist data beyond container lifecycle
- Environment variables configure containers dynamically
- Monitoring commands help debug and optimize containers
- Networks enable secure container communication
- Always use named volumes for production data
- Custom networks provide better isolation and DNS
- Monitor resource usage to prevent issues
- Use .env files for sensitive configuration
- This experiment covers essential Docker features for building, configuring, and managing production-ready containerized applications.

# ContainerizationAndDevOps

---

## Experiment 6A: Comparison of Docker Run and Docker Compose

---

### PART A – THEORY

---

#### 1. Objective

To understand the relationship between Docker run and Docker Compose, and compare their syntax and use cases.

#### 2. Background Theory

##### 2.1 Docker Run (Imperative Approach)

- Used to create and start containers.
- Requires manual flags for everything (Ports, Volumes, Networks, Restart policies, Resources, Names).

Example:

```
docker run -d \
 --name my-nginx \
 -p 8080:80 \
 -v ./html:/usr/share/nginx/html \
 -e NGINX_HOST=localhost \
 --restart unless-stopped \
 nginx:alpine
```

##### 2.2 Docker Compose (Declarative Approach)

- Uses `docker-compose.yml` to define services, networks, and volumes in a single file.
- One command ( `docker compose up -d` ) runs everything.

Equivalent Compose File:

```
version: '3.8'
services:
 nginx:
 image: nginx:alpine
 container_name: my-nginx
 ports:
 - "8080:80"
 volumes:
 - ./html:/usr/share/nginx/html
 environment:
 NGINX_HOST: localhost
 restart: unless-stopped
```

### 3. Mapping: Docker Run vs Compose

| Docker Run | Docker Compose                 |
|------------|--------------------------------|
| -p         | ports                          |
| -v         | volumes                        |
| -e         | environment                    |
| --name     | container_name                 |
| --network  | networks                       |
| --restart  | restart                        |
| --memory   | deploy.resources.limits.memory |
| --cpus     | deploy.resources.limits.cpus   |

### 4. Advantages of Docker Compose

- Multi-container support
- Reproducibility & Version control
- Unified lifecycle commands
- Scaling capability ( docker compose up --scale web=3 )

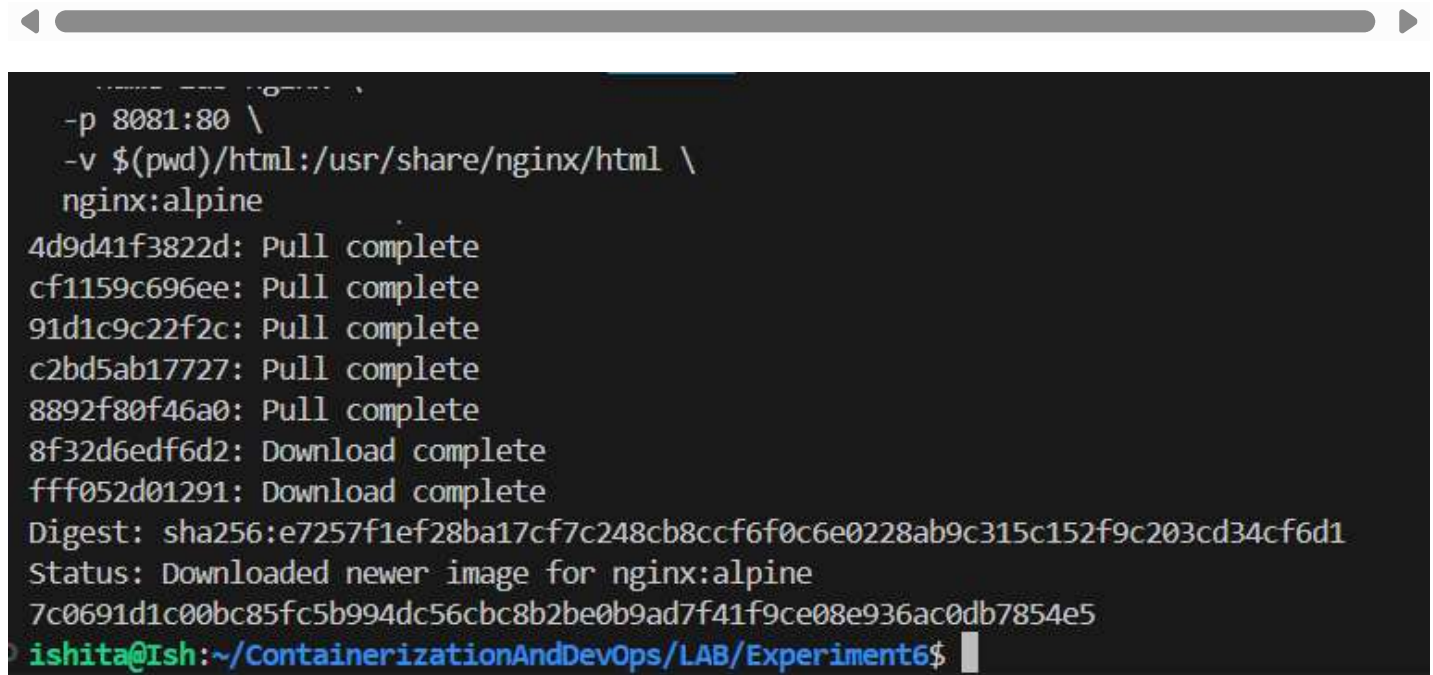
## PART B – PRACTICAL TASKS

### Task 1: Single Container Comparison



## Using Docker Run

```
docker run -d --name lab-nginx -p 8081:80 -v $(pwd)/html:/usr/share/nginx/html nginx:alpine
```



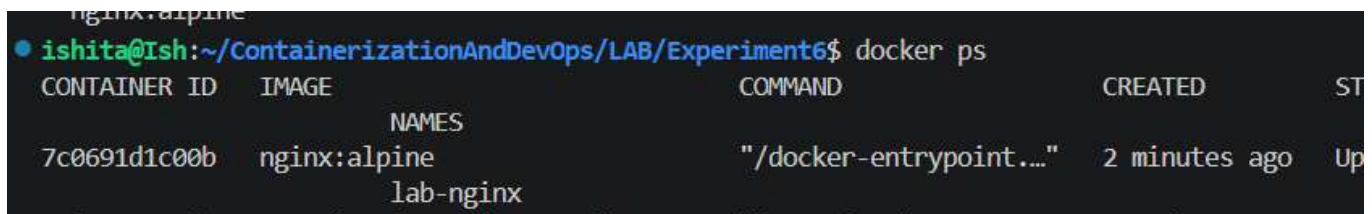
```

-p 8081:80 \
-v $(pwd)/html:/usr/share/nginx/html \
nginx:alpine
4d9d41f3822d: Pull complete
cf1159c696ee: Pull complete
91d1c9c22f2c: Pull complete
c2bd5ab17727: Pull complete
8892f80f46a0: Pull complete
8f32d6edf6d2: Download complete
fff052d01291: Download complete
Digest: sha256:e7257f1ef28ba17cf7c248cb8ccf6f0c6e0228ab9c315c152f9c203cd34cf6d1
Status: Downloaded newer image for nginx:alpine
7c0691d1c00bc85fc5b994dc56cbc8b2be0b9ad7f41f9ce08e936ac0db7854e5
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment6$

```

- verifying by using

```
docker ps
```



```

nginx:alpine
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment6$ docker ps
CONTAINER ID IMAGE NAMES COMMAND CREATED STATUS
7c0691d1c00b nginx:alpine lab-nginx "/docker-entrypoint..." 2 minutes ago Up

```

- Accessing via

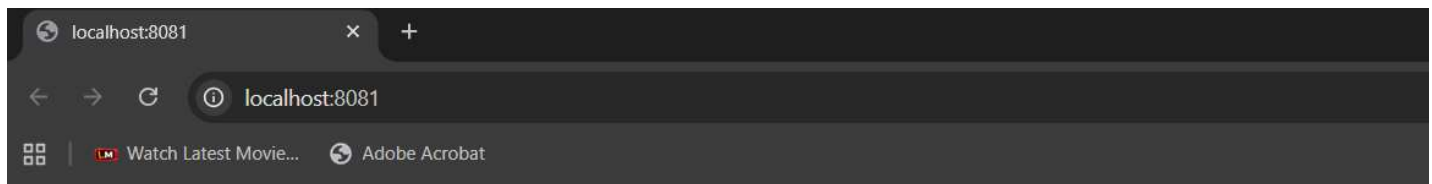
```
curl http://localhost:8081
```



```

ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment6$ echo "<h1>Experiment 6 by Ishita</h1>" > html/index.html
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment6$ curl http://localhost:8081
<h1>Experiment 6 by Ishita</h1>
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment6$

```



## Experiment 6 by Ishita

### Using Docker Compose

#### docker-compose.yml

```
version: '3.8'
services:
 nginx:
 image: nginx:alpine
 container_name: lab-nginx
 ports:
 - "8081:80"
 volumes:
 - ./html:/usr/share/nginx/html
```

```
docker compose up -d
docker compose ps
docker compose down
```

```

ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment6$ docker compose up -d
WARN[0000] /home/ishita/ContainerizationAndDevOps/LAB/Experiment6/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
WARN[0000] No services to build
[+] up 2/2
✓ Network experiment6_default Created 0.1s
✓ Container lab-nginx Created 0.2s
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment6$ docker compose ps
WARN[0000] /home/ishita/ContainerizationAndDevOps/LAB/Experiment6/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion

```

| NAME      | IMAGE        | COMMAND                 | SERVICE | CREATED       | STATUS       | PORTS                                   |
|-----------|--------------|-------------------------|---------|---------------|--------------|-----------------------------------------|
| lab-nginx | nginx:alpine | "/docker-entrypoint..." | nginx   | 8 seconds ago | Up 7 seconds | 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp |

```

ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment6$

```

## Task 2: Multi-Container App (WordPress + MySQL)

### Using Docker Run (Manual approach)

```
docker network create wp-net
```

```
docker run -d --name mysql --network wp-net -e MYSQL_ROOT_PASSWORD=secret -e MYSQL_DATABASE=wordpress mysql:5.7
```

```
docker run -d --name wordpress --network wp-net -p 8082:80 -e WORDPRESS_DB_HOST=mysql -e WORDPRESS_DB_PASSWORD=secret -e WORDPRESS_DB_NAME=wordpress wordpress:latest
```

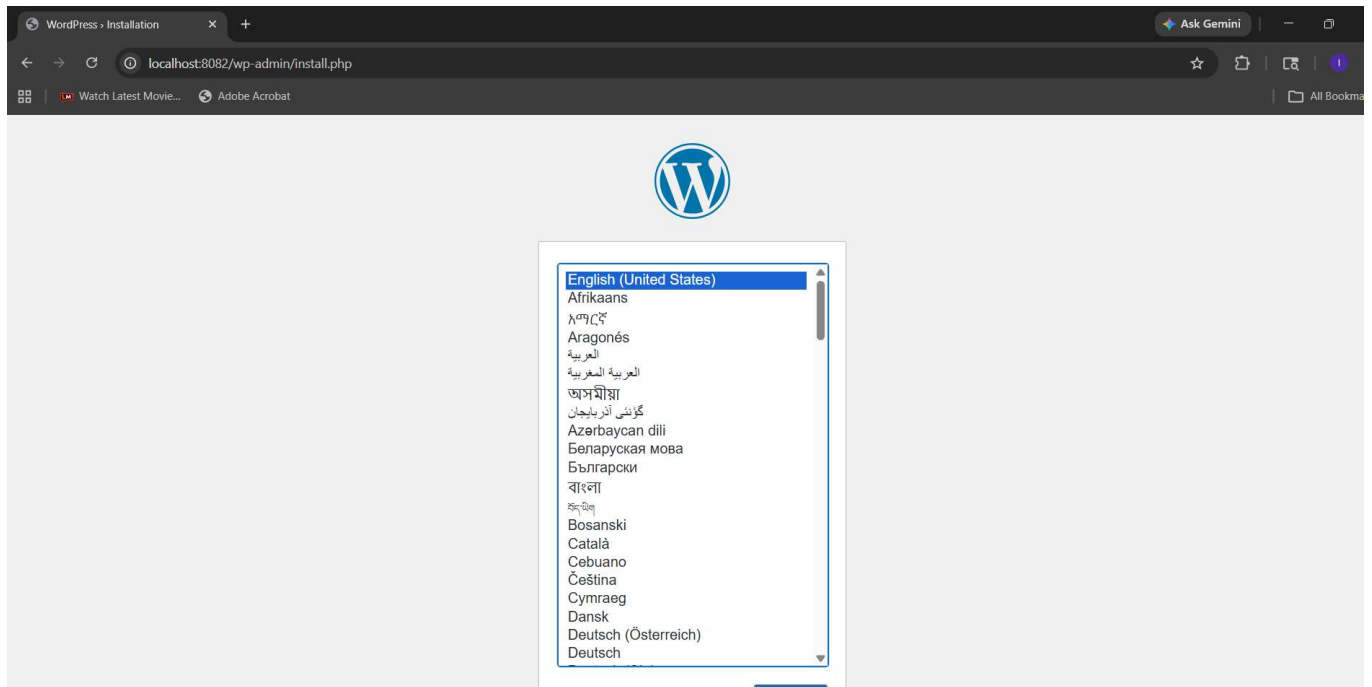
```

ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment6$ docker network create wp-net
e60f67a102e7d0810a0f8f9f57a9c1d48a0f6176b009fcf2128626ce2be8411f
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment6$ docker run -d --name mysql --network wp-net -e MYSQL_ROOT_PASSWORD=secret -e MYSQL_DATABASE=wordpress mysql:5.7
174e004550bb04659ba8cef4596cc173d7280b7d9bffd7bfa130e9bdfb152f3c
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment6$ docker run -d --name wordpress --network wp-net -p 8082:80 -e WORDPRESS_DB_HOST=mysql -e WORDPRESS_DB_PASSWORD=secret -e WORDPRESS_DB_NAME=wordpress wordpress:latest
dd4ab4543c81ca727eaf09cd618c13360ae0e8fb36a0c48d5c12922eb26a68ca

```

- Accessing using

<http://localhost:8082>



## Experiment 6B: WordPress + MySQL using Compose

Using a single Compose file completely replaces the manual networking and container execution from Task 2.

### docker-compose.yml

```
version: '3.9'
services:
 db:
 image: mysql:5.7
 container_name: wordpress_db
 restart: always
 environment:
 MYSQL_ROOT_PASSWORD: rootpass
 MYSQL_DATABASE: wordpress
 MYSQL_USER: wpuser
 MYSQL_PASSWORD: wppass
 volumes:
 - db_data:/var/lib/mysql

 wordpress:
 image: wordpress:latest
 container_name: wordpress_app
 depends_on:
 - db
```



```

ports:
 - "8080:80"
restart: always
environment:
 WORDPRESS_DB_HOST: db:3306
 WORDPRESS_DB_USER: wpuser
 WORDPRESS_DB_PASSWORD: wppass
 WORDPRESS_DB_NAME: wordpress
volumes:
 - wp_data:/var/www/html

```

```

volumes:
 db_data:
 wp_data:

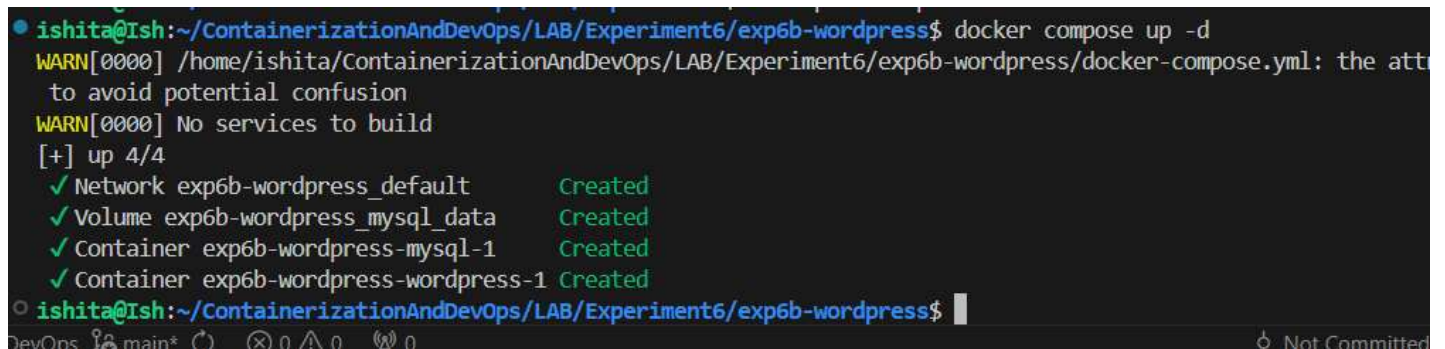
```

## 1. Run the Stack

```

docker compose up -d
docker ps

```



```

ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment6/exp6b-wordpress$ docker compose up -d
WARN[0000] /home/ishita/ContainerizationAndDevOps/LAB/Experiment6/exp6b-wordpress/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
WARN[0000] No services to build
[+] up 4/4
✓ Network exp6b-wordpress_default Created
✓ Volume exp6b-wordpress_mysql_data Created
✓ Container exp6b-wordpress-mysql-1 Created
✓ Container exp6b-wordpress-wordpress-1 Created
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment6/exp6b-wordpress$

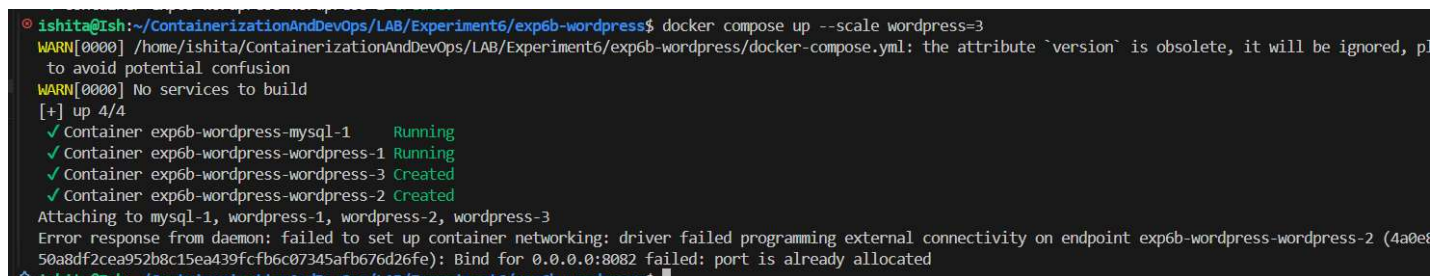
```

## 2. Scaling

```

docker compose up --scale wordpress=3

```



```

ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment6/exp6b-wordpress$ docker compose up --scale wordpress=3
WARN[0000] /home/ishita/ContainerizationAndDevOps/LAB/Experiment6/exp6b-wordpress/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
WARN[0000] No services to build
[+] up 4/4
✓ Container exp6b-wordpress-mysql-1 Running
✓ Container exp6b-wordpress-wordpress-1 Running
✓ Container exp6b-wordpress-wordpress-3 Created
✓ Container exp6b-wordpress-wordpress-2 Created
Attaching to mysql-1, wordpress-1, wordpress-2, wordpress-3
Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint exp6b-wordpress-wordpress-2 (4a0e550a8df2cea952b8c15ea439fcfb6c07345afb676d26fe): Bind for 0.0.0.0:8082 failed: port is already allocated
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment6/exp6b-wordpress$

```

**Important Learning Outcome:** The scaling intentionally throws an error because standard Docker Compose is limited here. Since `container_name: wordpress_app` and `ports: "8080:80"`

are hardcoded in the file, Compose cannot launch 3 replicas that all want the identical name and host port. This demonstrates the exact scenario where **Docker Swarm** (which handles dynamic naming and load balances identical ports) becomes necessary for production!

## PART B- Multicontainer App with Swarm

1. The application follows a standard tiered architecture:

- User Layer: Accesses the application via a web browser on a specific host port.
- WordPress Container: Handles the PHP processing and frontend. It connects to the database using the service name (db) as the hostname.
- SQL Container: The backend database that stores all site content.
- Persistent Volume: A dedicated storage area on the host system that ensures data remains intact even if containers are deleted.

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment6/exp6b-wordpress$ docker stack deploy -c docker-compose.yml wpstack
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network wpstack_default
Creating service wpstack_mysql
Creating service wpstack_wordpress
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment6/exp6b-wordpress$ docker service scale wpstack_wordpress=3
wpstack_wordpress scaled to 3
overall progress: 3 out of 3 tasks
1/3: running [=====>]
2/3: running [=====>]
3/3: running [=====>]
verify: Service wpstack_wordpress converged
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment6/exp6b-wordpress$
```

## PART C – CONVERSION TASKS (Theory)

Converting manual `docker run` commands and configurations into declarative `docker-compose.yml` configurations:

### Task 3: Convert Run → Compose

Given Run Command:

```
docker run -d --name webapp -p 5000:5000 -e APP_ENV=production -e DEBUG=false --restart unless-stopped
```

## Converted Compose File:

```
version: '3.8'
services:
 webapp:
 image: node:18-alpine
 container_name: webapp
 ports:
 - "5000:5000"
 environment:
 APP_ENV: production
 DEBUG: false
 restart: unless-stopped
```

```
ishita@ishita:~/ContainerizationAndDevOps/LAB/Experiment6/exp6b-wordpress$ docker compose up -d
WARN[0000] /home/ishita/ContainerizationAndDevOps/LAB/Experiment6/exp6b-wordpress/docker-compose.yml: the attribute `version` is obsolete, it will be i
to avoid potential confusion
WARN[0000] No services to build
[+] up 4/4
✓ Network exp6b-wordpress_default Created
✓ Volume exp6b-wordpress_mysql_data Created
✓ Container exp6b-wordpress-mysql-1 Created
✓ Container exp6b-wordpress-wordpress-1 Created
ishita@ishita:~/ContainerizationAndDevOps/LAB/Experiment6/exp6b-wordpress$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS
6e7a2bac934b wordpress:latest "docker-entrypoint.s..." 11 seconds ago Up 10 seconds 0.0.0.0:8082->80/tcp, [::]:8082->80/tcp
eb68ee0eb48f mysql:5.7 "docker-entrypoint.s..." 11 seconds ago Up 10 seconds 3306/tcp, 33060/tcp
```

## Task 4 & 5: Volumes, Networks, and Resource Limits

- the docker run command to be converted in compose ``docker docker run -d  
 -name limited-app  
 -p 9000:9000  
 -memory="256m"  
 -cpus="0.5"  
 -restart always  
 nginx:alpine

- The equivalent compose file is as follows

```
``docker
services:
 limited-app:
 image: nginx:alpine
 container_name: limited-app
 ports:
 - "9000:9000"
 restart: always
 deploy:
```

```
resources:
 limits:
 cpus: '0.50'
 memory: 256M
```

- How deploy works:
- Compose V3 (Legacy): Originally the deploy section was strictly for Docker Swarm. If we ran docker-compose up, these settings were ignored unless you used docker stack deploy.
- Compose Specification (Modern): In current versions of Docker Compose.. the deploy key is automatically honored for resource limits even in local, non-swarm environments. There's no longer need of the --compatibility flag that was required in older versions.

## Docker Compose vs Swarm Comparison

| Feature                           | Compose  | Swarm          | Scope | Single host | Multi-node   | Scaling |
|-----------------------------------|----------|----------------|-------|-------------|--------------|---------|
| Manual / Limited by port bindings | Built-in | Load balancing | No    | Yes         | Self-healing | No      |
|                                   |          |                | Yes   |             | Yes          | Yes     |

## PART D – DOCKERFILE WITH COMPOSE

We can combine custom image building with Compose orchestration.

app.js

```
const http = require('http');
http.createServer((req, res) => {
 res.end("Docker Compose Build Lab");
}).listen(3000);
```

Dockerfile

```
FROM node:18-alpine
WORKDIR /app
COPY app.js .
EXPOSE 3000
CMD ["node", "app.js"]
```

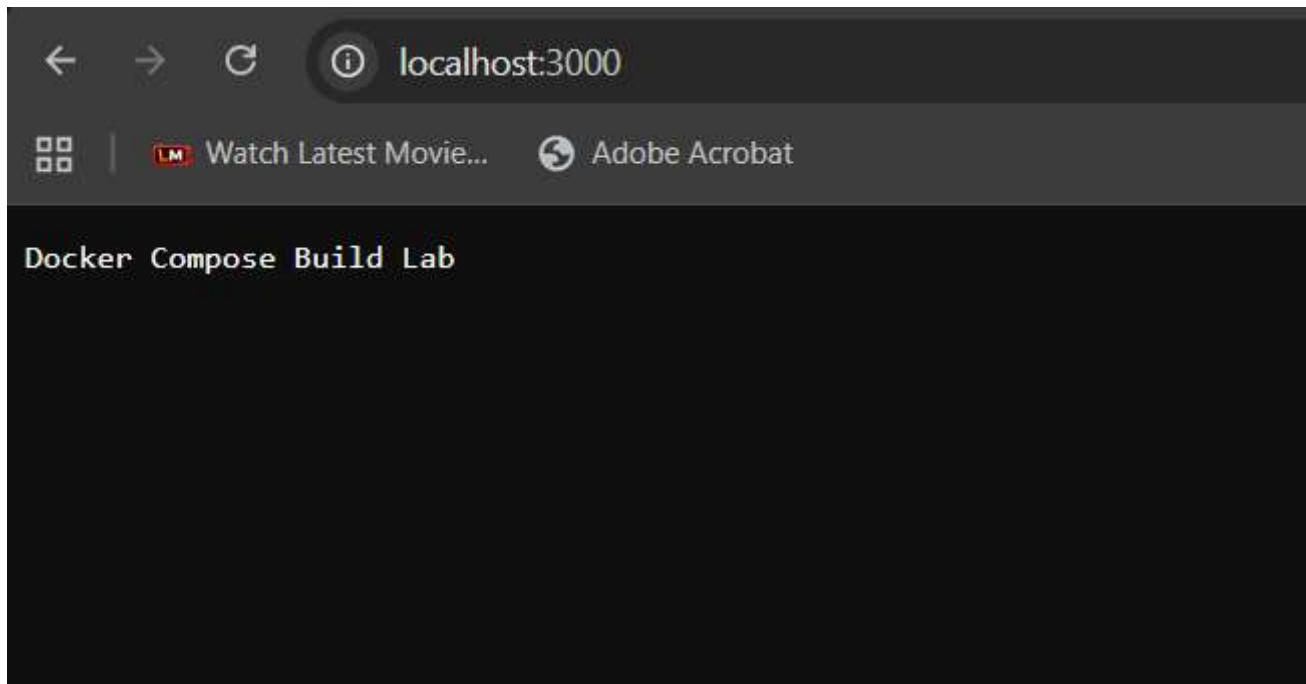
docker-compose.yml

```
version: '3.8'
services:
```

```
nodeapp:
 build:
 context: .
 dockerfile: Dockerfile
 container_name: custom-node-app
 ports:
 - "3000:3000"
```

```
docker compose up --build -d
curl http://localhost:3000
```

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment6/PartD$ docker compose up --build -d
WARN[0000] /home/ishita/ContainerizationAndDevOps/LAB/Experiment6/PartD/docker-compose.yml: the attribute `version` is obsolete, it w
potential confusion
[+] Building 3.8s (11/11) FINISHED
=> [internal] load local bake definitions
=> => reading from stdin 572B
=> [internal] load build definition from Dockerfile
```



## Key Takeaways

1. **Compose simplifies multi-container apps** by keeping settings in version control.
2. **Declarative approach** is much easier than manually tracking `docker run` flags.
3. **Compose is excellent for dev/test**, but limited in native scaling due to port/name collisions without strict dynamic tuning.
4. **Docker Swarm** is much better for production cluster orchestration, load balancing, and self-healing.



# ContainerizationAndDevOps

---

## Lab Experiment 7: CI/CD Pipeline using Jenkins, GitHub, and Docker Hub

---

### 1. Aim

---

To design and implement a complete CI/CD pipeline using **Jenkins**, integrating source code from **GitHub**, and automating the building and pushing of Docker images to **Docker Hub**.

### 2. Objectives

---

- Understand the CI/CD workflow using Jenkins (GUI-based tool).
- Create a structured GitHub repository with application code and a `Jenkinsfile`.
- Build Docker images from source code automatically.
- Securely store Docker Hub credentials in Jenkins.
- Automate the build and push process using GitHub Webhook triggers.
- Use the same host (Docker) as the Jenkins agent by mounting the Docker socket.

### 3. Implementation Details

---

#### 3.1 Project Structure

The project is structured to follow a "Pipeline as Code" approach:

- `app.py` : Flask-based Python application.
- `requirements.txt` : Python library dependencies.
- `Dockerfile` : Instructions for containerizing the app.
- `Jenkinsfile` : Defines the CI/CD pipeline stages.

#### Part A

#### 3.2 Application Code ( `app.py` )

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
 return "Hello from CI/CD Pipeline!, my sapid is 500119435"

if __name__ == "__main__":
 app.run(host="0.0.0.0", port=80)
```

## Dockerfile

```
FROM python:3.10-slim

WORKDIR /app
COPY . .

RUN pip install -r requirements.txt

EXPOSE 80
CMD ["python", "app.py"]
```

## Jenkinsfile

```
pipeline {
 agent any

 environment {
 IMAGE_NAME = "ishcosmo/my-app"
 }

 stages {
 stage('Clone Source') {
 steps {
 git: 'https://github.com/ishcosmo/my-app.git'
 }
 }

 stage('Build Docker Image') {
 steps {
 sh 'docker build -t $IMAGE_NAME:latest .'
 }
 }
 }
}
```

```

 stage('Login to Docker Hub') {
 steps {
 withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_T
 sh 'echo $DOCKER_TOKEN | docker login -u ishcosmo --password-stdin'
 }
 }
 }

 stage('Push to Docker Hub') {
 steps {
 sh 'docker push $IMAGE_NAME:latest'
 }
 }
}
}
}

```

## PART B: Jenkins setup using Docker

- A *my-app* repository with all the files (dockerfile, composefile, Jenkinsfile, requirements file etc) was made for the setup with content as previously
- Start and Access Jenkins using `` docker-compose up -d



- For Access `http://localhost:8080`
- Unlock Jenkins using:

`docker exec -it jenkins cat /var/jenkins_home/secrets/initialAdminPassword`





- followed by creating a new user (admin user)









### PART C & D: Jenkins Configuration and Webhook Creation

- Adding Dockerhub credentials and then creating a pipelining job







Since the Jenkins instance is running within **\*\*WSL (Windows Subsystem for Linux)\*\***, it is not

### ### 4.1 Localtunnel Implementation

To bypass command collisions and ensure a stable connection, the following commands were used

```
1. **Installation:**
``bash
npm install -g localtunnel
```

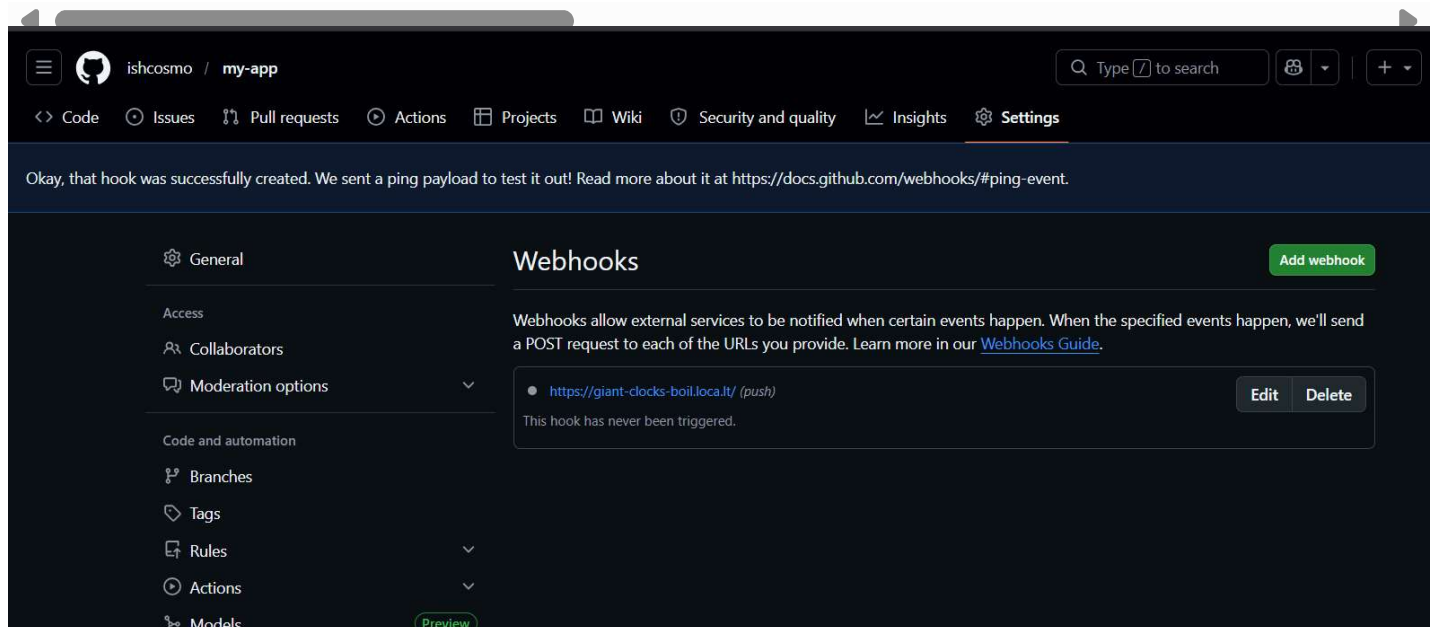
1. Establishing the Tunnel: Exposing Jenkins (running on port 8080) to the internet:

```
npx localtunnel --port 8080
```

2. To automate the build process a Webhook was created in the GitHub repository settings:

- Payload URL: Formatted as: <https://bright-tires-study.loca.lt/github-webhook/> (mine)
- Content Type: Changed from default to application/json for better compatibility with Jenkins.
- Trigger Events: Selected "Just the push event".

Verification: Upon clicking "Add Webhook," GitHub sent a ping payload. A Green Checkmark appeared, confirming that the tunnel successfully routed the request to the Jenkins container.



The screenshot shows the Jenkins web interface for a pipeline named 'ci-cd-pipeline'. On the left, there is a sidebar with navigation links: Status (selected), Changes, Build Now, Configure, Delete Pipeline, Stages, Rename, Pipeline Syntax, and GitHub Hook Log. The main area displays the pipeline's status as 'lab 7 exp' and includes a 'Permalinks' section. At the bottom, there is a 'Builds' section with a right arrow.

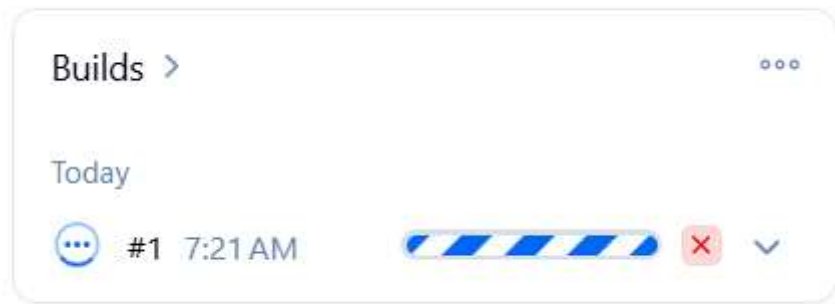
This screenshot shows the Jenkins 'Webhooks' configuration page. It explains that webhooks allow external services to be notified when certain events happen, and that Jenkins will send a POST request to the specified URLs. A single webhook is configured with the URL 'https://bright-tires-study.loca.lt/git...' and the event '(push)'. The status indicates 'Last delivery was successful.' There are 'Edit' and 'Delete' buttons for the configured webhook.

## PART E: Execution Flow

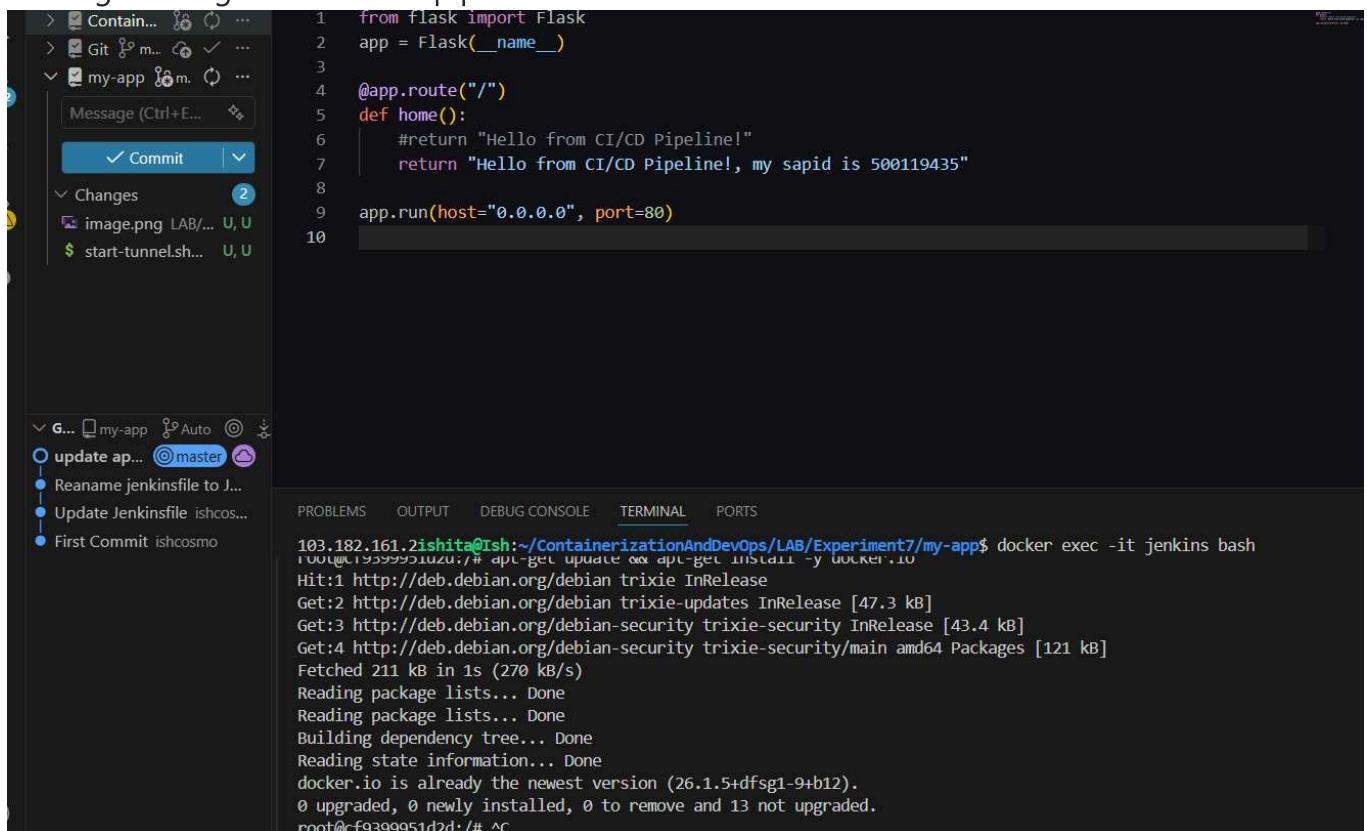
| Stage   | Action             | Description                                                                                           |
|---------|--------------------|-------------------------------------------------------------------------------------------------------|
| Stage 1 | Code Push          | Developer pushes code changes to the GitHub repository                                                |
| Stage 2 | Webhook Trigger    | GitHub detects the push and sends a JSON payload to the Localtunnel URL.                              |
| Stage 3 | Jenkins Processing | Jenkins receives the signal, matches the repo URL, and initiates the pipeline.                        |
| Stage 4 | Clone & Build      | Jenkins pulls the code and builds the Docker image via the host's <code>/var/run/docker.sock</code> . |
| Stage 5 | Push to Registry   | The image is authenticated and pushed to the <code>ishcosmo/my-app</code> Docker Hub repository.      |

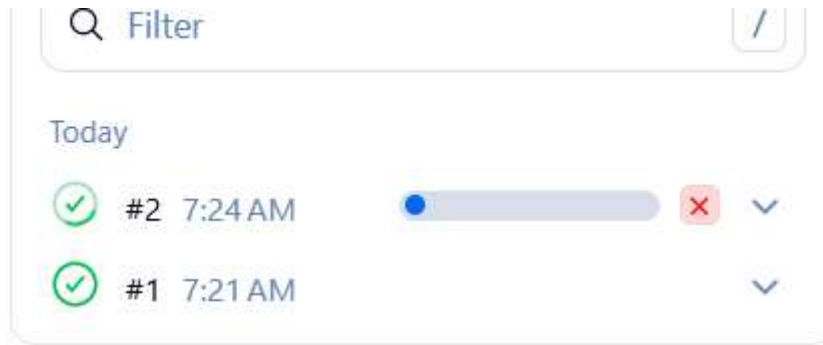


- Build in Process



- Making a change to see if the pipeline works





- execution build

 **#2 (Apr 11, 2026, 7:24:38 AM)**

 Started by [GitHub push by ishcosmo](#)

 This run spent:

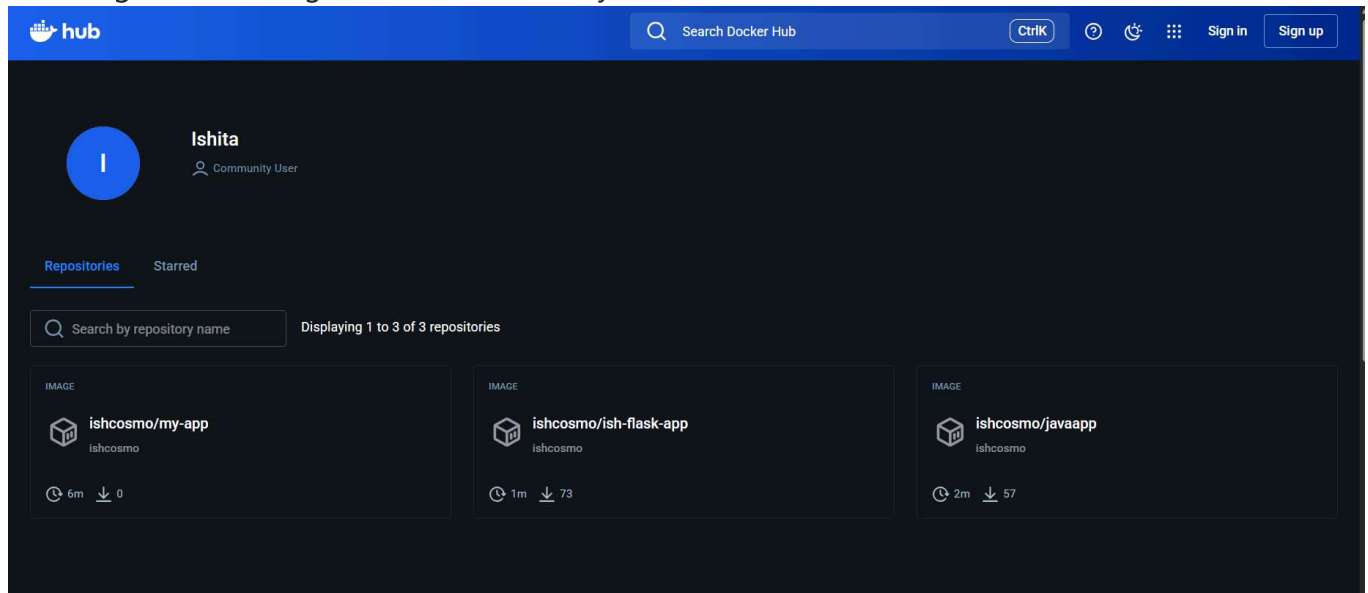
- 5.1 sec waiting;
- 31 sec build duration;
- 36 sec total from scheduled to completion.

 **Revision:** d374007ee007c5657cdce5d50b542c8b797b5970  
**Repository:** <https://github.com/ishcosmo/my-app.git>

- refs/remotes/origin/master

  update app.py d374007  
|  Ishita at 7:24 AM 4/11/26

- Last stage where image can be seen on my dockerhub



## Conclusion

- The implementation of this CI/CD pipeline successfully automates the transition from code to container.
- The use of Localtunnel effectively bridged the gap between a local WSL development environment and cloud-based triggers, while the "Same Host Agent" approach allowed Jenkins to perform high-performance Docker operations without nested virtualization overhead.

# ContainerizationAndDevOps

## Experiment 9: Ansible

### Theory

#### Problem Statement

Managing infrastructure manually across multiple servers leads to configuration drift, inconsistent environments, and time-consuming repetitive tasks. Scaling from one server to hundreds becomes nearly impossible with manual SSH-based administration.

#### What is Ansible?

Ansible is an open-source automation tool for configuration management, application deployment, and orchestration.

- Follows an **agentless architecture**, using SSH for Linux and WinRM for Windows.
- Uses **YAML-based playbooks** to define automation tasks.
- Ansible enables cross-platform automation and orchestration at scale and has become the standard choice among enterprise automation solutions.

#### How Ansible Solves the Problem

- **Agentless Architecture:** No software installation required on managed nodes
- **Idempotency:** Running playbooks multiple times yields the same result
- **Declarative Syntax:** Describe desired state, not the steps to achieve it
- **Push-based:** Initiates changes from control node immediately

#### Key Concepts

| Component                              | Description                                                      |
|----------------------------------------|------------------------------------------------------------------|
| Control Node                           | Machine with Ansible installed                                   |
| Managed Nodes                          | Target servers (no Ansible agent needed)                         |
| Inventory / <code>inventory.ini</code> | Defines the list of managed nodes (EC2 instances, servers, etc.) |

| Component | Description                                                                                                 |
|-----------|-------------------------------------------------------------------------------------------------------------|
| Playbooks | YAML files containing a sequence of automation steps                                                        |
| Tasks     | Individual actions in playbooks (e.g., installing a package)                                                |
| Modules   | Built-in functionality to perform tasks (e.g., <code>yum</code> , <code>apt</code> , <code>service</code> ) |
| Roles     | Pre-defined reusable automation scripts                                                                     |

## How Does Ansible Work?

Ansible connects from the **control node** to the **managed nodes**, sending commands and instructions. Units of code executed on managed nodes are called **modules**. Each module is invoked by a **task**, and an ordered list of tasks forms a **playbook**. Managed machines are represented in a simple **inventory file** that groups nodes into categories. Ansible uses YAML — a human-readable format — and requires no extra agents on managed nodes.

## Part A — Ansible Installation on Windows (via WSL)

**Note:** On Windows, Ansible runs through **Windows Subsystem for Linux (WSL)**. All commands below are executed inside a WSL terminal (Ubuntu).

### Step 1: Install Ansible inside WSL

```
Update packages
sudo apt update -y

Install Ansible
sudo apt install ansible -y

Verify installation
ansible --version
```



```

ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment9$ sudo apt update && sudo apt upgrade -y
Processing triggers for rsyslog (8.2312.0-3ubuntu9.1) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libglib2.0-0t64:amd64 (2.80.0-6ubuntu3.8) ...
Processing triggers for dbus (1.14.10-4ubuntu4.1) ...
Processing triggers for install-info (7.1-3build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment9$ sudo apt install ansible -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:

```

## Step 2: Test Local Ping

```
ansible localhost -m ping
```

### Expected Output:

```

localhost | SUCCESS => {
 "changed": false,
 "ping": "pong"
}

```

```

ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment9$
ansible localhost -m ping
[WARNING]: No inventory was parsed, only implicit localhost is available
localhost | SUCCESS => {
 "changed": false,
 "ping": "pong"
}

```

```

ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment9$ ansible --version
ansible [core 2.16.3]
 config file = None
 configured module search path = ['/home/ishita/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
 ansible python module location = /usr/lib/python3/dist-packages/ansible
 ansible collection location = /home/ishita/.ansible/collections:/usr/share/ansible/collections
 executable location = /usr/bin/ansible
 python version = 3.12.3 (main, Mar 3 2026, 12:15:18) [GCC 13.3.0] (/usr/bin/python3)
 jinja version = 3.1.2
 libyaml = True
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment9$

```

## Ansible Demo — Docker Containers as Servers

## Create SSH Key Pair in WSL

```
Generate RSA key pair (accept defaults when prompted)
ssh-keygen -t rsa -b 4096

This creates:
Private key: ~/.ssh/id_rsa
Public key: ~/.ssh/id_rsa.pub

Copy keys to current directory to be added to Docker images
cp ~/.ssh/id_rsa.pub .
cp ~/.ssh/id_rsa .
```

**Windows Note:** Keep your keys inside the WSL filesystem ( `~/.ssh/` ). Placing them on the Windows NTFS filesystem ( `/mnt/c/...` ) can cause permission issues with SSH.

### Key placement summary:

| File                    | Location                                                 | Purpose                                                                   |
|-------------------------|----------------------------------------------------------|---------------------------------------------------------------------------|
| id_rsa (Private Key)    | Your local WSL machine                                   | Used to authenticate when connecting to servers. <b>Never share this!</b> |
| id_rsa.pub (Public Key) | Remote server<br>( <code>~/.ssh/authorized_keys</code> ) | Grants access to anyone with the matching private key                     |

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment9$ ssh-keygen -t rsa -b 4096 -f ~/.ssh/id_rsa -N ""
cp ~/.ssh/id_rsa.pub .
cp ~/.ssh/id_rsa .
Generating public/private rsa key pair.
Created directory '/home/ishita/.ssh'.
Your identification has been saved in /home/ishita/.ssh/id_rsa
Your public key has been saved in /home/ishita/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:LerkeOCouCPJoIHV/QZ4JRLon6ebBCKJ40tO+o5hY14 ishita@Ish
```

## Create Dockerfile for Ubuntu SSH Server

Create a file named `Dockerfile` in your working directory:

```
FROM ubuntu
RUN apt update -y
RUN apt install -y python3 python3-pip openssh-server
```

```
RUN mkdir -p /var/run/sshd

Configure SSH
RUN mkdir -p /run/sshd && \
 echo 'root:password' | chpasswd && \
 sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config &
 sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config &&
 sed -i 's/#PubkeyAuthentication yes/PubkeyAuthentication yes/' /etc/ssh/sshd_config

Create .ssh directory and set proper permissions
RUN mkdir -p /root/.ssh && \
 chmod 700 /root/.ssh

Copy SSH keys
COPY id_rsa /root/.ssh/id_rsa
COPY id_rsa.pub /root/.ssh/authorized_keys

Set proper permissions for keys
RUN chmod 600 /root/.ssh/id_rsa && \
 chmod 644 /root/.ssh/authorized_keys

Fix for SSH Login
RUN sed -i 's@session\s*required\s*pam_loginuid.so@session optional pam_loginuid.so@g' /etc/p

Expose SSH port
EXPOSE 22

Start SSH service when container starts
CMD ["/usr/sbin/sshd", "-D"]
```

## Build the Docker Image



```
Build the Docker image
docker build -t ubuntu-server .

Remove the public key from build directory (optional)
rm id_rsa.pub
```

```

$ docker build -t ubuntu-server .
[+] Building 155.0s (15/15) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 1.10kB
=> [internal] load metadata for docker.io/library/ubuntu:latest
=> [internal] load .dockerignore
=> => transferring context: 2B
=> CACHED [1/10] FROM docker.io/library/ubuntu:latest@sha256:cd1dba651b3080c3686ecf4e3c4220f026b521fb76978881
=> => resolve docker.io/library/ubuntu:latest@sha256:cd1dba651b3080c3686ecf4e3c4220f026b521fb76978881
=> [internal] load build context
=> => transferring context: 3.43kB

```

## Run the Docker Container

```
docker run -d -p 2222:22 --rm -p 8221:8221 --name ssh-test-server ubuntu-server
```

```

$ docker run -d --rm -p 2222:22 -p 8221:8221 --name ssh-test-server ubuntu-server
452e80839af3514f2e5b39c0823200dae36b5c16cff3b7136654c639b1fcae99

```

## Find the Container IP Address

```
docker inspect -f '{{.NetworkSettings.Networks}}' ssh-test-server
```

Note this IP address (e.g., 172.17.0.4 ).

```

$ docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' ssh-test-server
172.17.0.4

```

## Test SSH Connections

### Key-based authentication (recommended):

```
ssh -i ~/.ssh/id_rsa root@localhost -p 2222
Should log in without a password prompt
```

### Using container IP directly:

```
ssh -i ~/.ssh/id_rsa root@172.17.0.2
```

**Windows Note:** Always use `~/.ssh/id_rsa` (WSL path) rather than `/mnt/c/...` to avoid NTFS permission errors.

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment9$ ssh root@localhost -p 2222
* Support: https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

root@452e80839af3:~#
```

## Clean Up

```
docker stop ssh-test-server
docker rm ssh-test-server
```

## Ansible with Docker Exercise

---

### Step 1: Start 4 Server Containers

```
for i in {1..4}; do
 echo -e "\n Creating server${i}\n"
 docker run -d --rm -p 220${i}:22 --name server${i} ubuntu-server
 echo -e "IP of server${i} is $(docker inspect -f ' ' server${i})"
done
```



```

Error: Response from daemon: No such container: ssh-test
• ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment9$ for i in {1..4}; do
 echo -e "\n Creating server${i}\n"
 docker run -d --rm -p 220${i}:22 --name server${i} ubuntu-server
 echo -e "IP of server${i} is $(docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' server${i})"
done

Creating server1

3334a1025f383e7fe53f6c248625a68cc2f230a12fb5445e5f0f34713ea01c44
IP of server1 is 172.17.0.5

Creating server2

02707d46e1b7330988a15f51b785d242ba0a746d73b8e00b188b8b2b969f9f1e
IP of server2 is 172.17.0.6

Creating server3

89118c1b6786a5072e31baf98f42cda9b4186f11acf2bea744d34a242070afd9
IP of server3 is 172.17.0.7

Creating server4

a06777e3e826c7c6778968935780cf766ffac8b2a54734ae3f98a76a827fd255
IP of server4 is 172.17.0.8

```

## Step 2: Create Ansible Inventory ( inventory.ini )

```

Get container IPs and write inventory
echo "[servers]" > inventory.ini
for i in {1..4}; do
 docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' server${i} >> inventory.ini
done

Add inventory variables
cat << EOF >> inventory.ini
[servers:vars]
ansible_user=root
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
EOF

```

```

• ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment9$ echo "[servers]" > inventory.ini
for i in {1..4}; do
 docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' server${i} >> inventory.ini
done
• ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment9$ cat << EOF >> inventory.ini

[servers:vars]
ansible_user=root
; ansible_ssh_private_key_file=./id_rsa
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
EOF
• ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment9$

```

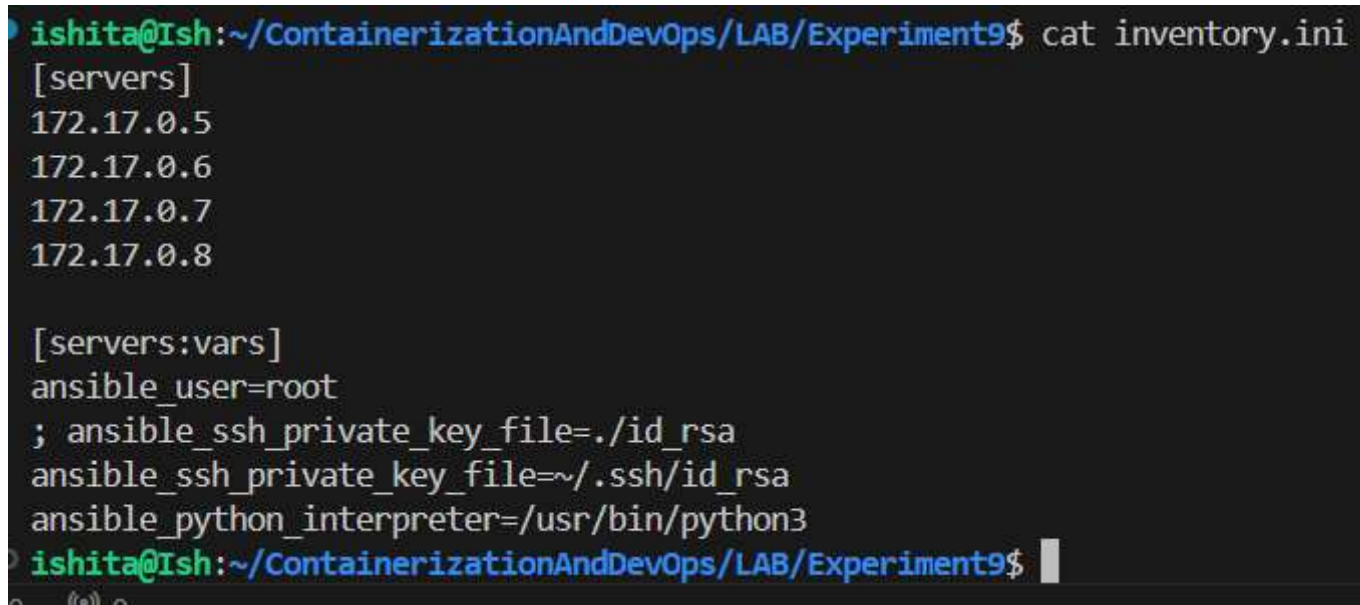
## Step 3: Review `inventory.ini`

```
cat inventory.ini
```

It should look like:

```
[servers]
172.17.0.3
172.17.0.4
172.17.0.5
172.17.0.6

[servers:vars]
ansible_user=root
; ansible_ssh_private_key_file=./id_rsa
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
```

A terminal window with a dark background. The prompt is 'ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment9\$'. The command 'cat inventory.ini' has been executed, displaying the following content:

```
[servers]
172.17.0.5
172.17.0.6
172.17.0.7
172.17.0.8

[servers:vars]
ansible_user=root
; ansible_ssh_private_key_file=./id_rsa
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
```

The prompt is now 'ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment9\$' with a cursor.

## Step 4: Test Connectivity

```
Ansible ping test
ansible all -i inventory.ini -m ping

Verbose mode (for troubleshooting)
ansible all -i inventory.ini -m ping -vvv
```

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment9$ ansible all -i inventory.ini -m ping
server4 | SUCCESS => {
 "changed": false,
 "ping": "pong"
}
server3 | SUCCESS => {
 "changed": false,
 "ping": "pong"
}
server1 | SUCCESS => {
 "changed": false,
 "ping": "pong"
}
server2 | SUCCESS => {
 "changed": false,
 "ping": "pong"
}
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment9$
```

## Step 5: Create Playbook ( playbook1.yml )

The YAML file must start with exactly three dashes: ---

```

- name: Update and configure servers
 hosts: all
 become: yes
 tasks:
 - name: Update apt packages
 apt:
 update_cache: yes
 upgrade: dist

 - name: Install required packages
 apt:
 name: ["vim", "htop", "wget"]
 state: present

 - name: Create test file
 copy:
 dest: /root/ansible_test.txt
 content: "Configured by Ansible on "
```

## Step 6: Run the Playbook

```
ansible-playbook -i inventory.ini playbook1.yml
```

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment9$ ansible-playbook -i inventory.ini playbook1.yml
PLAY [Update and configure servers] *****
TASK [Gathering Facts] *****
ok: [server3]
ok: [server4]
ok: [server1]
ok: [server2]
TASK [Update apt packages] *****
```

## Step 7: Verify Changes

```
Using Ansible
ansible all -i inventory.ini -m command -a "cat /root/ansible_test.txt"

Manually via Docker
for i in {1..4}; do
 docker exec server${i} cat /root/ansible_test.txt
done
```

Screenshot — Verification output showing test file contents:

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment9$ ansible all -i inventory.ini -m command -a "cat /root/ansible_test.txt"
server3 | CHANGED | rc=0 >>
Configured by Ansible on server3
server4 | CHANGED | rc=0 >>
Configured by Ansible on server4
server1 | CHANGED | rc=0 >>
Configured by Ansible on server1
server2 | CHANGED | rc=0 >>
Configured by Ansible on server2
```

## Step 8: Cleanup

```
for i in {1..4}; do docker rm -f server${i}; done
```

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment9$ for i in {1..4}; do
 docker exec server${i} cat /root/ansible_test.txt
done
Configured by Ansible on server1Configured by Ansible on server2Configured by Ansible on server3Configured by Ansible on server4:
```

# Workflow Summary

---

1. Setup SSH keys  
↓
2. Build Docker image  
↓
3. Launch 4 containers  
↓
4. Create inventory.ini  
↓
5. Test connectivity (ansible ping)  
↓
6. Run playbook  
↓
7. Verify changes  
↓
8. Cleanup containers

## The Need for Ansible in Server Management

---

1. **Scalability:** Managing multiple servers manually becomes impractical as infrastructure grows.
2. **Consistency:** Ensures identical configurations across all servers, reducing “works on my machine” issues.
3. **Efficiency:** Automates repetitive tasks, saving time and reducing human error.
4. **Idempotency:** Operations can be run multiple times without causing unintended changes.
5. **Infrastructure as Code:** Configuration is version-controlled and documented.

## Key Features of Ansible

---

1. **Agentless** – Uses SSH (no need to install software on managed nodes)
2. **Idempotent** – Ensures desired state without repeating changes unnecessarily
3. **YAML-Based Playbooks** – Simple, human-readable automation scripts
4. **Modules** – 3000+ built-in modules for cloud, containers, networking, etc.
5. **Push-Based** – Executes tasks from a control machine
6. **Infrastructure as Code (IaC)** – Supports declarative configuration management



# ContainerizationAndDevOps

## Experiment 10: Working with SonarQube

### Introduction

#### What is SonarQube? [🔗](#)

SonarQube is an open-source platform for continuous inspection of code quality. It performs automatic reviews with static analysis to detect bugs, code smells & security vulnerabilities.

#### How SonarQube Solves the Problem:

- **Continuous Inspection:** Scans code with every commit, providing immediate feedback.
- **Quality Gates:** Defines pass/fail criteria for code quality.
- **Technical Debt Quantification:** Measures effort needed to fix issues.
- **Multi-language Support:** Supports 20+ programming languages.
- **Visual Analytics:** Dashboard showing code quality metrics and trends.

#### Key Terms:

| Term           | What it means                                             |
|----------------|-----------------------------------------------------------|
| Quality Gate   | A set of rules; code must pass before deployment          |
| Bug            | Code that will likely break or behave incorrectly         |
| Vulnerability  | A security weakness in the code                           |
| Code Smell     | Code that works but is poorly written or hard to maintain |
| Technical Debt | Estimated time to fix all issues                          |
| Coverage       | Percentage of code tested by unit tests                   |
| Duplication    | Repeated code blocks (copy-paste)                         |

### Practical

## Step-1:- Create a `docker-compose.yml`

```
version: '3.8'

services:
 sonar-db:
 image: postgres:13
 container_name: sonar-db
 environment:
 POSTGRES_USER: sonar
 POSTGRES_PASSWORD: sonar
 POSTGRES_DB: sonarqube
 volumes:
 - sonar-db-data:/var/lib/postgresql/data
 networks:
 - sonarqube-lab

 sonarqube:
 image: sonarqube:lts-community
 container_name: sonarqube
 ports:
 - "9000:9000"
 environment:
 SONAR_JDBC_URL: jdbc:postgresql://sonar-db:5432/sonarqube
 SONAR_JDBC_USERNAME: sonar
 SONAR_JDBC_PASSWORD: sonar
 volumes:
 - sonar-data:/opt/sonarqube/data
 - sonar-extensions:/opt/sonarqube/extensions
 depends_on:
 - sonar-db
 networks:
 - sonarqube-lab

volumes:
 sonar-db-data:
 sonar-data:
 sonar-extensions:

networks:
 sonarqube-lab:
 driver: bridge
```

## Step-2:- Start Compose

```
docker-compose up -d
```

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment10$ docker-compose up -d
[+] up 28/28
[+] up 34/34 tgres:13 Pulled
✓ Image postgres:13 Pulled
✓ Image sonarqube:lts-community Pulled
✓ Network experiment10_sonarqube-lab Created
✓ Volume experiment10_sonar-extensions Created
✓ Volume experiment10_sonar-db-data Created
✓ Volume experiment10_sonar-data Created
✓ Container sonar-db Created
✓ Container sonarqube Created
```

### Step-3:- Verify SonarQube is running via Logs

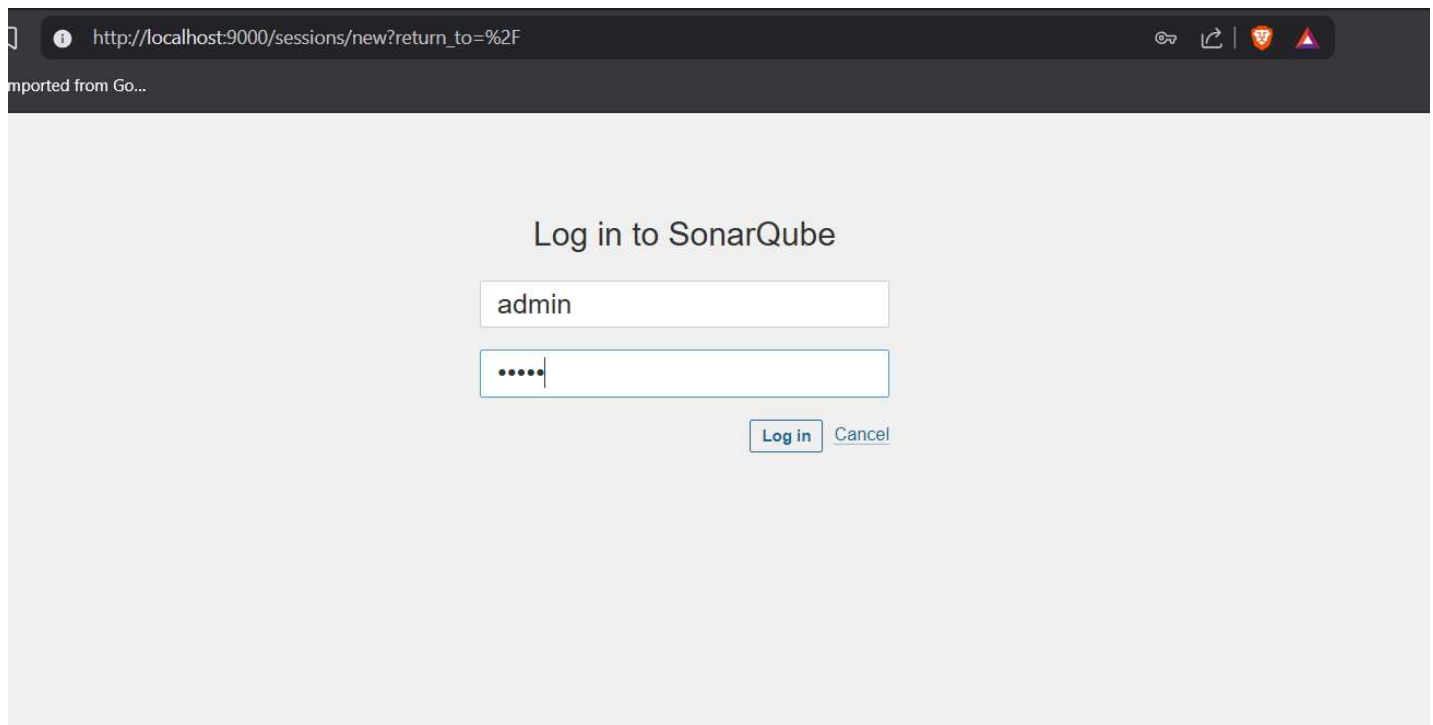
```
docker-compose logs -f sonarqube
```

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment10$ docker-compose logs -f sonarqube
sonarqube | 2026.04.24 05:09:10 INFO web[][o.s.s.e.IndexCreator] Create type rules/rule
sonarqube | 2026.04.24 05:09:10 INFO es[][o.e.c.m.MetadataMappingService] [rules/xKdjHUFjTQSWwke2y7SYwA] create_mapping [rule]
sonarqube | 2026.04.24 05:09:10 INFO web[][o.s.s.e.IndexCreator] Create index [issues]
sonarqube | 2026.04.24 05:09:10 INFO es[][o.e.c.m.MetadataCreateIndexService] [issues] creating index, cause [api], templates [], shards [5]/[0]
sonarqube | 2026.04.24 05:09:11 INFO es[][o.e.c.r.a.AllocationService] Cluster health status changed from [YELLOW] to [GREEN] (reason: [shards started [[issues][4]]]).
sonarqube | 2026.04.24 05:09:11 INFO web[][o.s.s.e.IndexCreator] Create type issues/auth
sonarqube | 2026.04.24 05:09:11 INFO es[][o.e.c.m.MetadataMappingService] [issues/DfhvUEjSMaNI4x2Fal2PA] create_mapping [auth]
sonarqube | 2026.04.24 05:09:11 INFO web[][o.s.s.e.IndexCreator] Create index [users]
sonarqube | 2026.04.24 05:09:11 INFO es[][o.e.c.m.MetadataCreateIndexService] [users] creating index, cause [api], templates [], shards [1]/[0]
sonarqube | 2026.04.24 05:09:11 INFO es[][o.e.c.r.a.AllocationService] Cluster health status changed from [YELLOW] to [GREEN] (reason: [shards started [[users][0]]]).
sonarqube | 2026.04.24 05:09:11 INFO web[][o.s.s.e.IndexCreator] Create type users/user
```

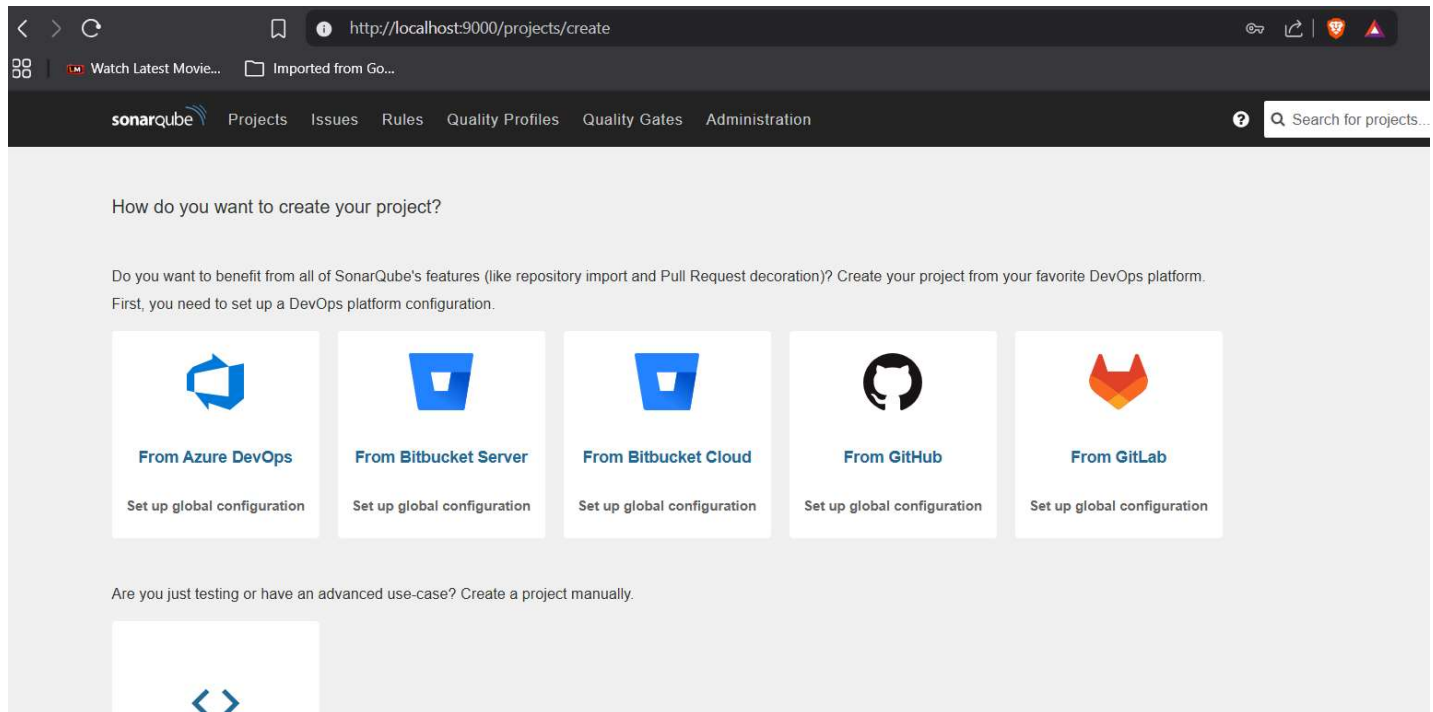
### Step-4:- Login to SonarQube

<http://localhost:9000>

- username: admin
- password: admin



## Step-5:- HomePage



## Step-6:- Create Sample Java App

```
mkdir -p sample-java-app/src/main/java/com/example
cd sample-java-app
```

```
[1]# stopped docker-compose logs -f sonarqube
ishita@ish:~/ContainerizationAndDevOps/LAB/Experiment10$ mkdir -p sample-java-app/src/main/java/com/example
cd sample-java-app
ishita@ish:~/ContainerizationAndDevOps/LAB/Experiment10/sample-java-app$
```

## Step-7:- Create Calculator.java

```
nano src/main/java/com/example/Calculator.java
```

```
package com.example;
```

```
public class Calculator {
```

```
 // BUG: Division by zero is not handled
```

```
 public int divide(int a, int b) {
```

```
 return a / b;
```

```
 }
```

```
 // CODE SMELL: Unused variable
```

```
 public int add(int a, int b) {
```

```
 int result = a + b;
```

```
 int unused = 100;
```

```
 return result;
```

```
 }
```

```
 // VULNERABILITY: SQL Injection risk
```

```
 public String getUser(String userId) {
```

```
 String query = "SELECT * FROM users WHERE id = " + userId;
```

```
 return query;
```

```
 }
```

```
 // CODE SMELL: Duplicated code
```

```
 public int multiply(int a, int b) {
```

```
 int result = 0;
```

```
 for (int i = 0; i < b; i++) {
```

```
 result = result + a;
```

```
 }
```

```
 return result;
```

```
 }
```

```
 public int multiplyAlt(int a, int b) {
```

```
 int result = 0;
```

```
 for (int i = 0; i < b; i++) {
```

```
 result = result + a;
```

```
 }
```

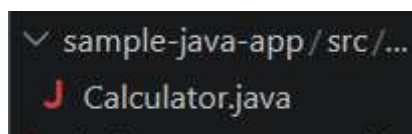
```
 return result;
```

```
 }
```



```
// BUG: Null pointer risk
public String getName(String name) {
 return name.toUpperCase();
}

// CODE SMELL: Empty catch block
public void riskyOperation() {
 try {
 int x = 10 / 0;
 } catch (Exception e) {
 }
}
}
```



### Step-8:- Create pom.xml

```
nano pom.xml
```

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
 http://maven.apache.org/xsd/maven-4.0.0.xsd">

 <modelVersion>4.0.0</modelVersion>

 <groupId>com.example</groupId>
 <artifactId>sample-app</artifactId>
 <version>1.0-SNAPSHOT</version>

 <properties>
 <maven.compiler.source>11</maven.compiler.source>
 <maven.compiler.target>11</maven.compiler.target>

 <!-- SonarQube connection settings -->
 <sonar.projectKey>sample-java-app</sonar.projectKey>
 <sonar.host.url>http://localhost:9000</sonar.host.url>
 <!-- Replace with your actual token (generated in Step 9) -->
 <sonar.login>YOUR_TOKEN_HERE</sonar.login>
 </properties>

 <dependencies>
```

```

<!-- JUnit for unit tests -->
<dependency>
 <groupId>junit</groupId>
 <artifactId>junit</artifactId>
 <version>4.13.2</version>
 <scope>test</scope>
</dependency>
</dependencies>

<build>
 <plugins>
 <!-- This plugin lets us run: mvn sonar:sonar -->
 <plugin>
 <groupId>org.sonarsource.scanner.maven</groupId>
 <artifactId>sonar-maven-plugin</artifactId>
 <version>3.9.1.2184</version>
 </plugin>
 </plugins>
</build>

</project>

```

```

LAB > Experiment10 > sample-java-app > pom.xml
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5 http://maven.apache.org/xsd/maven-4.0.0.xsd">
6
7 <modelVersion>4.0.0</modelVersion>
8
9 <groupId>com.example</groupId>
10 <artifactId>sample-app</artifactId>
11 <version>1.0-SNAPSHOT</version>
12
13 <properties>
14 <maven.compiler.source>11</maven.compiler.source>
15 <maven.compiler.target>11</maven.compiler.target>
16
17 <!-- SonarQube configuration settings -->

```

### Step-9:- Generate Token

Navigate to: Profile → My Account → Security

- Token Name: scanner-token
- Click **Generate**
- Copy the token immediately — it is shown only once!

**Important:** Store the token somewhere safe. You cannot retrieve it again after closing the page.

Enter Token Name

Select Token Type

30 days

Generate

**!** New token "scanner-token" has been created. Make sure you copy it now, you won't be able to see it again!

 Copy `squ_3ff1e919d247a9f0c7e045e100670da001860bd0`

| Name          | Type | Project | Last use | Created        | Expiration   |                        |
|---------------|------|---------|----------|----------------|--------------|------------------------|
| scanner-token | User |         | Never    | April 24, 2026 | May 24, 2026 | <a href="#">Revoke</a> |

## Step-10:- Run Scanner

```
mvn sonar:sonar -Dsonar.login=TOKEN
```

```
ishita@ish:~/ContainerizationAndDevOps/LAB/Experiment10/sample-java-app$ mvn sonar:sonar -Dsonar.login=squ_3ff1e919d247a9f0c7e045e100670da001860bd0
[INFO] Scanning for projects...
[INFO] -----< com.example:sample-app >-----
[INFO] Building sample-app 1.0-SNAPSHOT
[INFO] -----[jar]-----
[INFO] --- sonar-maven-plugin:3.9.1.2184:sonar (default-cli) @ sample-app ---
[INFO] User cache: /home/ishita/.sonar/cache
[INFO] SonarQube version: 9.9.8.100196
[INFO] Default locale: "en", source code encoding: "UTF-8" (analysis is platform dependent)
[INFO] Load global settings
[INFO] Load global settings (done) | time=185ms
[INFO] Server id: 74C15348-AZ295DX1ktg-Vwxzggwe
[INFO] User cache: /home/ishita/.sonar/cache
[INFO] Load/download plugins
[INFO] Load plugins index
[INFO] Load plugins index (done) | time=209ms
[INFO] Load/download plugins (done) | time=2607ms
[INFO] Process project properties
[INFO] Process project properties (done) | time=17ms
[INFO] Execute project builders
[INFO] Execute project builders (done) | time=2ms
[INFO] Project key: sample-java-app
```

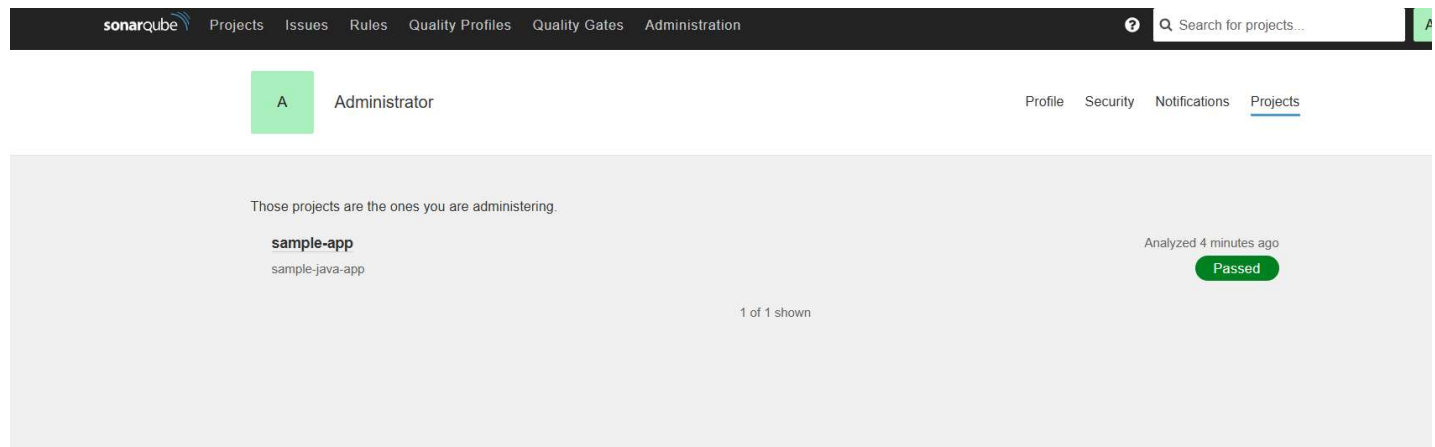
## Step-11:- Scroll down to verify Build Success

```
[INFO] ANALYSIS SUCCESSFUL, you can find the results at: http://localhost:9000/dashboard?id=sample-java-app
[INFO] Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
[INFO] More about the report processing at http://localhost:9000/api/ce/task?id=AZ2982RLktg-Vwxzggp4
[INFO] Analysis total time: 10.561 s
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 15.188 s
[INFO] Finished at: 2026-04-24T05:25:38Z
[INFO] -----
```

## Step-12:- View Analysis Results on Browser

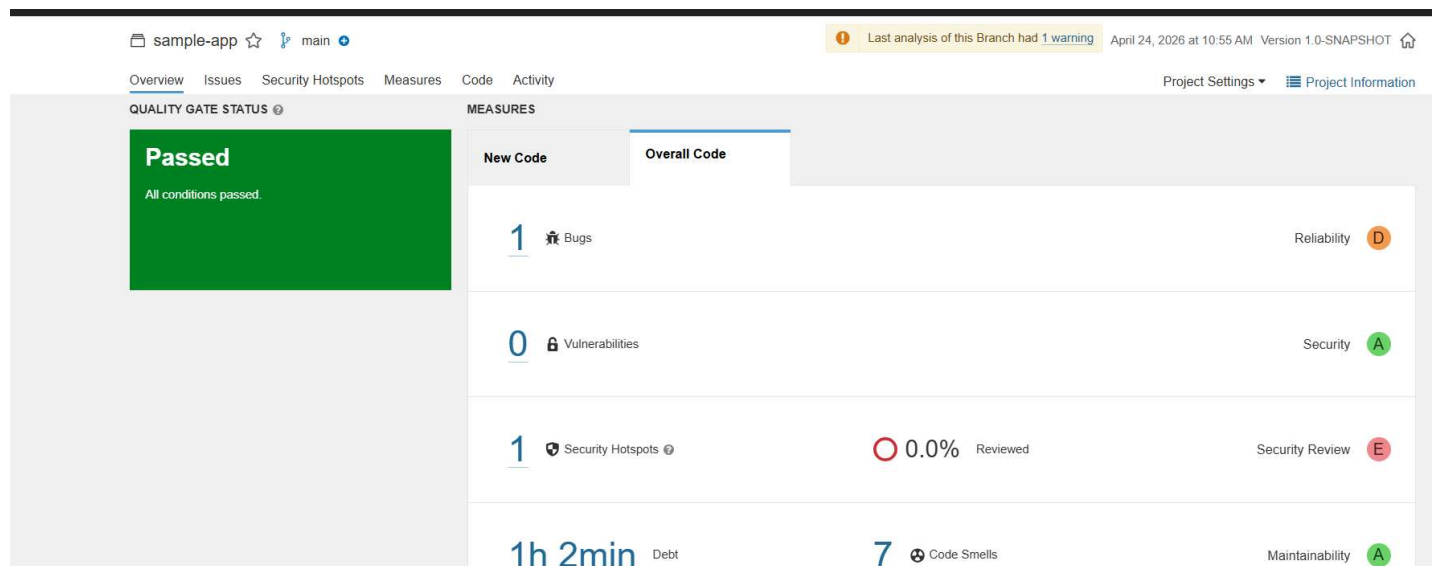
Open the dashboard:

<http://localhost:9000/dashboard?id=sample-java-app>



## Step-13:- View Detailed Report

Click on any issue number to see the exact line and reason for each issue.



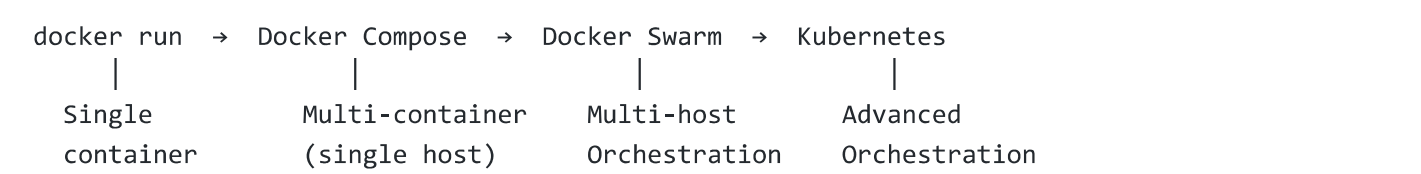
# ContainerizationAndDevOps

## Experiment 11: Container Orchestration via Docker Swarm

### What is Docker Swarm?

Docker Swarm is Docker’s built-in container orchestration tool. It lets us manage a group of Docker hosts as a single cluster, automatically distributing containers across machines, scaling them up or down, and recovering from failures — all with commands you already know from Docker.

### The Progression Path



**This experiment focuses on:** Moving from Compose → Swarm

### Key Concepts:

| Term         | What it means                                                                |
|--------------|------------------------------------------------------------------------------|
| Swarm        | A cluster of Docker hosts (nodes) managed together                           |
| Manager Node | The node that controls the swarm and schedules tasks                         |
| Worker Node  | The node that runs the actual containers                                     |
| Service      | The Swarm equivalent of a Compose service — defines how many replicas to run |
| Stack        | A group of services deployed together from a Compose file                    |
| Replica      | A single running copy of a container in a service                            |
| Self-healing | Swarm automatically recreates containers if they crash                       |

### Docker Compose vs Docker Swarm:



| Aspect         | Docker Compose                  | Docker Swarm                            |
|----------------|---------------------------------|-----------------------------------------|
| Hosts          | Single machine only             | Multiple machines (cluster)             |
| Scaling        | Manual ( <code>--scale</code> ) | Built-in ( <code>service scale</code> ) |
| Self-healing   | No                              | Yes — auto-restarts failed containers   |
| Load balancing | No                              | Yes — built-in across replicas          |
| Use case       | Development                     | Production / multi-host                 |

### Step-1:- Create a `docker-compose.yml`

```
version: '3.9'

services:
 db:
 image: mysql:5.7
 restart: always
 environment:
 MYSQL_ROOT_PASSWORD: rootpass
 MYSQL_DATABASE: wordpress
 MYSQL_USER: wpuser
 MYSQL_PASSWORD: wppass
 volumes:
 - db_data:/var/lib/mysql

 wordpress:
 image: wordpress:latest
 depends_on:
 - db
 ports:
 - "8080:80"
 restart: always
 environment:
 WORDPRESS_DB_HOST: db:3306
 WORDPRESS_DB_USER: wpuser
 WORDPRESS_DB_PASSWORD: wppass
 WORDPRESS_DB_NAME: wordpress
 volumes:
 - wp_data:/var/www/html

volumes:
 db_data:
 wp_data:
```

```

LAB > Experiment11 > docker-compose.yml
1 version: '3.9'
2
3 services:
4 db:
5 image: mysql:5.7
6 restart: always
7 environment:
8 MYSQL_ROOT_PASSWORD: rootpass
9 MYSQL_DATABASE: wordpress
10 MYSQL_USER: wpuser
11 MYSQL_PASSWORD: wppass
12 volumes:
13 - db_data:/var/lib/mysql
14
15 wordpress:
16 image: wordpress:latest
17 depends_on:

```

## Step-2:- Initialize Swarm

This turns the current machine into a Swarm **manager node**:

```
docker swarm init
```

```

ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment11$ docker swarm init
Swarm initialized: current node (pewsd5r5ith0jewvr6gag4r9gy) is now a manager.

To add a worker to this swarm, run the following command:

docker swarm join --token SWMTKN-1-48jk5f4njsft046iwtio244p01v2s1k00ydh5f47fhy2luok

```

## Step-3:- Verify Swarm is Active

```
docker node ls
```

I saw my machine listed with role `Leader` and status `Ready` .

```

ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment11$ docker node ls
ID HOSTNAME STATUS AVAILABILITY MANAGER STATUS ENGINE VERSION
pewsd5r5ith0jewvr6gag4r9gy * docker-desktop Ready Active Leader 29.1.3
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment11$

```

## Step-4:- Deploy the Stack

A stack is how we deploy a Compose file onto a Swarm. The name `wpstack` is the stack name:

```
docker stack deploy -c docker-compose.yml wpstack
```

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment11$ docker stack deploy -c docker-compose.yml wpstack
Ignoring unsupported options: restart

Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network wpstack_default
Creating service wpstack_wordpress
Creating service wpstack_db
```

## Step-5:- Verify Running Services

```
docker service ls
```

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment11$ docker service ls
```

| ID           | NAME              | MODE       | REPLICAS | IMAGE            | PORTS          |
|--------------|-------------------|------------|----------|------------------|----------------|
| egodyq2b0qb5 | wpstack_db        | replicated | 1/1      | mysql:5.7        |                |
| s2s4t4iq9dg3 | wpstack_wordpress | replicated | 1/1      | wordpress:latest | *:8080->80/tcp |

## Step-6:- Inspect the WordPress Service Tasks

```
docker service ps wpstack_wordpress
```

This shows which node each replica is running on and its current state.

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment11$ docker service ps wpstack_wordpress
```

| ID           | NAME                | IMAGE            | NODE           | DESIRED STATE | CURRENT STATE              | ERROR | PORTS |
|--------------|---------------------|------------------|----------------|---------------|----------------------------|-------|-------|
| dn1ne61th5kn | wpstack_wordpress.1 | wordpress:latest | docker-desktop | Running       | Running about a minute ago |       |       |

## Step-7:- List Running Containers

```
docker ps
```

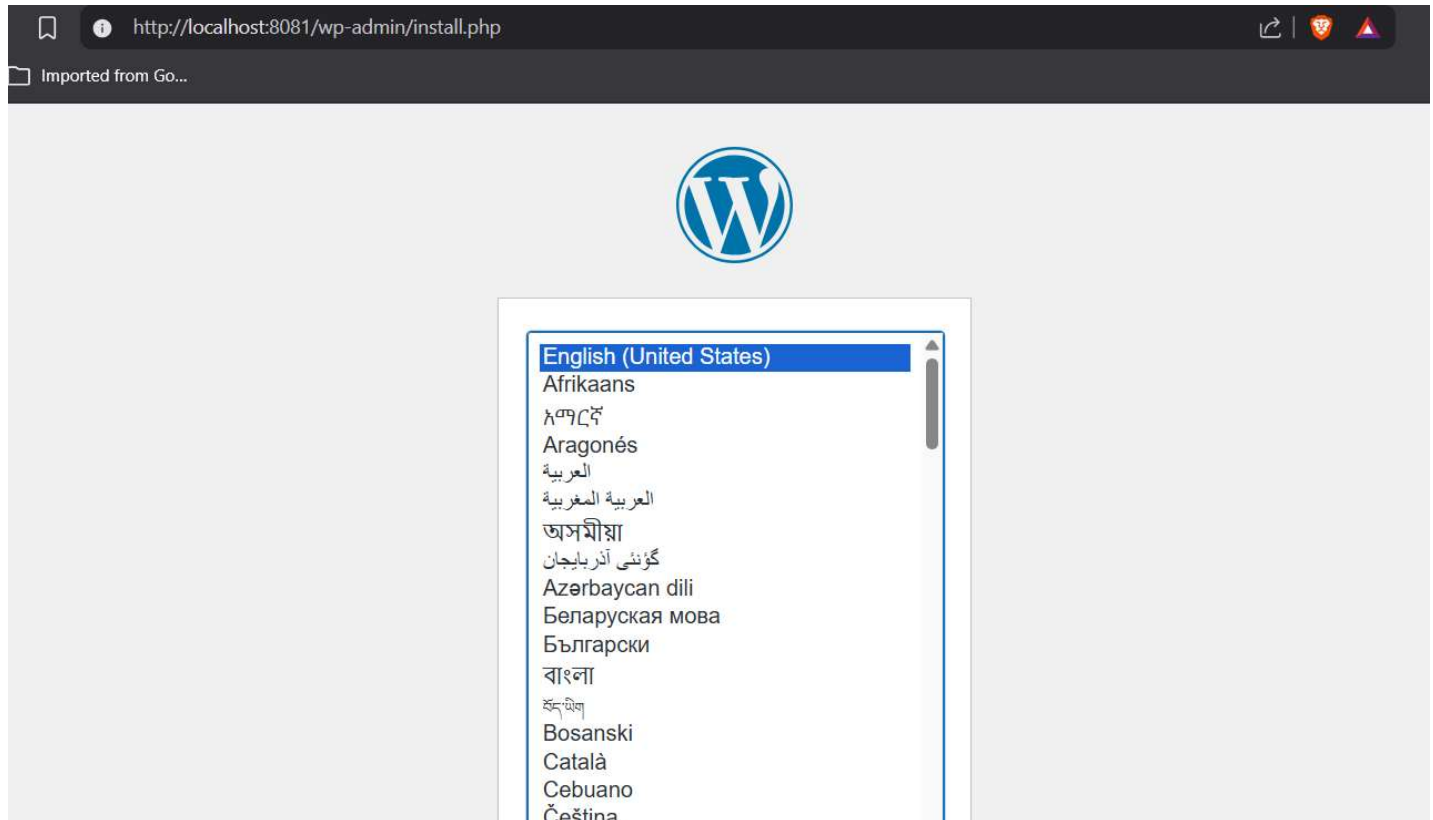
```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment11$ docker ps
```

| CONTAINER ID | IMAGE            | COMMAND                  | CREATED       | STATUS        | PORTS               |
|--------------|------------------|--------------------------|---------------|---------------|---------------------|
| b38efddc95e2 | wordpress:latest | "docker-entrypoint.s..." | 2 minutes ago | Up 2 minutes  | 80/tcp              |
| 09e372574d4a | mysql:5.7        | "docker-entrypoint.s..." | 2 minutes ago | Up 2 minutes  | 3306/tcp, 33060/tcp |
| 03a6f6abcc8a | mysql:5.7        | "docker-entrypoint.s..." | 2 weeks ago   | Up 27 minutes | 3306/tcp, 33060/tcp |

## Step-8:- Access the Application in Browser

`http://localhost:8081/`

WordPress is now accessible..



## Step-9:- Scale the WordPress Service

Increase the number of WordPress replicas from 1 to 3:

```
docker service scale wpstack_wordpress=3
```

Swarm automatically starts 2 new containers to reach the desired count of 3.

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment11$ docker service scale wpstack_wordpress=3
wpstack_wordpress scaled to 3
overall progress: 3 out of 3 tasks
1/3: running [=====>]
2/3: running [=====>]
3/3: running [=====>]
verify: Service wpstack_wordpress converged
```

## Step-10:- Verify the Scaling

```
docker service ls
```

The `REPLICAS` column for `wpstack_wordpress` now shows `3/3`.

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment11$ docker service ls
```

| ID           | NAME              | MODE       | REPLICAS | IMAGE            | PORTS          |
|--------------|-------------------|------------|----------|------------------|----------------|
| egodyq2b0qb5 | wpstack_db        | replicated | 1/1      | mysql:5.7        |                |
| s2s4t4iq9dg3 | wpstack_wordpress | replicated | 3/3      | wordpress:latest | *:8081->80/tcp |

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment11$
```

### Step-11:- List WordPress Containers

```
docker ps | grep wordpress
```

You should see 3 wordpress containers running.

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment11$ docker ps | grep wordpress
```

| Container ID | Image            | Command                  | Created       | Up            | Ports               |
|--------------|------------------|--------------------------|---------------|---------------|---------------------|
| eac81ad2380a | wordpress:latest | "docker-entrypoint.s..." | 2 minutes ago | Up 2 minutes  | 80/tcp              |
| ba4cf7d9f92b | wordpress:latest | "docker-entrypoint.s..." | 2 minutes ago | Up 2 minutes  | 80/tcp              |
| 06ea551318f4 | wordpress:latest | "docker-entrypoint.s..." | 2 minutes ago | Up 2 minutes  | 80/tcp              |
| 03a6f6abcc8a | mysql:5.7        | "docker-entrypoint.s..." | 2 weeks ago   | Up 35 minutes | 3306/tcp, 33060/tcp |

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment11$
```

### Step-12:- Kill a Container to Trigger Self-Healing

Copy any container ID from the list above and kill it:

```
docker kill <container-id>
```

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment11$ docker kill eac8
eac8
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment11$
```

### Step-13:- Watch Swarm Recreate the Container

Swarm detects the missing replica and immediately starts a replacement:

```
docker service ps wpstack_wordpress
```

Here i Saw the killed container listed as `shutdown` and a new one as `Running`.



```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment11$ docker service ps wpstack_wordpress
```

| ID           | NAME                  | IMAGE            | NODE           | DESIRED STATE | CURRENT STATE  |
|--------------|-----------------------|------------------|----------------|---------------|----------------|
| 2dsnm4pc3t9  | wpstack_wordpress.1   | wordpress:latest | docker-desktop | Running       | Running 4 min  |
| dnln61th5kn  | _ wpstack_wordpress.1 | wordpress:latest | docker-desktop | Shutdown      | Shutdown 4 min |
| 4n3ubzhxy4qy | wpstack_wordpress.2   | wordpress:latest | docker-desktop | Running       | Running 3 min  |
| pq19ozq6kslx | wpstack_wordpress.3   | wordpress:latest | docker-desktop | Running       | Running 50 sec |

#### Step-14:- Confirm the Container is Replaced

```
docker ps | grep wordpress
```

You should again see 3 running WordPress containers — the swarm restored the desired state automatically.

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment11$ docker ps | grep wordpress
```

| ID           | NAME             | IMAGE                    | ENTRYPOINT         | STATUS | UP             | PORTS  |
|--------------|------------------|--------------------------|--------------------|--------|----------------|--------|
| 426a86f365c8 | wordpress:latest | "docker-entrypoint.s..." | About a minute ago | Up     | About a minute | 80/tcp |
| ba4cf7d9f92b | wordpress:latest | "docker-entrypoint.s..." | 4 minutes ago      | Up     | 4 minutes      | 80/tcp |
| 06ea551318f4 | wordpress:latest | "docker-entrypoint.s..." | 5 minutes ago      | Up     | 5 minutes      | 80/tcp |

#### Step-15:- Remove the Stack

```
docker stack rm wpstack
```

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment11$ docker stack rm wpstack
```

```
Removing service wpstack_db
Removing service wpstack_wordpress
Removing network wpstack_default
```

#### Step-16:- Confirm Clean Removal

```
docker service ls
docker ps
```

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment11$ docker service ls
```

```
docker ps
```

| ID            | NAME                          | MODE | REPLICAS | IMAGE | PORTS                    | CREATED     |
|---------------|-------------------------------|------|----------|-------|--------------------------|-------------|
| 03a6f6abcc8a  | mysql:5.7                     |      |          |       | "docker-entrypoint.s..." | 2 weeks ago |
| aeaba2aaa7cf1 | portainer/portainer-ce:latest |      |          |       | "/portainer"             | 5 weeks ago |

## Conclusion

| Started with                                 | Now                                                  |
|----------------------------------------------|------------------------------------------------------|
| Single container ( <code>docker run</code> ) | Multi-container stack (Compose + Swarm)              |
| Manual startup                               | One-command deployment ( <code>stack deploy</code> ) |
| Manual scaling                               | Instant scaling ( <code>service scale</code> )       |
| No fault tolerance                           | Automatic self-healing                               |
| Single host                                  | Multi-host cluster ready                             |

## Quick Reference Card

*# Initialize Swarm (makes current machine the manager)*

```
docker swarm init
```

*# Deploy a stack from a Compose file*

```
docker stack deploy -c docker-compose.yml <stack-name>
```

*# List all services in the swarm*

```
docker service ls
```

*# See tasks (containers) for a specific service*

```
docker service ps <service-name>
```

*# Scale a service to N replicas*

```
docker service scale <stack-name_service-name>=<N>
```

*# Remove an entire stack*

```
docker stack rm <stack-name>
```

*# Leave the swarm (cleanup)*

```
docker swarm leave --force
```

**Key Takeaway:** Compose *defines* the application. Swarm *runs it reliably* — with scaling, load balancing, and automatic recovery built in.

# ContainerizationAndDevOps

## Experiment 12: Study and Analyse Container Orchestration using Kubernetes

### What is Kubernetes?

Kubernetes (K8s) is an open-source container orchestration platform that automates deployment, scaling, and management of containerized applications. It ensures your app keeps running, scales automatically, and recovers from failures — without manual intervention.

### Why Kubernetes over Docker Swarm?

| Reason               | Explanation                                 |
|----------------------|---------------------------------------------|
| Industry standard    | Most companies use Kubernetes               |
| Powerful scheduling  | Automatically decides where to run your app |
| Large ecosystem      | Many tools and plugins available            |
| Cloud-native support | Works on AWS, Google Cloud, Azure, etc.     |

### Core Kubernetes Concepts:

| Docker Concept  | Kubernetes Equivalent | What it means                                                          |
|-----------------|-----------------------|------------------------------------------------------------------------|
| Container       | Pod                   | A group of one or more containers. Smallest unit in K8s.               |
| Compose service | Deployment            | Describes how your app should run (e.g., 2 copies, which image to use) |
| Load balancing  | Service               | Exposes your app to the outside world or other pods                    |
| Scaling         | ReplicaSet            | Ensures a certain number of pod copies are always running              |

### Tool Selection Guide:

| Tool     | Best for                       |
|----------|--------------------------------|
| k3d      | Quick learning on your laptop  |
| Minikube | Single-node cluster testing    |
| kubeadm  | Real, production-style cluster |

## Part A — Basic Deployment & Service (k3d)

### Step-1:- Create `wordpress-deployment.yaml`

```
nano wordpress-deployment.yaml
```

Paste This:

```
wordpress-deployment.yaml
apiVersion: apps/v1 # Which Kubernetes API to use
kind: Deployment # Type of resource
metadata:
 name: wordpress # Name of this deployment
spec:
 replicas: 2 # Run 2 identical pods
 selector:
 matchLabels:
 app: wordpress # Pods with this label belong to this deployment
 template:
 metadata:
 labels:
 app: wordpress # Label applied to each pod
 spec:
 containers:
 - name: wordpress
 image: wordpress:latest # Docker image
 ports:
 - containerPort: 80 # Port inside the container
```

```
LAB > Experiment12 > ! wordpress-deployment.yaml
```

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4 name: wordpress
5 spec:
6 replicas: 2
7 selector:
8 matchLabels:
9 app: wordpress
10 template:
11 metadata:
12 labels:
13 app: wordpress
14 spec:
15 containers:
16 - name: wordpress
```

## Step-2:- Apply the Deployment

```
kubectl apply -f wordpress-deployment.yaml
```

What happens? Kubernetes creates 2 pods running WordPress.

```
ishita@ish:~/ContainerizationAndDevOps/LAB/Experiment12$ kubectl apply -f wordpress-deployment.yaml
deployment.apps/wordpress created
ishita@ish:~/ContainerizationAndDevOps/LAB/Experiment12$
```

## Step-3:- Create wordpress-service.yaml

Pods are **temporary** (they can be deleted or recreated). A **Service** gives them a fixed IP and exposes them to the outside.

```
nano wordpress-service.yaml
```

Paste This:

```
wordpress-service.yaml
apiVersion: v1
kind: Service
metadata:
 name: wordpress-service
```



```
spec:
 type: NodePort # Exposes service on a port of each node (VM)
 selector:
 app: wordpress # Send traffic to pods with this label
 ports:
 - port: 80 # Service port
 targetPort: 80 # Pod port
 nodePort: 30007 # External port (range: 30000-32767)
```

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment12$ nano wordpress-service.yaml
```

#### Step-4:- Apply the Service

```
kubectl apply -f wordpress-service.yaml
```

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment12$ kubectl apply -f wordpress-service.yaml
service/wordpress-service created
```

#### Step-5:- Verify Pods are Running

```
kubectl get pods
```

Expected output:

| NAME                  | READY | STATUS  | RESTARTS | AGE |
|-----------------------|-------|---------|----------|-----|
| wordpress-xxxxx-yyyyy | 1/1   | Running | 0        | 1m  |
| wordpress-xxxxx-zzzzz | 1/1   | Running | 0        | 1m  |

```
wordpress-7d6f6db8d8-cfsnr 1/1 Running 0 2m29s
wordpress-7d6f6db8d8-nhqfc 1/1 Running 0 2m29s
```

#### Step-6:- Verify the Service

```
kubectl get svc
```

Expected output:

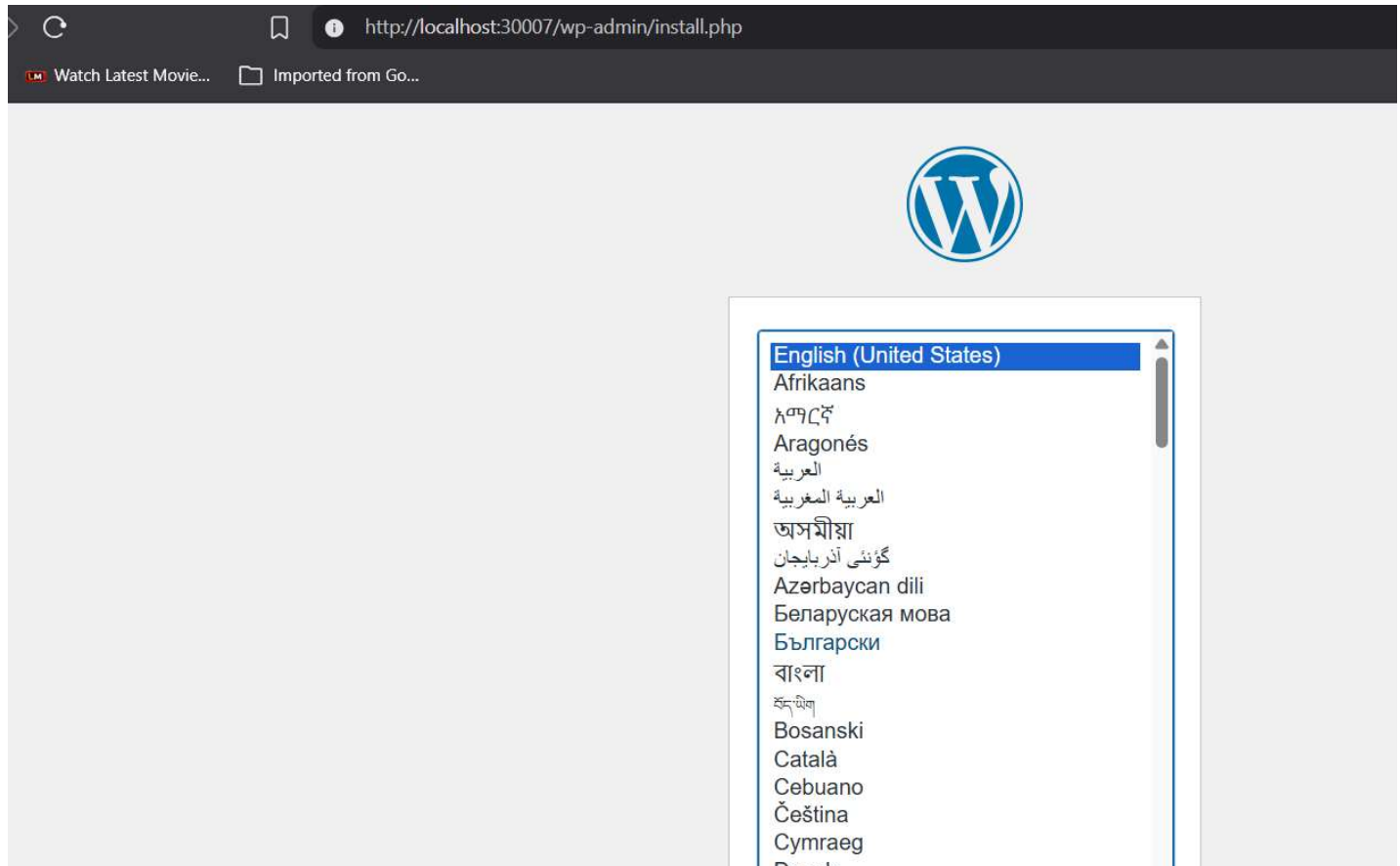
| NAME              | TYPE     | CLUSTER-IP | PORT(S)      | AGE |
|-------------------|----------|------------|--------------|-----|
| wordpress-service | NodePort | 10.43.x.x  | 80:30007/TCP | 1m  |

```
wordpress-service NodePort 10.43.181.51 <none> 80:30007/TCP 69s
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment12$
```

## Step-7:- Access WordPress in Browser

`http://<node-ip>:30007`

- **Minikube:** run `minikube ip` to get the node IP
- **k3d:** use `localhost`



## Step-8:- Scale the Deployment

Increase the number of pods from 2 to 4:

```
kubectl scale deployment wordpress --replicas=4
```

Verify:

```
kubectl get pods
```

You should now see **4 running pods**. **Why scale?** More traffic → more copies → better performance.

```
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment12$ kubectl scale deployment wordpress --replicas=4
deployment.apps/wordpress scaled
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment12$ kubectl get pods
```

| NAME                       | READY | STATUS  | RESTARTS | AGE |
|----------------------------|-------|---------|----------|-----|
| mysql-6d96494b7f-2l89h     | 1/1   | Running | 0        | 89s |
| wordpress-5cb9cb9489-6d5l7 | 1/1   | Running | 0        | 66s |
| wordpress-5cb9cb9489-btsxx | 1/1   | Running | 0        | 73s |
| wordpress-5cb9cb9489-mkc6n | 1/1   | Running | 0        | 13s |
| wordpress-5cb9cb9489-mthrf | 1/1   | Running | 0        | 13s |

## Step-9:- Self-Healing Demonstration

Kubernetes automatically replaces failed pods. Delete one pod manually to observe:

```
First, get pod names
kubectl get pods
```

```
Delete one (replace <pod-name> with an actual name from above)
kubectl delete pod <pod-name>
```

Now check pods again:

```
kubectl get pods
```

You will still see **4 pods** — the deleted one was **automatically recreated**.

**Why?** The deployment ensures the desired number (4) is always running.

```
Error from server (notfound): pods "mysql-6d96494b7f-2l89h" not found
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment12$ kubectl delete pod wordpress-5cb9cb9489-6d5l7
pod "wordpress-5cb9cb9489-6d5l7" deleted from default namespace
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment12$ kubectl get pods
```

| NAME                       | READY | STATUS  | RESTARTS | AGE   |
|----------------------------|-------|---------|----------|-------|
| mysql-6d96494b7f-2l89h     | 1/1   | Running | 0        | 3m29s |
| wordpress-5cb9cb9489-btsxx | 1/1   | Running | 0        | 3m13s |
| wordpress-5cb9cb9489-mkc6n | 1/1   | Running | 0        | 2m13s |
| wordpress-5cb9cb9489-mthrf | 1/1   | Running | 0        | 2m13s |
| wordpress-5cb9cb9489-z9l69 | 1/1   | Running | 0        | 8s    |

## Part B — Docker Swarm vs Kubernetes Comparison

| Feature      | Docker Swarm | Kubernetes                 |
|--------------|--------------|----------------------------|
| Setup        | Very easy    | More complex               |
| Scaling      | Basic        | Advanced (auto-scaling)    |
| Ecosystem    | Small        | Huge (monitoring, logging) |
| Industry use | Rare         | Standard                   |

**Verdict:** Learn Kubernetes — it's what companies use.

## Part C — Advanced Lab: Real Cluster with kubeadm

**Requirements:** 2 or 3 VMs (VirtualBox/VMware), Ubuntu 22.04 or 24.04, each with 2+ CPU & 2+ GB RAM.

### Step-10:- Install kubeadm, kubelet, kubectl on ALL nodes

```
Update system
sudo apt update

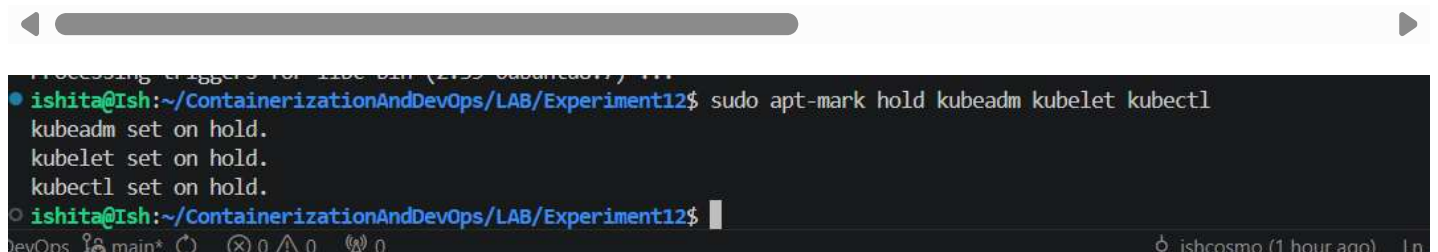
Install required packages
sudo apt install -y apt-transport-https ca-certificates curl

Add Kubernetes signing key
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.29/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keys/kubernetes-keyring.gpg

Add Kubernetes repository
echo 'deb [signed-by=/etc/apt/keys/kubernetes-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.29/deb/' > /etc/apt/sources.list.d/kubernetes.list

Install kubeadm, kubelet, kubectl
sudo apt update
sudo apt install -y kubeadm kubelet kubectl

Hold versions to prevent auto-update
sudo apt-mark hold kubeadm kubelet kubectl
```



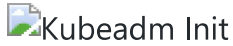
```
Processing triggers for libc-bin (2.39-0ubuntu7) ...
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment12$ sudo apt-mark hold kubeadm kubelet kubectl
kubeadm set on hold.
kubelet set on hold.
kubectl set on hold.
ishita@Ish:~/ContainerizationAndDevOps/LAB/Experiment12$
```

## Step-11:- Initialize the Control Plane (Master Node only)

Run only on the master node:

```
sudo kubeadm init
```

This sets up the control plane, generates a token for worker nodes to join, and takes 2–3 minutes.



## Step-12:- Set up kubeconfig (Master Node only)

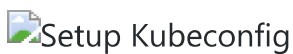
```
Create .kube directory for your user
mkdir -p $HOME/.kube

Copy admin config
sudo cp /etc/kubernetes/admin.conf $HOME/.kube/config

Fix permissions
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Now you can run:

```
kubectl get nodes
```

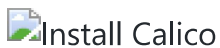


## Step-13:- Install a Network Plugin (Master Node)

Required for pods to communicate across nodes:

```
kubectl apply -f https://docs.projectcalico.org/manifests/calico.yaml
```

Wait 1–2 minutes for Calico to start.



## Step-14:- Join Worker Nodes

After `kubeadm init` on the master, you'll see a join command like:



```
kubeadm join 192.168.1.100:6443 --token abcdef.0123456789abcdef \
--discovery-token-ca-cert-hash sha256:...
```

Run **that exact command** on each worker node.



**Step-15:- Verify the Cluster (Master Node)**

```
kubectl get nodes
```

Expected output:

| NAME    | STATUS | ROLES         | AGE | VERSION |
|---------|--------|---------------|-----|---------|
| master  | Ready  | control-plane | 5m  | v1.29.0 |
| worker1 | Ready  | <none>        | 2m  | v1.29.0 |
| worker2 | Ready  | <none>        | 2m  | v1.29.0 |



**Command Reference**

| Goal               | Command                                      |
|--------------------|----------------------------------------------|
| Apply a YAML file  | kubectl apply -f file.yaml                   |
| See all pods       | kubectl get pods                             |
| See all services   | kubectl get svc                              |
| Scale a deployment | kubectl scale deployment <name> --replicas=N |
| Delete a pod       | kubectl delete pod <pod-name>                |
| See all nodes      | kubectl get nodes                            |
| Describe a pod     | kubectl describe pod <pod-name>              |
| View pod logs      | kubectl logs <pod-name>                      |